


MCAST

# Evaluating Multimodal Accessibility and Cognitive Load using AI: A Predictive and Educational Tool for Web Designers and Developers

*Mireya Francesca Vella*

*Supervisor: Paul Pulis*

**June - 2026**

**A dissertation submitted to the Institute of Information and Communication Technology in partial fulfilment of the requirements for the degree of BSc (Hons) Creative Computing**

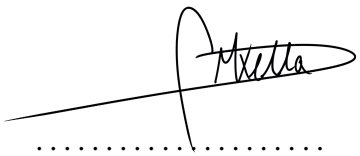
# Authorship Statement

This dissertation is based on the results of research carried out by myself, is my own composition, and has not been previously presented for any other certified or uncertified qualification.

The research was carried out under the supervision of Mr Paul Pulis – Evaluating Multimodal Accessibility and Cognitive Load using AI: A Predictive and Educational Tool for Web Designers and Developers, Mireya Francesca Vella

05/06/2026

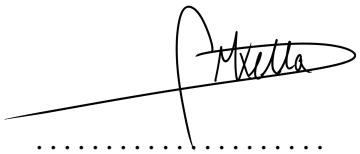
Date



Signature

# Copyright Statement

In submitting this dissertation to the MCAST Institute of Information and Communication Technology, I understand that I am giving permission for it to be made available for use in accordance with the regulations of MCAST and the Library and Learning Resource Centre. I accept that my dissertation may be made publicly available at MCAST's discretion.



05/06/2026 .....

Date

..... Signature

# Acknowledgements

The completion of this dissertation would not have been possible without the support, guidance, and encouragement of several individuals and organisations.

Firstly, I would like to express my sincere gratitude to my dissertation supervisor, Mr Paul Pulis, Deputy Director at MCAST, for his continuous guidance, support, and constructive feedback throughout the development of this research. His advice helped me refine my ideas, strengthen the direction of the study, and improve the overall quality of this dissertation.

I would also like to thank the MCAST lecturers who participated in the developer evaluation stage of this study. Their professional feedback, technical insight, and willingness to engage with the prototype were extremely valuable in assessing the usefulness, clarity, and relevance of the AI-assisted accessibility tool from a development perspective.

Special thanks are also extended to the Foundation for Information Technology Accessibility (FITA), the Commission for the Rights of Persons with Disability (CRPD), and the participants who took part in the accessibility evaluation. Their support with participant outreach, together with the participants' time, experiences, and feedback, was essential in helping me understand practical accessibility needs and evaluate the tool in a more meaningful and inclusive manner.

Finally, I would like to thank my fiancé for his constant patience and encouragement throughout this dissertation. His support during stressful moments helped me stay motivated and continue working towards completing this research.

# Abstract

European accessibility requirements, including the European Accessibility Act and EN 301 549, increasingly align digital products and services with WCAG-based accessibility expectations, yet accessibility barriers remain widespread. This dissertation presents the design, implementation, and evaluation of an AI Accessibility Assistant: a VS Code extension that analyses HTML, CSS, JavaScript, and TypeScript/React source files for WCAG 2.2 issues and provides actionable remediation guidance within the code-authoring environment.

The accessibility detection pipeline combines retrieval-augmented generation with a dual large language model configuration (kimi-k2.5 and qwen3.5:397b). A related NASA-TLX-informed workload estimation feature uses a single primary model to generate workload scores from detected accessibility barriers. A six-study evaluation was conducted across four experimental conditions. Under controlled benchmark aggregation the best configuration achieved F1 = 70.5%; under real-world single-run validation it achieved high precision (94.4%) with F1 = 0.399. Expert developers confirmed the majority of AI-flagged issues as genuine barriers, vision-impaired participants reported lower NASA-TLX workload scores on the AI-remediated website, and a NASA-TLX-informed workload estimation pipeline showed moderate directional alignment with participant-reported scores.

The findings support the feasibility of AI-assisted, source-level accessibility analysis as a complement to existing automated tools within a shift-left development workflow.

**Keywords:** web accessibility, WCAG 2.2, large language models, retrieval-augmented generation, VS Code extension, static source-code analysis, NASA-TLX workload estimation, European Accessibility Act, shift-left accessibility.

# Table of Contents

**Authorship Statement** i

**Copyright Statement** ii

**Acknowledgements** iii

**Abstract** iv

**List of Figures** xi

**List of Tables** xiv

**List of Abbreviations** xv

**Symbols** xvii

**1 Introduction** 1
- 1.1 Statement of the Problem . . . 1
- 1.2 Motivation of Research . . . 2
- 1.3 Research Context . . . 3
- 1.4 Research Positioning . . . 4
- 1.5 Detailed Aim . . . 4
- 1.6 Outline of the Research . . . 6

**2 Literature Review** 7
- 2.1 Overview . . . 7
- 2.2 Why Web Accessibility Fails in Practice . . . 8
  - 2.2.1 The Global Scale of Digital Exclusion . . . 9
  - 2.2.2 Legal Frameworks and Compliance Requirements . . . 10
- 2.3 Technical Foundations of Web Accessibility . . . 11
  - 2.3.1 WCAG 2.2: The Technical Standard . . . 12
  - 2.3.2 WAI-ARIA: Programmatic Accessibility for Dynamic Interfaces . . . 13
- 2.4 Automated Accessibility Evaluation . . . 14
  - 2.4.1 DOM-Based Rule Engines . . . 15
  - 2.4.2 Static Code Analysers and Developer-Facing Tools . . . 16
  - 2.4.3 Hybrid and Manual-Assisted Methods . . . 17
  - 2.4.4 Limitations of Automated Accessibility Evaluation . . . 18
- 2.5 Large Language Models for Accessibility Analysis . . . 19
  - 2.5.1 The Transformer Architecture and Foundation Models . . . 20
  - 2.5.2 Code-Aware LLMs and Accessibility Analysis . . . 21
  - 2.5.3 Hallucination and the Need for Knowledge Grounding . . . 22

2.5.3.1 Effect of Knowledge Grounding on LLM Reliability . . . 23
2.5.3.2 Prompt Sensitivity and Output Variability . . . 24
2.5.4 Limitations of LLM-Based Accessibility Analysis . . . 24
2.6 Improving LLM Reliability for Accessibility Analysis . . . 25
2.6.1 RAG as Knowledge Grounding . . . 25
2.6.2 Sparse, Dense, and Hybrid Retrieval . . . 25
2.6.3 Prompting, Reasoning, and Inference Controls . . . 26
2.6.4 Evaluating LLM Output Reliability . . . 27
2.7 Cognitive Load, NASA-TLX, and Evaluation Metrics . . . 28
2.7.1 Cognitive Load Theory . . . 28
2.7.2 The NASA Task Load Index . . . 29
2.7.3 Evaluation Metrics for Binary Classification Systems . . . 30
2.8 Research Synthesis: Identified Gaps . . . 31
2.9 Synthesis and Methodological Implications . . . 32

**3 Research Methodology 34**
3.1 Overview of Methodology . . . 34
3.2 Research Positioning and Research Onion . . . 35
3.2.1 Research Objectives, Questions, and Hypothesis . . . 36
3.2.2 High-Level System Architecture . . . 37
3.2.3 End-to-End Pipeline Workflow . . . 39
3.3 Scope, Limitations, and Reproducibility . . . 40
3.3.1 Research Philosophy and Design Strategy . . . 41
3.3.2 Domain Scope: EAA, WCAG 2.2, and File Type Coverage 42
3.3.3 Experimental Variables . . . 44
3.3.4 Boundary Conditions and Reproducibility Controls . . . 46
3.3.5 Experimental Design Structure and Workflow . . . 47
3.4 System Implementation . . . 49
3.4.1 Core Components . . . 49
3.4.2 Functional Behaviour . . . 50
3.4.3 Core Algorithm . . . 51
3.4.4 System Behaviour and Execution Conditions . . . 53
3.4.5 Reliability Testing . . . 53
3.5 Benchmarking and Validation Studies . . . 56
3.5.1 Study 1: Candidate Model Screening . . . 57
3.5.2 Study 2: Parametric Optimisation and Multi-File Evaluation 58
3.5.3 Study 3: Advanced Configuration and Ensemble Evaluation 59
3.5.4 Study 4: NASA-TLX-Informed Workload Estimation Benchmark . . . 60
3.5.5 Study 5: Real-World Expert and User Validation . . . 61
3.5.6 Study 6: Supplementary Comparison . . . 64
3.6 Evaluation Framework and Testing Matrix . . . 65
3.6.1 Benchmark Test File Suite . . . 66
3.6.2 Classification Metrics . . . 67
3.6.3 Composite Scoring and Acceptance Criteria . . . 68

3.6.4 NASA-TLX as Dual-Role Evaluation Instrument . . . . . 69
3.6.5 Statistical Analysis Plan . . . . . 70
3.7 Ethical Considerations . . . . . 70
3.8 Chapter Summary . . . . . 71

**4 Analysis of Results and Discussion 72**
4.1 Overview . . . . . 72
4.2 Evaluation Dataset and Reporting Approach . . . . . 72
4.3 Study 5: Expert Review and User Validation Results . . . . . 74
4.3.1 Expert Review Findings . . . . . 75
4.3.2 Detection and Re-run Results . . . . . 76
4.3.2.1 Per-Participant Detection Results . . . . . 77
4.3.2.2 Aggregate Detection Metrics by File Type . . . 78
4.3.2.3 Clean Test file Results . . . . . 80
4.3.2.4 Re-run Results . . . . . 81
4.3.2.5 Survey Responses and Qualitative Feedback . . 82
4.3.3 Participant NASA-TLX Results . . . . . 84
4.3.4 Predicted versus Reported Workload . . . . . 85
4.4 Study 6: Supplementary Tool Comparison Results . . . . . 89
4.4.1 Findings by File Type . . . . . 91
4.4.1.1 HTML . . . . . 92
4.4.1.2 CSS . . . . . 93
4.4.1.3 JavaScript . . . . . 95
4.4.1.4 TSX . . . . . 96
4.4.2 Aggregate Results . . . . . 97
4.4.3 Ground-Truth Recall Analysis . . . . . 100
4.4.4 Discussion of Tool Comparison . . . . . 103
4.4.4.1 Raw Counts Versus Ground-Truth Recall . . . . 103
4.4.4.2 AI Performance Compared to Prior Tool Studies 103
4.4.4.3 Deterministic Tool Performance in Context . . . 103
4.4.4.4 Practical Implications . . . . . 103
4.4.4.5 Threats to Validity . . . . . 104
4.5 Cross-Study Synthesis . . . . . 105
4.6 Answers to the Research Questions . . . . . 106
4.6.1 RQ1: Alignment Between AI-Derived and Participant-Reported NASA-TLX Workload Estimates . . . . . 106
4.6.2 RQ2: Effectiveness of AI-Assisted Source-Code Analysis for WCAG 2.2 Detection and Remediation . . . . . 107
4.6.3 RQ3: Actionability and Quality of Developer-Facing Accessibility Feedback . . . . . 108
4.7 Discussion in Relation to Existing Literature . . . . . 109
4.8 Threats to Validity and Result Limitations . . . . . 110
4.9 Chapter Summary . . . . . 111

**5 Conclusions and Recommendations** **112**
5.1 Overview . . . 112
5.2 Summary of the Research . . . 112
5.3 Final Answers to the Research Questions . . . 114
5.3.1 RQ1: Alignment Between the AI-Derived and Participant-Reported NASA-TLX Workload Estimate . . . 114
5.3.2 RQ2: Effectiveness of AI-Assisted Source-Code Analysis for WCAG 2.2 Detection and Remediation . . . 115
5.3.3 RQ3: Actionability and Quality of Developer-Facing Accessibility Feedback . . . 116
5.4 Overall Conclusion on the Hypothesis . . . 117
5.5 Contribution of the Study . . . 118
5.5.1 Technical Contribution . . . 118
5.5.2 Methodological Contribution . . . 119
5.5.3 Practical Contribution . . . 119
5.6 Limitations of the Study . . . 120
5.7 Recommendations . . . 122
5.7.1 Recommendations for Developers . . . 122
5.7.2 Recommendations for Accessibility Evaluators . . . 123
5.7.3 Recommendations for Future Researchers . . . 124
5.8 Future Work . . . 125
5.9 Final Closing Statement . . . 127

**List of References** **128**

**Appendix A Extended System Architecture Notes** **138**
A.1 Purpose of the Architecture . . . 138
A.2 Component Responsibilities . . . 138
A.3 Rationale for the Three-Layer Design . . . 139
A.4 Failure-Tolerant Behaviour . . . 140
A.5 Relationship to the Experimental Conditions . . . 140

**Appendix B Extended Domain Scope and Reproducibility Notes** **141**
B.1 Test-File Size Rationale . . . 141
B.2 Extended Error-Level Calibration Rationale . . . 141
B.3 Extended Boundary Conditions . . . 142
B.4 Deployment Notes . . . 143

**Appendix C Detailed System Implementation** **144**
C.1 VS Code Extension Implementation . . . 144
C.2 Baseline Accessibility Checks . . . 145
C.3 RAG Service and Knowledge Base Indexing . . . 146
C.4 Hybrid Retrieval and Caching . . . 147
C.5 Prompt Construction . . . 148
C.6 Response Handling and Deduplication . . . 149
C.7 Diagnostic Rendering . . . 151

**Appendix D Prototype Reliability Verification Notes** **152**

**Appendix E Supplementary Benchmarking and Validation Notes** **153**
E.1 Study 1 Supplementary Notes . . . . . 154
E.1.1 Study 1 Model Elimination Record . . . . . 155
E.1.2 Study 1 Terminal and Graph Evidence . . . . . 156
E.2 Study 2 Supplementary Notes . . . . . 158
E.2.1 Study 2 Model Elimination Record . . . . . 159
E.2.2 Study 2 Terminal and Graph Evidence . . . . . 160
E.3 Study 3 Supplementary Notes . . . . . 162
E.3.1 Study 3 Ensemble Configuration Comparison . . . . . 163
E.3.2 Study 3 Terminal and Graph Evidence . . . . . 164
E.4 Study 4 Supplementary Notes . . . . . 168
E.4.1 TLX Test 1 Ensemble Comparison Summary . . . . . 169
E.4.2 TLX Tests 1 to 3 Across Conditions . . . . . 169
E.4.3 Study 4 Terminal and Graph Evidence . . . . . 170
E.5 Study 5 Supporting Details . . . . . 172
E.6 Study 6 Supporting Notes . . . . . 173
E.6.1 Study 6 Discussion Detail . . . . . 175

**Appendix F Evaluation Metric and NASA-TLX Mapping Notes** **177**
F.1 Keyword-Match Ground-Truth Scoring . . . . . 177
F.2 Metric Interpretation . . . . . 178
F.3 Original NASA-TLX Rating Scale Definitions . . . . . 178
F.4 NASA-TLX Barrier Mapping . . . . . 180
F.5 Expert Review Coding . . . . . 181

**Appendix G Participant Materials and Ethical Safeguards** **182**
G.1 Participant Information Materials . . . . . 182
G.2 Consent Procedure . . . . . 189
G.3 Professional Developer Reviewer Consent Form . . . . . 189
G.4 Accessibility Participant Consent Form . . . . . 191
G.5 Accessibility of Research Materials . . . . . 192
G.6 User Testing Task Sheet . . . . . 193
G.7 NASA-TLX Questionnaire Used in Study 5 . . . . . 194
G.8 Data Protection Measures . . . . . 195
G.9 Risk Mitigation Measures . . . . . 196
G.10 Evidence of Participant Materials . . . . . 196

# List of Figures


| | | |
|---|---|---|
| 2.1 | Literature Map. | 8 |
| 2.3.1 | WCAG 2.2 Structure: Principles, Guidelines, Success Criteria, and Conformance Levels. | 12 |
| 2.4.2 | Traditional and Shift-Left Accessibility Evaluation in the Development Workflow. | 17 |
| 2.5.3.1 | Effect of Knowledge Grounding on LLM Reliability. | 23 |
| 2.6.1 | Retrieval-Augmented Generation Process. | 25 |
| 3.2 | Research Onion for the Study | 35 |
| 3.2.2 | High-Level System Architecture Diagram | 38 |
| 3.3.1 | 2×2 Factorial Experimental Design | 41 |
| 3.3.5 | Experimental Workflow | 48 |
| 3.4.1 | Schematic Diagram of the System Implementation Pipeline | 49 |
| 3.4.3 | Flowchart of the End-to-End Analysis Algorithm | 52 |
| 3.5.5 | Study 5 Real-World Validation Workflow | 63 |
| 3.5.6 | Supplementary Tool Comparison Process | 65 |
| 4.3.2a | Study 5: Detection Counts (left) and Precision, Recall, and F1 (right) by File Type | 79 |
| 4.3.2b | Study 5: Survey Response Distribution: Workflow Usefulness (left) and Quality Ratings (right) | 82 |
| 4.3.2c | Study 5: Per-Participant Issue Detection Rate Across Chosen File Types | 83 |
| 4.3.4a | Study 5: NASA-TLX AI Predicted vs Participant Reported Scores by Dimension and Condition | 86 |
| 4.3.4b | Study 5: Overall NASA-TLX Workload: AI Predicted vs Participant Mean | 86 |
| 4.3.4c | Study 5: AI-Generated NASA-TLX Output: Inaccessible Website (AI-estimated NASA-TLX workload: 60.8/100). Top: per-dimension workload analysis with reasoning; bottom: summary assessment table. | 87 |
| 4.3.4d | Study 5: AI-Generated NASA-TLX Output: Accessible Website (AI-estimated NASA-TLX workload: 13.3/100). Top: per-dimension workload analysis with reasoning; bottom: summary assessment table. | 88 |
| 4.4.2 | Study 6: AI Assistant Findings Breakdown per File Type. Each bar shows the composition of AI findings into GT-TP (planted ground-truth matches, green), Extra TP (genuine unplanted issues, blue), and FP (false positives with no code evidence, red). The total finding count per file is shown above each bar. | 99 |

| | | |
|---|---|---|
| 4.4.3 | Study 6: Ground-Truth Recall by Tool per File Type. Bars show the percentage of 30 planted defects recalled by each tool. Tools not applicable to a given file type are shown as 0% (N/A). The AI assistant provided the broadest cross-artefact coverage in this test setup; deterministic tools remained valuable within their intended operating scopes. . . . | 101 |
| E.1.2a | Study 1 Terminal-Based Model Screening Evidence . . . . . | 156 |
| E.1.2b | Study 1 Candidate Model Screening Results . . . . . | 156 |
| E.1.2c | Study 1 False-Positive and Instability Pattern Summary . . . . | 157 |
| E.2.2a | Study 2 Terminal-Based Multi-File Evaluation Evidence . . . | 160 |
| E.2.2b | Study 2 Multi-File Performance by Model . . . . . | 160 |
| E.2.2c | Study 2 Condition-Level Performance Across File Types . . . | 161 |
| E.2.2d | Study 2 Three-Run Aggregation Comparison . . . . . | 161 |
| E.3.2a | Study 3 Terminal-Based Ensemble Testing Evidence . . . . . | 164 |
| E.3.2b | Study 3 Dual-Model and Triple-Model F1 Comparison . . . . | 165 |
| E.3.2c | Study 3 Condition Variance Across Ensemble Configurations | 166 |
| E.3.2d | Study 3 False-Positive Comparison Across Ensemble Configurations . . . . . | 167 |
| E.4.3a | Study 4 Terminal-Based Computational NASA-TLX Testing Evidence . . . . . | 170 |
| E.4.3b | Study 4 Predicted TLX by Model Configuration . . . . . | 170 |
| E.4.3c | Study 4 Predicted TLX Across Experimental Conditions . . . | 171 |
| E.4.3d | Study 4 TLX Refinement Impact Across Test Runs . . . . . | 171 |
| F.3 | Original NASA-TLX Rating Scale Definitions . . . . . | 179 |
| G.1a | Participant Information Letter for Accessibility Participants, Page 1 . . . . . | 183 |
| G.1b | Participant Information Letter for Accessibility Participants, Page 2 . . . . . | 184 |
| G.1c | Participant Information Letter for Accessibility Participants, Page 3 . . . . . | 185 |
| G.1d | Participant Information Letter for Developer Reviewers, Page 1 | 186 |
| G.1e | Participant Information Letter for Developer Reviewers, Page 2 | 187 |
| G.1f | Participant Information Letter for Developer Reviewers, Page 3 | 188 |
| G.3 | Professional Developer Reviewer Consent Form . . . . . | 190 |
| G.4a | Accessibility Participant Consent Form, Page 1 . . . . . | 191 |
| G.6 | User Testing Task Sheet . . . . . | 193 |
| G.7 | Adapted NASA-TLX Questionnaire . . . . . | 194 |

# List of Tables


| | | |
|---|---|---|
| 2.2.1 | Summary of Accessibility Studies and Findings | 10 |
| 2.2.2 | Comparison of Accessibility Regulations | 11 |
| 2.3.1 | WCAG 2.2 Principles and Relevance to Source-Code Analysis | 13 |
| 2.3.2 | Overview of ARIA Roles, States, and Properties | 14 |
| 2.4.1 | DOM-Based Accessibility Evaluation Tools | 15 |
| 2.4.2 | Static Code Analysers and Developer-Facing Accessibility Tools | 16 |
| 2.4.4 | Limitations of Automated Accessibility Evaluation Tools | 19 |
| 2.5.1 | Evolution of Transformer-Based Language Models | 20 |
| 2.5.2 | LLM-Based Accessibility Studies Relevant to the Research | 22 |
| 2.5.4 | Summary of LLM Capabilities and Limitations in Accessibility Analysis | 24 |
| 2.6.2 | Comparison of Retrieval Approaches | 26 |
| 2.6.3 | Inference Sampling Parameters in LLM Output Control | 27 |
| 2.7.1 | Types of Cognitive Load | 28 |
| 2.7.2 | NASA-TLX Workload Dimensions | 29 |
| 2.7.3 | Comparison of Evaluation Metrics | 30 |
| 2.8 | Summary of Research Gaps Identified from the Literature | 31 |
| 3.2 | Research Onion Positioning | 35 |
| 3.2.1 | Research Questions | 36 |
| 3.2.2 | System Component Responsibilities | 37 |
| 3.2.3 | End-to-End Analysis Pipeline | 39 |
| 3.3.2a | WCAG 2.2 Success Criterion Groups in Scope | 43 |
| 3.3.2b | Test-File Error Level Calibration | 44 |
| 3.3.3a | Independent Variables | 44 |
| 3.3.3b | Dependent Variables | 45 |
| 3.3.3c | Control Variables | 45 |
| 3.3.4 | Default Inference Settings Used Where No Preset-Specific Override Applied | 47 |
| 3.4.2 | Inference Parameter Presets by Experimental Condition | 50 |
| 3.4.5 | Prototype Reliability Verification IPO Chart | 56 |
| 3.5.1 | Study 1 Model Screening Summary | 57 |
| 3.5.2 | Study 2 Evaluation Scope | 58 |
| 3.5.3 | Study 3 Configuration Summary | 59 |
| 3.5.4 | Study 4 NASA-TLX Benchmark Focus | 60 |
| 3.5.5a | Study 5 Validation Artefacts | 62 |
| 3.5.5b | Expert Developer Review Focus | 62 |
| 3.5.6 | Supplementary Tool Comparison Scope | 64 |
| 3.6.1 | Benchmark Test File Matrix | 67 |
| 4.3 | Study 5 Developer Participant Demographics | 74 |

| | | |
|---|---|---|
| 4.3.1 | Study 5 Expert Review: Task Observations on the Defective Website | 75 |
| 4.3.2a | Study 5: Per-Participant Detection Results by File Type | 77 |
| 4.3.2b | Study 5: Aggregate Detection Metrics by File Type (combined session: first run and re-run) | 78 |
| 4.3.2c | Study 5: Re-run Summary: Fixed Issue Re-appearances and New Detections | 81 |
| 4.3.2d | Study 5: Participant Survey Responses | 82 |
| 4.3.3 | Study 5: Participant-Reported NASA-TLX Scores by Condition and Subscale | 84 |
| 4.3.4a | Study 5: AI-Predicted vs Participant-Reported NASA-TLX Scores: Defective Website Condition | 85 |
| 4.3.4b | Study 5: AI-Predicted vs Participant-Reported NASA-TLX Scores: Improved Website Condition | 85 |
| 4.4.1a | Study 6: Raw Findings Per Test Script File (2 June 2026 run) | 91 |
| 4.4.1b | Study 6: Tool Applicability Matrix for Test Script Files | 92 |
| 4.4.1c | Study 6: HTML Test Script File Classification Metrics (30 GT defects) | 92 |
| 4.4.1d | Study 6: CSS Test Script File Classification Metrics (30 GT defects) | 94 |
| 4.4.1e | Study 6: JavaScript Test Script File Classification Metrics (30 GT defects) | 95 |
| 4.4.1f | Study 6: TSX Test Script File Classification Metrics (30 GT defects) | 97 |
| 4.4.2a | Study 6: Total Raw Findings by Tool Across All Test Script Files | 97 |
| 4.4.2b | Study 6: Full Classification Metrics Per Tool Per Test Script File (AI source-audited; other tools GT-TP only) | 98 |
| 4.4.2c | Study 6: AI Assistant Aggregate Metrics Across All Four Test Script Files (120 GT issues, source-audited) | 99 |
| 4.4.3a | Study 6: True Positive Counts (GT Matches) Out of 30 Planted Defects | 100 |
| 4.4.3b | Study 6: GT Recall Percentage Out of 30 Planted GT Defects | 100 |
| A.4 | Failure-Tolerant Behaviour | 140 |
| C.1 | Extension Commands and Panel Functionality | 144 |
| C.5 | Prompt Components | 148 |
| C.6 | Speculative-Language Filters | 150 |
| E.1 | Study 1 Prompt and RAG Refinements | 154 |
| E.1.1 | Study 1 Model Elimination Record | 155 |
| E.2a | Study 2 Language-Specific Sweep Coverage | 158 |
| E.2.1a | Study 2 Model Elimination Record | 159 |
| E.2.1b | Study 2 Final Shortlist Standing | 159 |
| E.3.1 | Study 3 Ensemble Configuration Comparison | 163 |
| E.4 | Study 4 TLX Refinements | 168 |

E.4.1 TLX Test 1 Ensemble Comparison Summary . . . . . 169
E.4.2 TLX Tests 1 to 3, Kimi-Qwen Voting Across Conditions . . 169
E.5a Study 5 Supporting Materials . . . . . 172
E.5b Study 5 User Task Types . . . . . 172
E.5c NASA-TLX Mapping Used in Study 5 . . . . . 173
E.6a Study 6 Issue Alignment Categories . . . . . 173
E.6b Supplementary Tool Comparison Scope . . . . . 174
E.6c Architectural Differences Between Tool Types . . . . . 174

F.2 Metric Interpretation Summary . . . . . 178
F.4 Barrier-to-NASA-TLX Mapping . . . . . 180
F.5 Expert Review Coding Scheme . . . . . 181

G.1 Participant Information Sheet Coverage . . . . . 182
G.6 User Testing Task Overview . . . . . 194
G.7 NASA-TLX Dimensions Used in the Participant Questionnaire 195
G.8 Data Protection Measures . . . . . 195
G.9 Risk Mitigation Measures . . . . . 196
G.10 Participant Material Evidence . . . . . 196

# List of Abbreviations

| | |
|---|---|
| **ACT** | Accessibility Conformance Testing |
| **AI** | Artificial Intelligence |
| **API** | Application Programming Interface |
| **APG** | ARIA Authoring Practices Guide |
| **ARIA** | Accessible Rich Internet Applications |
| **AT** | Assistive Technology |
| **BERT** | Bidirectional Encoder Representations from Transformers |
| **BM25** | Best Matching 25 |
| **CI** | Continuous Integration |
| **CNN** | Convolutional Neural Network |
| **COGA** | Cognitive Accessibility |
| **CSS** | Cascading Style Sheets |
| **DL** | Deep Learning |
| **DOM** | Document Object Model |
| **EAA** | European Accessibility Act |
| **EN** | European Norm |
| **EU** | European Union |
| **F1** | F1 Score |
| **FCN** | Fully Convolutional Network |
| **FN** | False Negative |
| **FP** | False Positive |
| **GDPR** | General Data Protection Regulation |
| **GPT** | Generative Pre-trained Transformer |
| **GT** | Ground Truth |
| **HTML** | HyperText Markup Language |
| **HTTP** | Hypertext Transfer Protocol |
| **ICT** | Information and Communication Technology |
| **IICT** | Institute of Information and Communication Technology |
| **IPO** | Input, Process, Output |
| **JSX** | JavaScript XML |
| **JS** | JavaScript |
| **kB** | Kilobyte |
| **LLM** | Large Language Model |
| **MCC** | Matthews Correlation Coefficient |

| | |
|---|---|
| **ML** | Machine Learning |
| **MiniLM** | MiniLM sentence-transformer model family |
| **NASA** | National Aeronautics and Space Administration |
| **NASA-TLX** | NASA Task Load Index |
| **NN** | Neural Network |
| **NLP** | Natural Language Processing |
| **RAG** | Retrieval-Augmented Generation |
| **regex** | Regular Expression |
| **RQ** | Research Question |
| **SC** | Success Criterion |
| **SCs** | Success Criteria |
| **TLX** | Task Load Index |
| **TN** | True Negative |
| **TP** | True Positive |
| **TS** | TypeScript |
| **TSX** | TypeScript XML |
| **UI** | User Interface |
| **UX** | User Experience |
| **VS Code** | Visual Studio Code |
| **W3C** | World Wide Web Consortium |
| **WAI** | Web Accessibility Initiative |
| **WAI-ARIA** | Web Accessibility Initiative – Accessible Rich Internet Applications |
| **WCAG** | Web Content Accessibility Guidelines |
| **WCAG-EM** | Web Content Accessibility Guidelines Evaluation Methodology |
| **WHO** | World Health Organization |

# Symbols

| | |
|---|---|
| TP | True Positive |
| FP | False Positive |
| FN | False Negative |
| TN | True Negative |
| $N$ | Sample size |
| $\times$ | Multiplication / cross-product (e.g. $2\times2$ factorial design) |
| $S$ | Normalised latency score |
| $t$ | Latency value (ms) |
| $t_{\min}$ | Minimum observed latency across conditions |
| $t_{\max}$ | Maximum observed latency across conditions |

# Chapter 1: Introduction

## 1.1 Statement of the Problem

Web accessibility concerns the extent to which digital interfaces can be perceived, understood, navigated, and operated by users with diverse abilities, assistive technologies, and interaction needs. In web development, accessibility is commonly evaluated using technical standards such as the Web Content Accessibility Guidelines (WCAG), which define success criteria for issues including alternative text, semantic structure, keyboard access, focus visibility, form labels, error handling, accessible names, and Accessible Rich Internet Applications (ARIA) relationships [1]. Within Europe, accessibility is also connected to legal and organisational obligations through the European Accessibility Act (EAA) and EN 301 549, which align many web accessibility expectations with WCAG-based Level A and AA conformance criteria [2, 3].

The problem addressed in this dissertation is that many accessibility issues are still identified too late in the development process, often after a website has already been implemented, deployed, or externally audited. Traditional accessibility tools such as Lighthouse, axe-core, Axe Linter, ESLint jsx-a11y, Webhint, and stylelint-a11y are useful because they detect many rule-based issues, but they operate within different technical boundaries [4–10]. Some analyse the rendered browser Document Object Model (DOM), while others inspect source code, Cascading Style Sheets (CSS), JavaScript XML (JSX)/TypeScript XML (TSX), or

configuration files. This can leave gaps where barriers require interpretation of source-code context, component behaviour, ARIA relationships, or developer intent, and developers working directly with source files have limited automated support for catching these issues during implementation. A further gap is that existing tools report violations as discrete checklist items with no indication of the cognitive workload likely experienced by assistive technology users, making it difficult to prioritise remediation by user impact.

## 1.2 Motivation of Research

This research is motivated by the real impact of accessibility failures on users. Missing labels, unclear structure, broken ARIA references, weak keyboard support, inaccessible form validation, and poor focus visibility can increase mental demand, effort, frustration, and task difficulty, especially for users relying on assistive technologies, keyboard navigation, screen readers, magnification, or simplified interface structures. These barriers are not merely standards violations; they correspond directly to difficulties experienced during interaction, as established in user research connecting WCAG violations to measurable user burden [11]. The research therefore investigates whether artificial intelligence (AI) can support developers by identifying accessibility issues earlier in the development cycle, explaining their likely user impact, and providing actionable remediation guidance directly within the coding environment.

The research is further motivated by the growing use of large language models (LLMs) in software development and the need to understand whether they can be

applied responsibly in accessibility contexts. Although LLMs can interpret code, produce explanations, and suggest fixes, they can also hallucinate, over-report, miss barriers, or generate unsupported recommendations. This creates an open research problem: AI may support accessibility work, but its reliability must be evaluated through controlled test files, measurable performance metrics, expert judgement, and user-focused validation before it can be recommended to practitioners.

## 1.3 Research Context

The research is situated within human-computer interaction, software engineering, and web accessibility. It focuses on a developer-facing tool designed to support "shift-left" accessibility practice, where issues are identified earlier in the development lifecycle before they become embedded in a completed interface.

The AI Accessibility Assistant is a Visual Studio Code (VS Code) extension supported by deterministic baseline checks, retrieval-augmented generation (RAG), LLM inference, structured issue parsing, and NASA Task Load Index (NASA-TLX)-based workload prediction. The system analyses HyperText Markup Language (HTML), CSS, JavaScript, and TypeScript/React source files as the main front-end artefacts where accessibility-relevant structure, styling, interaction logic, ARIA state management, component behaviour, and user-facing patterns are authored.

The research connects accessibility detection with workload prediction. Detected barriers are mapped to the six NASA-TLX dimensions [12, 13] as a computational estimate of user burden, allowing accessibility to be explored as a contributor to workload and interaction difficulty, not only compliance.

## 1.4 Research Positioning

This study adopts a post-positivist, primarily quantitative artefact-evaluation approach. It follows a deductive research strategy, testing whether RAG and reasoning-mode configurations improve developer-facing accessibility feedback through controlled benchmarking, expert review, participant validation, and supplementary tool comparison. The full Research Onion positioning, including the research philosophy, approach, strategy, methodological choices, time horizon, and techniques, is presented in Chapter 3 alongside the methodology.

## 1.5 Detailed Aim

The aim of this research is to design, implement, and evaluate an AI-assisted accessibility tool that can identify WCAG 2.2 accessibility issues in web source code, provide actionable developer-facing remediation guidance, and estimate the likely cognitive workload associated with detected accessibility barriers.

This study tests the following hypothesis:

> **H:** AI-based techniques that combine source-code accessibility analysis with NASA-TLX-informed workload estimation can provide developer-facing feedback that better reflects accessibility barriers and likely user burden than approaches focused only on rule-based accessibility checking.

The purpose was to test whether RAG-based, LLM-assisted accessibility analysis can improve developer-facing feedback by relating AI configuration to detection

performance and workload outcomes. The independent variables were retrieval augmentation (RAG-enabled or disabled), reasoning mode (think or no-think), and model configuration. The dependent variables were detection performance and workload prediction, measured through precision, recall, F1, Matthews Correlation Coefficient (MCC), latency, false positives, predicted and participant-reported NASA-TLX scores, and expert review outcomes.

Three research questions operationalise the aim:

- **RQ1:** To what extent does the accessibility-issue-derived NASA-TLX workload estimate align with, approximate, or correlate with participant-reported NASA-TLX workload scores?
- **RQ2:** How effectively can AI-assisted source-code analysis support the identification and remediation of WCAG 2.2 accessibility issues in HTML, CSS, JavaScript, and TypeScript/React files?
- **RQ3:** How effectively can a developer-facing AI assistant provide actionable accessibility feedback within a source-code authoring environment?

## 1.6 Outline of the Research

The remainder of this dissertation is structured as follows.

Chapter 2 reviews the literature: the scale of web accessibility failure, WCAG 2.2 and Web Accessibility Initiative, Accessible Rich Internet Applications (WAI-ARIA) foundations, existing automated tools, LLM-based code analysis, RAG, and the NASA-TLX framework, concluding with the identified research gaps.

Chapter 3 describes the methodology: system architecture, AI Accessibility Assistant implementation, and the six-study evaluation programme (Studies 1–3 for model optimisation, Study 4 for NASA-TLX benchmarking, Study 5 for real-world validation, Study 6 for tool comparison).

Chapter 4 presents the analysis of results, reporting Study 5 and 6 findings, answering each research question, and discussing results in relation to the literature and threats to validity.

Chapter 5 presents conclusions and recommendations, including RQ answers, the hypothesis verdict, contributions, limitations, and directions for future work.

# Chapter 2: Literature Review

## 2.1 Overview

This chapter reviews literature relevant to AI-assisted accessibility evaluation for web development. It is organised around five themes: accessibility standards and digital exclusion, automated evaluation tools, LLMs, RAG, and cognitive load. These themes connect the wider problem of inaccessible interfaces to the potential role of AI-assisted tools in accessibility evaluation.

The reviewed literature indicates that accessibility failures are not caused only by the absence of standards, but also by the difficulty of applying them during development. Figure 2.1 shows how the reviewed themes support accessibility detection, developer understanding, and cognitive-load-aware feedback.

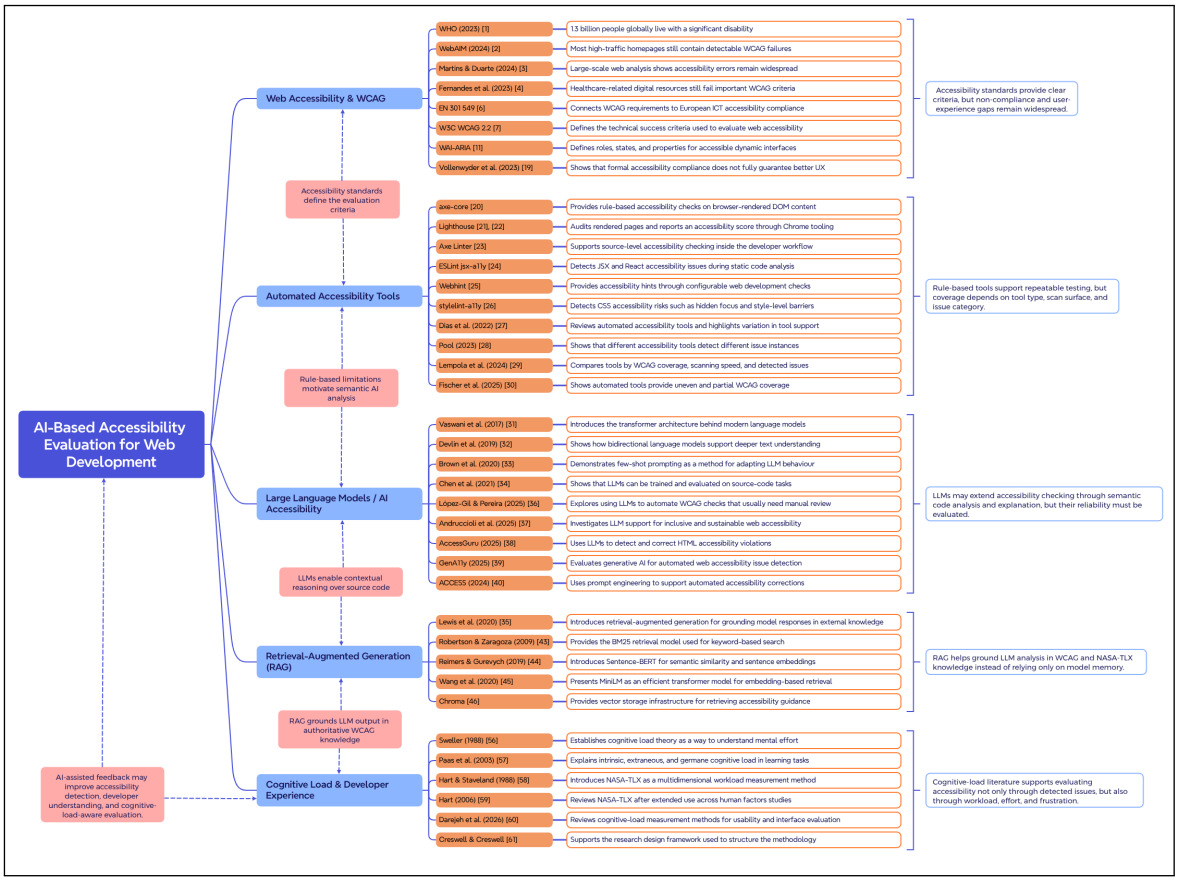



***Figure 2.1:*** *Literature Map.*

## 2.2 Why Web Accessibility Fails in Practice

Web accessibility remains a significant challenge because many digital interfaces continue to exclude users with disabilities. Although WCAG 2.2 and the EAA define clear accessibility expectations, empirical evidence shows that implementation remains inconsistent across real websites. For example, the WebAIM Million report found widespread accessibility failures across the top one million home pages, while large-scale academic analysis by Martins and Duarte also identified recurring accessibility errors across millions of web pages [14, 15]. This indicates that the existence of standards does not automatically lead to effective implementation in practice.

### *2.2.1 The Global Scale of Digital Exclusion*

The World Health Organisation (WHO) reports that approximately 1.3 billion people, or 16% of the global population, live with a significant disability [16]. Large-scale accessibility studies also show that barriers remain widespread. WebAIM found that 95.9% of the top one million homepages contained detectable WCAG failures [14], while Martins and Duarte [15] reported recurring accessibility errors across nearly three million web pages. Fernandes et al. [17] further showed that healthcare-related websites and resources frequently failed WCAG criteria, a finding that is particularly relevant given that inaccessible healthcare information can affect autonomy and informed decision-making.

Although these studies differ in scope and method, they show that accessibility failures remain common across general, large-scale, and high-impact digital environments. However, many findings are based on automatically detectable errors, meaning semantic and context-dependent issues may still be underrepresented. For example, a tool may detect whether an image has an alt attribute, but not whether the text meaningfully communicates the image's purpose.

| Study | Scope | Key Finding | Implication |
|---|---|---|---|
| **WHO (2023) [16]** | Global population | Approximately 1.3 billion people, or 16%, live with a significant disability | Accessibility is a global inclusion requirement |
| **WebAIM (2024) [14]** | Top one million homepages | 95.9% contained detectable WCAG failures | Accessibility failures remain widespread across high-traffic websites |
| **Martins and Duarte (2024) [15]** | Nearly three million web pages | Recurring accessibility errors were found at large scale | Accessibility barriers persist and are influenced by web technology adoption |
| **Fernandes et al. (2023) [17]** | Healthcare-related websites and resources | Websites and PDFs frequently failed WCAG criteria | Accessibility failures affect high-impact information domains such as healthcare |

***Table 2.2.1:*** *Summary of Accessibility Studies and Findings*

### *2.2.2 Legal Frameworks and Compliance Requirements*

Accessibility failures are also significant because digital accessibility is increasingly linked to legal compliance. In the European Union (EU), the EAA establishes accessibility requirements for selected products and services, including e-commerce, banking, transport, e-books, and consumer-facing digital services [2]. EN 301 549 defines accessibility requirements for Information and Communication Technology (ICT) products and services and connects European accessibility expectations to technical web accessibility requirements [3]. WCAG 2.2 then provides the success criteria used to evaluate many web accessibility barriers in practice [1].

For developers, these frameworks create a practical challenge: accessibility requirements must be translated into correct code, semantic structure, keyboard interaction, assistive technology support, and meaningful content relationships. When tools mainly detect rule-based failures, contextual issues may remain unchecked.

This limitation suggests that developer-facing tools with source-level accessibility feedback may address gaps that rule-based checkers leave unresolved.

| Framework | Role in This Dissertation | Relevance |
|---|---|---|
| **European Accessibility Act, Directive (EU) 2019/882 [2]** | Legal and regulatory motivation | Establishes accessibility obligations for selected digital products and services in the EU |
| **EN 301 549 [3]** | European ICT accessibility standard | Connects accessibility expectations to ICT products and services |
| **WCAG 2.2 [1]** | Technical evaluation standard | Provides the success criteria used to identify and classify web accessibility barriers |

***Table 2.2.2:*** *Comparison of Accessibility Regulations*

## 2.3 Technical Foundations of Web Accessibility

Web accessibility is evaluated through technical standards defining how digital content should be structured, presented, and interacted with. Two standards are particularly relevant: WCAG 2.2 and WAI-ARIA. EN 301 549, introduced in Section 2.2.2, provides the wider European ICT accessibility framework. Together, they provide the accessibility expectations used to evaluate source code and digital interfaces.

### *2.3.1 WCAG 2.2: The Technical Standard*

The Web Content Accessibility Guidelines 2.2 provide the main technical reference for evaluating web accessibility [1]. WCAG is organised around four principles: Perceivable, Operable, Understandable, and Robust, with testable success criteria across Levels A, AA, and AAA. WCAG 2.2 is especially relevant to modern accessibility evaluation because it includes criteria relating to focus appearance, dragging movements, target size, and accessible authentication.

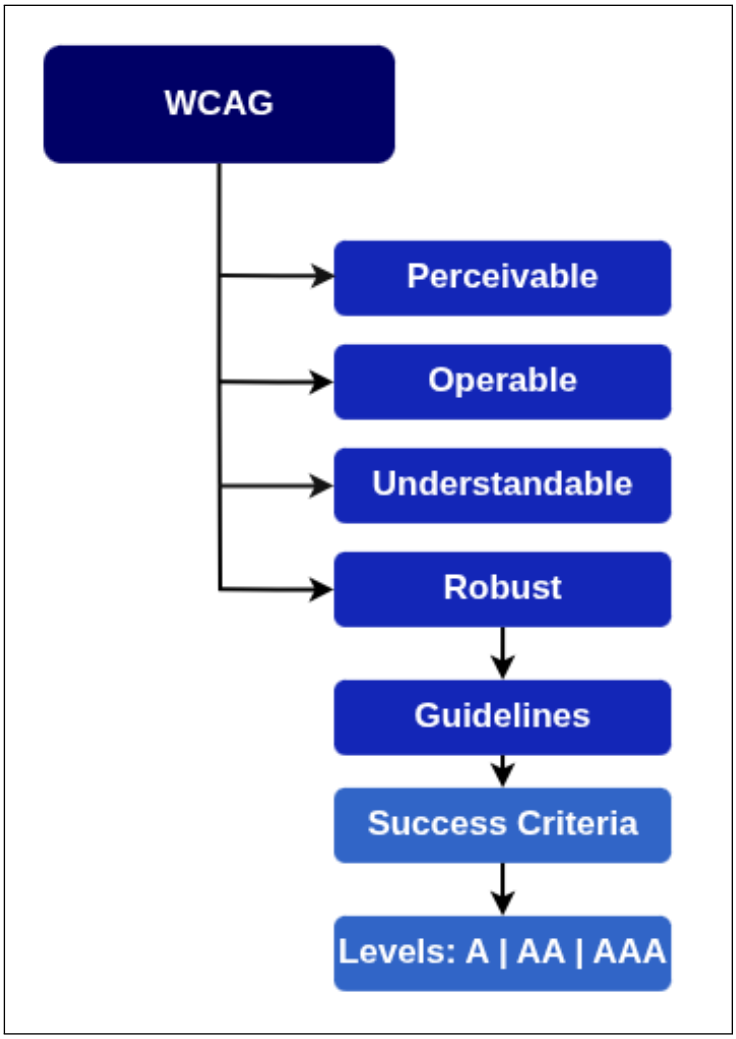



***Figure 2.3.1:*** *WCAG 2.2 Structure: Principles, Guidelines, Success Criteria, and Conformance Levels.*

A key challenge for automated evaluation is that some WCAG requirements are not purely syntactic. A tool may detect whether an image has an alt attribute, but not whether the text communicates the image's purpose. Similarly, a form control may exist without having meaningful labels, instructions, or programmatic relationships. World Wide Web Consortium (W3C) guidance on Success Criterion 1.3.1 and HTML form controls shows that accessibility depends on correct programmatic relationships, not just visual presentation [18], [19].

| **WCAG Principle** | **Meaning** | **Relevance to Source-Code Analysis** |
|---|---|---|
| **Perceivable** | Information must be available to users in ways they can perceive | Checks alternative text, contrast, labels, and content structure |
| **Operable** | Interface components must be usable through different input methods | Checks keyboard access, focus order, focus visibility, and interaction support |
| **Understandable** | Content and interface behaviour must be clear and predictable | Checks form instructions, error messages, link purpose, and consistency |
| **Robust** | Content must work reliably with assistive technologies | Checks semantic HTML, ARIA use, and valid programmatic relationships |

***Table 2.3.1:*** *WCAG 2.2 Principles and Relevance to Source-Code Analysis*

### *2.3.2 WAI-ARIA: Programmatic Accessibility for Dynamic Interfaces*

WAI-ARIA provides roles, states, and properties that communicate interface behaviour to assistive technologies [20]. It is especially relevant in JavaScript and component-based interfaces, where developers may create custom controls instead of using native HTML elements. The ARIA Authoring Practices Guide also defines expected interaction patterns for widgets such as disclosure components, tabs, dialogs, menus, and accordions [21].

ARIA can improve accessibility when used correctly, but incorrect ARIA can

also create barriers. For example, a custom button may visually expand a section, but if aria-expanded is not updated, screen reader users may receive incorrect state information. Similarly, aria-labelledby and aria-describedby only work if the referenced elements exist and provide meaningful information. These relationship-based requirements can be difficult for simple rule-based tools because the issue may depend on several connected parts of the code.

| **Type** | **Purpose** | **Example** |
|---|---|---|
| **Role** | Defines the type or purpose of an element | `role="button"` |
| **State** | Describes a dynamic condition | `aria-expanded="true"` |
| **Property** | Provides additional information or relationships | `aria-label`, `aria-describedby` |

***Table 2.3.2:*** *Overview of ARIA Roles, States, and Properties*

## 2.4 Automated Accessibility Evaluation

Automated accessibility evaluation tools identify recurring accessibility failures and reduce manual inspection. They differ in analysis surface: some inspect rendered DOM, some run browser audits, and others operate directly on source code. This distinction matters because tools that evaluate different representations of a webpage produce findings that should be interpreted as complementary rather than interchangeable.

### *2.4.1 DOM-Based Rule Engines*

DOM-based and browser-based accessibility tools inspect rendered page content and apply predefined rules to detectable patterns. Tools such as axe-core and Lighthouse can identify common issues such as missing alternative text, absent form labels, invalid ARIA attributes, missing accessible names, and insufficient colour contrast [4–6]. Their strength is that they provide fast and repeatable checks on browser-rendered output, making them useful for testing pages after they have been rendered.

However, these tools are limited to issues visible in the inspected page state. They may miss inactive menus, untriggered dialogs, dynamic states, or source-code relationships absent from the rendered output. This suggests that source-code-level accessibility evaluation may surface issues before they become detectable through rendered-page testing.

| **Tool** | **Category** | **Typical Workflow** | **Common Use** |
|---|---|---|---|
| **axe-core [6]** | DOM-based rule engine | Browser, automated testing, Continuous Integration (CI) integration | Automated WCAG-related checks on rendered content |
| **Lighthouse [4, 5]** | Browser audit tool | Chrome DevTools audit | Accessibility scoring and page-level audit reporting |

***Table 2.4.1:*** *DOM-Based Accessibility Evaluation Tools*

### *2.4.2 Static Code Analysers and Developer-Facing Tools*

Static accessibility tools evaluate source code before the interface is executed or deployed. This is useful in developer workflows because accessibility feedback can be provided while HTML, CSS, JavaScript, or component-based code is being written. Source-level tools are particularly relevant to editor-integrated workflows, since development environments such as VS Code support tooling, diagnostics, and developer-facing feedback directly within the coding environment [22].

Representative source-level tools include Axe Linter, ESLint jsx-a11y, Webhint, and stylelint-a11y [7–10]. These tools can detect issues closer to the point of implementation, such as JSX accessibility problems, CSS focus issues, and configuration-related accessibility concerns. However, static tools cannot fully evaluate the rendered DOM or runtime behaviour unless the relevant evidence exists in the source file.

| **Tool** | **Category** | **Typical Workflow** | **Common Use** |
|---|---|---|---|
| **Axe Linter [7]** | Source-level accessibility linter | Editor or development workflow | Detecting accessibility issues while coding |
| **ESLint jsx-a11y [8]** | JSX accessibility linter | React/JSX static analysis | Detecting accessibility issues in JSX and TSX components |
| **Webhint [9]** | Configurable web hinting tool | Development and audit configuration | Reporting accessibility and web quality hints |
| **stylelint-a11y [10]** | CSS accessibility linter | CSS/static style analysis | Detecting style-level accessibility risks |

***Table 2.4.2:*** *Static Code Analysers and Developer-Facing Accessibility Tools*

Figure 2.4.2 shows how developer-facing accessibility feedback can identify issues during coding rather than after implementation, reducing delayed fixes and context switching.

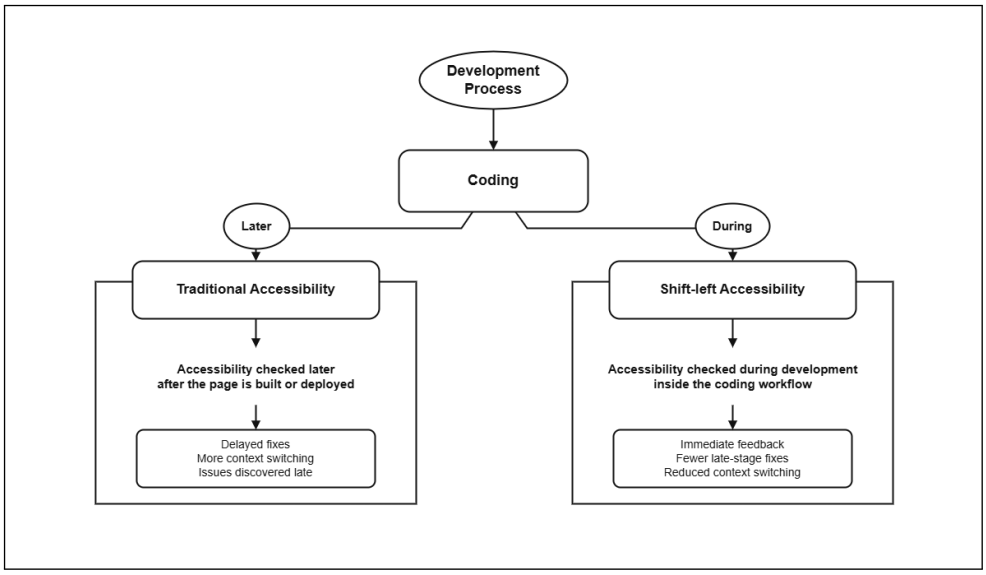



***Figure 2.4.2:*** *Traditional and Shift-Left Accessibility Evaluation in the Development Workflow.*

### *2.4.3 Hybrid and Manual-Assisted Methods*

Hybrid and manual-assisted methods combine automated output with human judgement. This is necessary because issues such as alternative-text quality or heading accuracy require interpretation rather than pattern matching.

The W3C Accessibility Conformance Testing (ACT) Rules Format supports more consistent accessibility test definitions [23], while the Website Accessibility Conformance Evaluation Methodology (WCAG-EM) provides guidance for structured accessibility evaluation [24]. Systematic and comparative studies show that automated accessibility tools vary in coverage, platform support, and consistency [25–28]. Therefore, automated tools are valuable for scalable checks, but do not remove the need for expert judgement, especially where accessibility depends on meaning, context, or interaction behaviour.

### *2.4.4 Limitations of Automated Accessibility Evaluation*

Although automated tools are useful, their limitations remain central to accessibility evaluation literature. Rule-based tools are strongest when issues are detectable from explicit code or DOM evidence, such as missing labels, missing accessible names, invalid ARIA attributes, or insufficient contrast. They are weaker when accessibility depends on semantic meaning, cross-file relationships, dynamic state changes, or the quality of user-facing content.

Comparative studies support this limitation. Abascal et al. [29] provided a broad review of web accessibility evaluation tools, documenting that no single tool comprehensively covers all WCAG requirements and that tools differ substantially in the types of issues they detect. Pool [26] showed that accessibility testing tools can detect different issue instances, meaning tool outputs are often complementary. Lempola et al. [27] also found that tools differ in WCAG coverage, scanning speed, and number of detected issues. Similarly, Fischer et al. [28] showed that automated tools provide partial and uneven WCAG coverage. These findings suggest that tools may be more meaningfully compared by applicability, issue type, and ground-truth recall than by raw finding counts alone.

| Tool / Method | Main Limitation |
|---|---|
| **axe-core [6]** | Limited to rule-based checks on detectable rendered DOM patterns |
| **Lighthouse [4, 5]** | Provides useful audits and scores, but does not represent full WCAG conformance |
| **Axe Linter [7]** | Useful for source-level checks, but cannot fully evaluate rendered runtime behaviour |
| **ESLint jsx-a11y [8]** | Strong for JSX/React accessibility patterns, but limited outside JSX/TSX contexts |
| **Webhint [9]** | Provides configurable hints, but coverage depends on enabled rules and analysed context |
| **stylelint-a11y [10]** | Useful for CSS accessibility risks, but limited to style-level evidence |
| **Dynamic runtime scanning** | May miss interaction states that are not triggered during testing |
| **Hybrid/manual-assisted review** | More accurate for context, but requires time and evaluator expertise |

***Table 2.4.4:*** *Limitations of Automated Accessibility Evaluation Tools*

## 2.5 Large Language Models for Accessibility Analysis

LLMs may extend accessibility evaluation beyond fixed rule matching. Unlike traditional tools, LLMs can process natural language and source code, identify relationships between elements, and generate explanatory remediation guidance. However, their findings must be evaluated critically because accessibility feedback needs to be accurate, evidence-based, and useful within developer workflows.

### *2.5.1 The Transformer Architecture and Foundation Models*

The technical basis for modern LLMs can be traced to the transformer architecture introduced by Vaswani et al. [30]. Transformers use attention mechanisms to model relationships across token sequences, making them suitable for language understanding and generation tasks. Later models extended this architecture in different directions. Bidirectional Encoder Representations from Transformers (BERT) demonstrated bidirectional language understanding [31], while Generative Pre-trained Transformer (GPT)-3 showed that large models could perform tasks through prompting and few-shot learning [32]. Codex then applied LLM capabilities to source-code generation and interpretation, showing that models could respond to programming tasks using natural-language instructions [33].

These developments establish the foundation for LLM-based code intelligence, but accessibility auditing also requires interpretation of WCAG criteria, semantic HTML, ARIA relationships, interaction states, and likely user impact. This suggests that LLM-based accessibility evaluation is better treated as an empirical question than as an assumed capability.

| **Model / Study** | **Contribution** | **Relevance to Accessibility Analysis** |
|---|---|---|
| **Transformer / Attention Is All You Need [30]** | Introduced attention-based sequence modelling | Provides the architecture foundation for modern LLMs |
| **BERT [31]** | Demonstrated bidirectional transformer-based language understanding | Shows the value of transformer-based language comprehension |
| **GPT-3 [32]** | Demonstrated few-shot and prompt-based task performance | Supports the use of prompt-based task execution |
| **Codex [33]** | Applied LLM capabilities to programming tasks | Supports the relevance of LLMs for source-code analysis |

***Table 2.5.1:*** *Evolution of Transformer-Based Language Models*

### *2.5.2 Code-Aware LLMs and Accessibility Analysis*

Code-aware LLMs may support accessibility work by analysing relationships that are difficult to evaluate through isolated rule checks. For example, an accessibility issue may involve a custom button, an ARIA attribute, a controlled region, and a JavaScript state update. A rule-based checker may detect whether an attribute exists, while an LLM may be able to assess whether the relationship is meaningful and updated correctly.

Recent studies show growing interest in using LLMs for accessibility evaluation and remediation. López-Gil and Pereira [34] explored whether LLMs can help automate WCAG success criteria that normally require manual review. Andruccioli et al. [35] investigated LLM support for inclusive web accessibility, while AccessGuru [36] used LLMs to detect and correct HTML accessibility violations. GenA11y [37] further evaluated generative AI for automated web accessibility issue detection, ACCESS [38] explored prompt engineering for automated accessibility corrections, and Paternò et al. [39] examined how LLMs can improve automatic web accessibility validation. These studies position LLMs as support tools for accessibility detection, explanation, and remediation rather than replacements for WCAG, automated tools, or expert judgement.

The literature on ensemble methods suggests that cross-validation between independent model outputs can reduce false positives. Where findings are reported by only one model, they may reflect model-specific hallucination rather than genuine accessibility barriers, and retaining only issues on which independent models agree may therefore raise the confidence threshold for each reported finding.

| Study | Focus | Relevance to This Dissertation |
|---|---|---|
| **López-Gil and Pereira [34]** | LLMs for manual WCAG criteria | Supports the use of LLMs for context-dependent accessibility checks |
| **Andruccioli et al. [35]** | LLMs for inclusive web accessibility | Shows the relevance of LLMs in developer-facing accessibility support |
| **AccessGuru [36]** | Detecting and correcting HTML accessibility violations | Closely relates to AI-assisted accessibility remediation |
| **GenA11y [37]** | Generative AI for accessibility issue detection | Provides a comparison point for AI-based detection performance |
| **ACCESS [38]** | Prompt engineering for accessibility corrections | Supports the role of structured prompting in remediation guidance |
| **Paternò et al. [39]** | LLM improvement of automatic accessibility validation | Supports the use of LLMs as a validation layer in automated accessibility workflows |

***Table 2.5.2:*** *LLM-Based Accessibility Studies Relevant to the Research*

### *2.5.3 Hallucination and the Need for Knowledge Grounding*

Although LLMs can support contextual code analysis, they generate probabilistic outputs rather than deterministic rule results. Their responses may vary depending on prompt wording, model configuration, retrieved context, and inference settings [32, 33]. In accessibility analysis, this creates risks such as false-positive violations, incorrect WCAG references, fabricated line numbers, or remediation advice that does not match the analysed code. Such variability can mislead developers or create false confidence in inaccessible code.

### *2.5.3.1 Effect of Knowledge Grounding on LLM Reliability*

Retrieval-augmented generation addresses this limitation by supplying relevant external documentation during inference [40]. Supplying domain-specific documentation during inference, rather than relying solely on internal model knowledge, may improve the relevance of generated findings and reduce unsupported recommendations. Issuing multiple targeted queries per file type, rather than a single generic query, may also help surface relevant guidance for different categories of accessibility issues that can co-occur in the same source file, such as image alternatives, keyboard interaction, form labelling, and ARIA reference correctness.

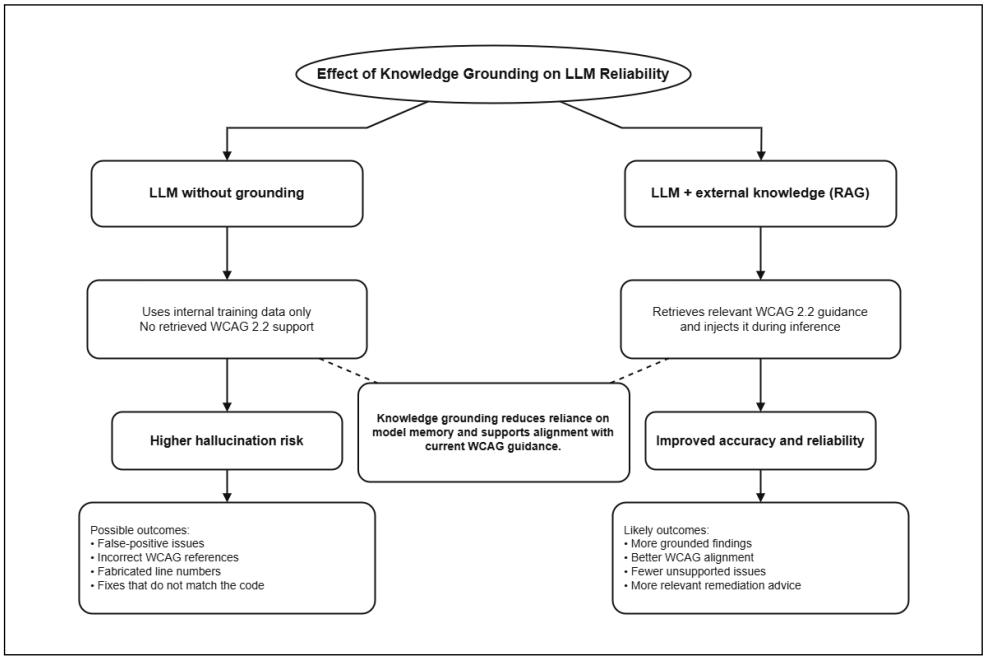



***Figure 2.5.3.1:*** *Effect of Knowledge Grounding on LLM Reliability.*

For this reason, knowledge grounding is best treated as a variable condition when evaluating LLM-based accessibility tools. Comparing retrieval-enabled and non-retrieval conditions offers a means of assessing whether retrieved accessibility guidance improves detection reliability and reduces unsupported findings.

#### *2.5.3.2 Prompt Sensitivity and Output Variability*

LLM outputs vary with prompt wording, task framing, and inference parameters [32, 33], which can affect trust and repeatability in developer-facing tools. Controlled prompts, structured output formats, and consistent metrics may therefore improve reliability.

### *2.5.4 Limitations of LLM-Based Accessibility Analysis*

Overall, the literature shows that LLMs may support accessibility detection and remediation, but their outputs remain dependent on prompt design, retrieved context, model configuration, and source-code evidence [30–38, 40]. Table 2.5.4 summarises the main capabilities and reliability risks discussed in this section.

| Section | LLM Capability / Issue | Potential Benefit | Limitation / Reliability Risk |
|---|---|---|---|
| **2.5.1** | Transformer-based language modelling | Supports natural-language and code understanding | Foundation models are not accessibility-specific by default |
| **2.5.2** | Code-aware analysis | Can inspect relationships across elements, attributes, and logic | May misunderstand framework-specific behaviour or cross-file dependencies |
| **2.5.2** | Explanatory feedback | Can provide developer-friendly explanations and fixes | May produce vague, incomplete, or unsuitable remediation advice |
| **2.5.3** | Probabilistic generation | Allows flexible responses to complex code contexts | Output may vary between runs or configurations |
| **2.5.3.1** | Knowledge grounding through RAG | Can supply relevant WCAG documentation during analysis | Retrieval may still provide incomplete, irrelevant, or noisy context |
| **2.5.3.2** | Prompt-based task performance | Can adapt to structured accessibility auditing instructions | Results may change depending on prompt wording, task framing, and inference settings |

***Table 2.5.4:*** *Summary of LLM Capabilities and Limitations in Accessibility Analysis*

## 2.6 Improving LLM Reliability for Accessibility Analysis

The literature suggests that LLM-based accessibility outputs may benefit from reliability controls before they can be usefully applied in developer-facing workflows.

### *2.6.1 RAG as Knowledge Grounding*

RAG combines language generation with external document retrieval. Lewis et al. [40] introduced RAG for knowledge-intensive natural language processing (NLP) tasks, which is relevant here because WCAG and ARIA guidance is detailed, technical, and periodically updated. However, RAG remains dependent on retrieval quality, as incomplete or noisy context can still lead to unsupported conclusions. Vector databases such as Chroma can provide the search infrastructure needed to support this kind of retrieval workflow [41].

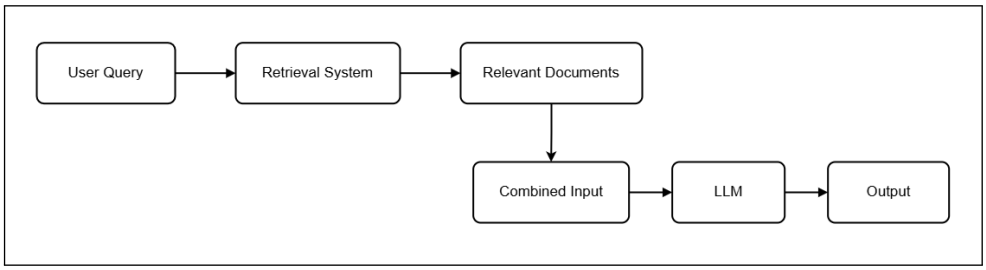



***Figure 2.6.1:*** *Retrieval-Augmented Generation Process.*

### *2.6.2 Sparse, Dense, and Hybrid Retrieval*

Retrieval methods can be sparse, dense, or hybrid. Best Matching 25 (BM25) supports exact keyword matching, which is useful for technical identifiers such as `aria-expanded`, `aria-labelledby`, and WCAG success criterion numbers [42]. Dense retrieval uses embeddings to identify semantic similarity, with

Sentence-BERT and MiniLM providing efficient approaches for sentence-level and compact embedding-based retrieval [43, 44]. Hybrid retrieval may be well-suited to accessibility documentation because WCAG and ARIA guidance contain both precise technical terms and broader conceptual explanations.

| Method | Strength | Limitation |
|---|---|---|
| **BM25 [42]** | Strong exact-term matching | Limited semantic understanding |
| **Dense retrieval [43, 44]** | Strong semantic similarity | May miss exact identifiers |
| **Hybrid retrieval** | Combines exact and semantic retrieval | More complex to implement |

***Table 2.6.2:*** *Comparison of Retrieval Approaches*

### *2.6.3 Prompting, Reasoning, and Inference Controls*

Prompt engineering is relevant because LLM performance depends on task framing, context, and output constraints [32, 33]. In accessibility analysis, structured prompts can reduce vague output by defining evidence boundaries, WCAG linking, user impact, remediation format, and expected specificity. Chain-of-thought, zero-shot reasoning, and self-consistency have been shown to support complex reasoning tasks [45–47], but reasoning prompts may need to be constrained to reduce verbosity and unsupported speculation.

Inference parameters also influence how deterministic or variable LLM outputs are. For structured evaluation tasks without extended reasoning, consistency is more important than creative variation, and lower temperature values can support more stable output [48]. However, models configured in chain-of-thought or extended reasoning mode may require a moderate temperature to allow adequate exploration during the internal reasoning trace before producing structured output. In such

cases, model providers may recommend a higher temperature than would typically be used for non-reasoning structured tasks. Top-p, top-k, and repetition-related settings further help control token selection and reduce repeated text [49, 50].

Table 2.6.3 summarises the main inference sampling parameters used to control LLM output behaviour and reliability.

| **Parameter** | **Purpose** | **Reliability Relevance** |
|---|---|---|
| **Temperature** | Controls randomness of output | Lower values can improve consistency in structured tasks |
| **Top-p** | Limits sampling to the most probable token set | Helps control open-ended generation |
| **Top-k** | Restricts token selection to the top-k options | Reduces unlikely token choices |
| **Repetition penalty** | Reduces repeated output | Helps prevent repetitive or degenerated responses |

***Table 2.6.3:*** *Inference Sampling Parameters in LLM Output Control*

### *2.6.4 Evaluating LLM Output Reliability*

LLM outputs can appear convincing even when incorrect, suggesting that reliability is better evaluated against an independent reference than judged by fluency alone. In accessibility analysis, this would mean comparing generated findings against known or expert-annotated issues. Precision, recall, F1 score, and MCC are relevant because they distinguish between correct detections, missed issues, and unsupported findings. Chicco and Jurman [51] argue that MCC is especially informative in imbalanced classification because it considers true positives, true negatives, false positives, and false negatives.

## 2.7 Cognitive Load, NASA-TLX, and Evaluation Metrics

### *2.7.1 Cognitive Load Theory*

Cognitive load theory explains how limited working memory affects task performance. Sweller [52] argues that excessive cognitive demand can reduce learning and task effectiveness, while Paas et al. [53] distinguish between intrinsic, extraneous, and germane cognitive load. This distinction is relevant to accessibility work because developers must simultaneously interpret standards, inspect code, understand tool output, and decide on appropriate fixes. Table 2.7.1 summarises the three types of cognitive load and their relevance to accessibility tooling.

| **Load Type** | **Description** | **Relevance** |
|---|---|---|
| **Intrinsic** | Complexity of the task itself | Accessibility standards and ARIA patterns are inherently complex |
| **Extraneous** | Load caused by poor presentation or workflow | Tool switching, unclear reports, and vague guidance can increase effort |
| **Germane** | Effort used for learning and understanding | Clear feedback can help developers build accessibility knowledge |

***Table 2.7.1:*** *Types of Cognitive Load*

### *2.7.2 The NASA Task Load Index*

NASA-TLX was developed by Hart and Staveland [12] to measure subjective workload. It evaluates six dimensions: Mental Demand, Physical Demand, Temporal Demand, Performance, Effort, and Frustration. Hart [13] later noted that the raw, unweighted TLX score remains practical for evaluating workload across different task contexts. Darejeh et al. [54] further support the relevance of cognitive-load measurement methods in usability and interface evaluation. Table 2.7.2 summarises the six NASA-TLX workload dimensions.

| **Dimension** | **Description** |
|---|---|
| **Mental Demand** | Cognitive effort required to complete the task |
| **Physical Demand** | Physical effort required, including interaction effort |
| **Temporal Demand** | Time pressure experienced during the task |
| **Performance** | Perceived success in completing the task |
| **Effort** | Overall effort required to complete the task |
| **Frustration** | Stress, irritation, or discouragement experienced |

***Table 2.7.2:*** *NASA-TLX Workload Dimensions*

The literature therefore suggests examining accessibility tooling not only for detection accuracy, but also for the workload it creates or reduces. Browser-based tools may require switching between editors, audit reports, documentation, and code fixes, while clearer developer-facing feedback may reduce extraneous load.

NASA-TLX is relevant to accessibility evaluation because barriers such as unclear labels, poor focus visibility, confusing navigation, or inaccessible dynamic content may increase mental demand, effort, and frustration.

### *2.7.3 Evaluation Metrics for Binary Classification Systems*

Accessibility detection can also be evaluated as a binary classification task: a system either detects a known violation or it does not. However, class imbalance can make accuracy misleading because non-violations may outnumber violations. Chicco and Jurman [51] argue that MCC is more informative than accuracy and F1 in imbalanced binary classification because it considers true positives, true negatives, false positives, and false negatives. Table 2.7.3 compares F1 score and MCC as evaluation metrics for accessibility detection.

| **Metric** | **Strength** | **Limitation** |
|---|---|---|
| **F1 Score** | Balances precision and recall | Does not include true negatives |
| **MCC** | Accounts for all confusion-matrix outcomes | More complex to interpret |

***Table 2.7.3:*** *Comparison of Evaluation Metrics*

F1 score captures the balance between missed violations and false positives, while MCC provides a broader view of classification performance under uneven class distributions.

## 2.8 Research Synthesis: Identified Gaps

The reviewed literature shows that web accessibility remains problematic despite mature standards, legal requirements, and automated tools. Rule-based tools support repeatable checks, but remain limited when accessibility depends on semantic meaning, contextual relationships, dynamic behaviour, or workload-aware evaluation [25–29]. Recent LLM-based studies suggest that AI can support detection and remediation, but reliability, grounding, and evaluation against known defects remain important concerns [34–39].

Five key research gaps were identified from the literature. Table 2.8 summarises these gaps and their implications for AI-assisted accessibility research.

| Gap | Description | Literature-Based Research Need |
|---|---|---|
| **Gap 1: Semantic detection ceiling** | Rule-based tools struggle to evaluate meaning, context, and dynamic state relationships | Further investigation is needed into source-level methods for identifying semantic WCAG 2.2 issues |
| **Gap 2: Limited developer-facing accessibility support** | Existing tools often focus on browser-based or rule-based checking rather than contextual source-level guidance | More accessibility support is needed within code-authoring workflows, especially where developers need explanations and fixes |
| **Gap 3: Limited RAG use in accessibility analysis** | RAG has been used for knowledge grounding in NLP, but its use for WCAG-grounded source-code accessibility analysis remains underdeveloped | Further work is needed to examine whether retrieved WCAG guidance can improve LLM grounding and detection reliability |
| **Gap 4: Limited comparative benchmarking of AI accessibility tools** | Existing LLM accessibility studies show promise, but benchmarking across file types, tool categories, and ground-truth defects remains limited | Accessibility-specific LLM evaluation needs clearer comparison against established automated tools |
| **Gap 5: Limited cognitive-load evidence** | Accessibility tools are rarely evaluated in relation to user or developer workload | Accessibility evaluation research needs stronger links with workload frameworks such as NASA-TLX |

***Table 2.8:*** *Summary of Research Gaps Identified from the Literature*

The reviewed literature suggests that existing accessibility evaluation methods leave gaps in addressing semantic, contextual, and dynamic barriers within developer workflows. These gaps point to the need for evaluation approaches that combine detection performance, tool comparison, and workload-related evidence.

## 2.9 Synthesis and Methodological Implications

This chapter reviewed the literature on web accessibility, automated evaluation tools, LLM-based code intelligence, RAG, prompt engineering, and cognitive load. The review showed that accessibility barriers remain widespread despite the existence of established standards, legal obligations, and supporting guidance. It also showed that automated accessibility tools are useful for repeatable rule-based detection, but remain limited where accessibility depends on semantic meaning, source-code context, dynamic behaviour, component-level relationships, or likely user workload.

These findings carry several methodological implications for evaluating AI-assisted accessibility tools. The limitations of established automated tools suggest that any AI-based approach should be assessed alongside those tools rather than in isolation, so that differences in detection coverage can be understood. The literature on LLMs, RAG, and inference controls indicates that comparing retrieval-enabled and non-retrieval conditions, as well as different reasoning configurations, would provide evidence about the contribution of knowledge grounding. The evidence on inconsistent automated tool coverage further suggests that controlled benchmark test files with known ground-truth issues would support consistent measurement of precision, recall, F1 score, MCC, latency, and false positives.

The literature on cognitive load and NASA-TLX indicates that workload-related measures could complement technical performance evidence, since accessibility barriers may contribute not only to rule violations but also to user effort and task difficulty.

Taken together, the reviewed literature identifies gaps in current accessibility evaluation practice that span semantic detection, developer-facing tooling, knowledge grounding, comparative benchmarking, and cognitive load measurement. These represent areas where further empirical investigation may extend the current evidence base and inform the design of more effective developer-facing accessibility tools.

# Chapter 3: Research Methodology

## 3.1 Overview of Methodology

This chapter presents the methodology used to design, implement, and evaluate the AI Accessibility Assistant, a VS Code extension that analyses HTML, CSS, JavaScript, and TypeScript/React source files. The study adopts a post-positivist, primarily quantitative artefact-evaluation approach, examining whether a developer-facing AI tool can identify accessibility issues in source code and provide actionable remediation guidance within the code-authoring environment.

The artefact follows a shift-left approach, integrating feedback into the development workflow. Performance was assessed through precision, recall, F1, MCC, and latency on benchmark test files containing intentionally planted defects. Where applicable, detected barriers were mapped to NASA-TLX dimensions as a workload proxy, not a direct cognitive-load measurement. Further architectural detail is in Appendix A.

## 3.2 Research Positioning and Research Onion

Following Saunders, Lewis, and Thornhill's Research Onion framework, Table 3.2 summarises the methodological positioning of this study across each layer, illustrated by Figure 3.2.

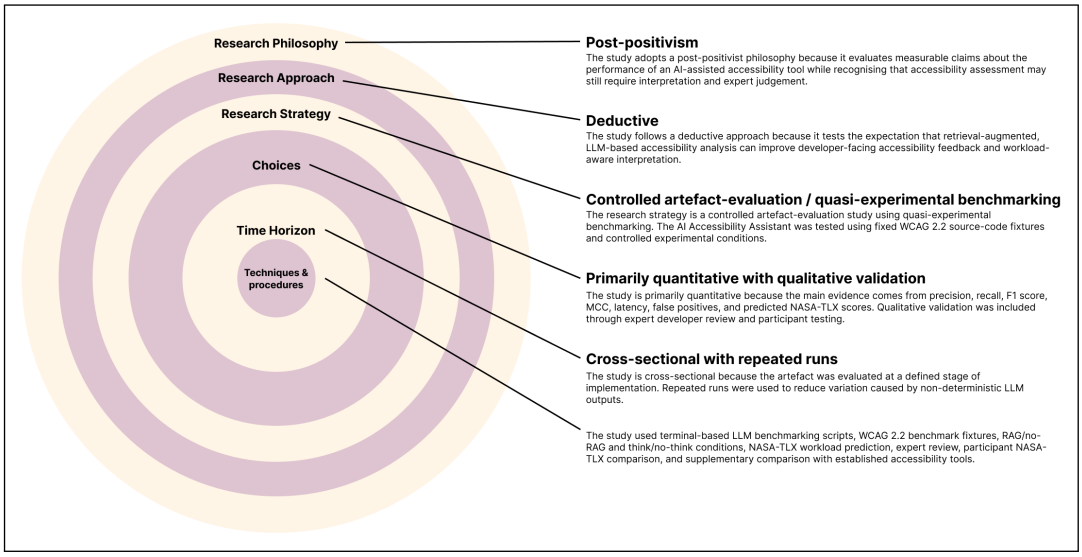



***Figure 3.2:*** *Research Onion for the Study*

| **Research Onion Layer** | **Dissertation Position** |
|---|---|
| **Research Philosophy** | Post-positivism |
| **Research Approach** | Deductive |
| **Research Strategy** | Controlled artefact-evaluation / quasi-experimental benchmarking |
| **Choices** | Primarily quantitative with qualitative validation |
| **Time Horizon** | Cross-sectional artefact evaluation with repeated benchmark runs |
| **Techniques and Procedures** | Terminal-based LLM benchmarking, WCAG 2.2 test-file evaluation, RAG/no-RAG and think/no-think conditions, precision/recall/F1/MCC/latency scoring, NASA-TLX prediction, expert review, participant validation, and supplementary tool comparison |

***Table 3.2:*** *Research Onion Positioning*

### *3.2.1 Research Objectives, Questions, and Hypothesis*

The primary objective is to evaluate whether an AI-assisted accessibility tool can detect WCAG 2.2 issues in web source code and support developers during implementation. The study also examines whether detected issues can be mapped to NASA-TLX dimensions to produce a workload-informed estimate, treated as a proxy rather than a direct cognitive-load measurement, and compared with participant-reported NASA-TLX scores.

Three research questions guide this study:

| **Research Question** | **Description** |
|---|---|
| **RQ1** | To what extent does the accessibility-issue-derived NASA-TLX workload estimate align with, approximate, or correlate with participant-reported NASA-TLX workload scores? |
| **RQ2** | How effectively can AI-assisted source-code analysis support the identification and remediation of WCAG 2.2 accessibility issues in HTML, CSS, JavaScript, and TypeScript/React files? |
| **RQ3** | How effectively can a developer-facing AI assistant provide actionable accessibility feedback within a source-code authoring environment? |

***Table 3.2.1:*** *Research Questions*

The overarching research hypothesis is:

> **H:** AI-based techniques that combine source-code accessibility analysis with NASA-TLX-informed workload estimation can provide developer-facing feedback that better reflects accessibility barriers and likely user burden than approaches focused only on rule-based accessibility checking.

### *3.2.2 High-Level System Architecture*

The AI Accessibility Assistant was implemented as a modular system: a VS Code extension, a RAG service, an LLM inference layer, and a structured output layer. The extension provided the developer-facing interface, the RAG service retrieved WCAG 2.2 and NASA-TLX guidance, the inference layer generated accessibility findings, and the structured output layer converted model responses into measurable issue objects.

Table 3.2.2 summarises each component's responsibilities.

| **Component** | **Main Responsibility** | **Research Purpose** |
|---|---|---|
| **VS Code Extension** | Reads source files, builds prompts, displays findings, and publishes diagnostics where line mapping is available | Provides the developer-facing environment and controls the analysis workflow |
| **RAG Service** | Retrieves relevant WCAG/accessibility and NASA-TLX guidance from the knowledge base | Grounds the model response in domain-specific guidance |
| **LLM Inference Layer** | Generates structured accessibility findings from the source-code prompt and retrieved context | Enables AI-assisted detection and explanation of accessibility issues |
| **Structured Output Layer** | Parses model responses into issue objects and prepares findings for display/evaluation | Converts free-text model output into analysable findings |

***Table 3.2.2:*** *System Component Responsibilities*

Figure 3.2.2 presents the high-level system architecture and shows how the main components interact during the accessibility analysis workflow.

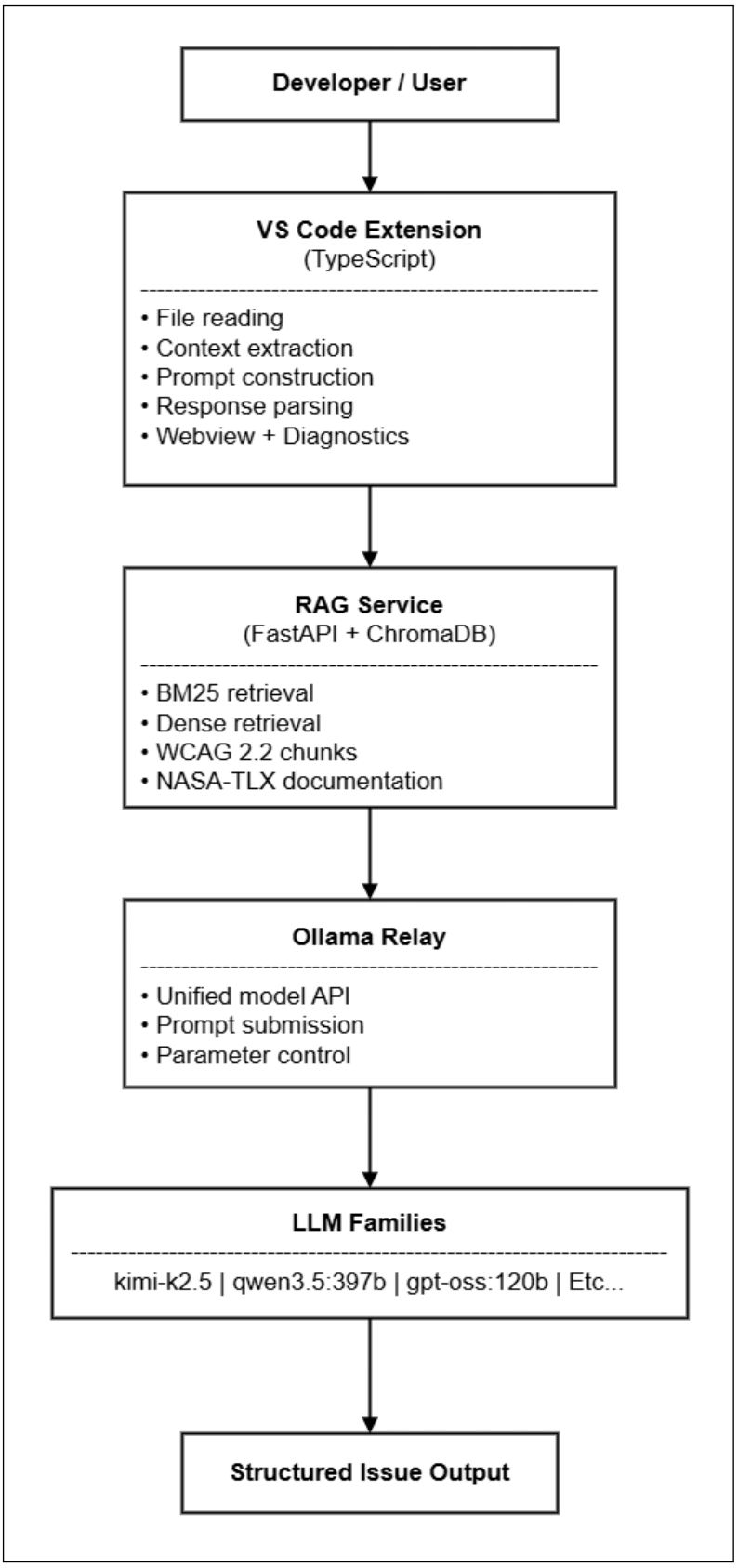



***Figure 3.2.2:*** *High-Level System Architecture Diagram*

### *3.2.3 End-to-End Pipeline Workflow*

The system followed a fixed analysis pipeline from source-code input to structured accessibility output, supporting repeatability across test files and configurations. The workflow is summarised in Table 3.2.3; component details are in Appendix A.

| Step | Stage | Description |
|---|---|---|
| **1** | Trigger | Developer activates analysis via the VS Code command palette or extension interface. |
| **2** | File Reading | Extension reads the active document and detects language context (HTML, CSS, JS, TSX). |
| **3** | Source-Code Preparation | Source-code excerpt prepared with language context and file metadata. |
| **4** | RAG Query and Retrieval | Accessibility themes converted to retrieval queries; RAG service returns relevant WCAG/NASA-TLX guidance. |
| **5** | Prompt Assembly | Prompt assembled from system instructions, file context, retrieved guidance, and source-code excerpt. |
| **6** | LLM Inference | Prompt submitted to the LLM pipeline; response returned in structured accessibility-finding format. |
| **7** | Response Parsing | Response parsed into issue objects: severity, line reference, WCAG criterion, user impact, and remediation. |
| **8** | Webview Rendering | Parsed issues displayed in the VS Code extension panel. |
| **9** | Diagnostic Publication | Findings published to the VS Code Problems panel where line mapping is available. |
| **10** | Metric Capture | Issue count, latency, and ground-truth match results recorded for analysis. |

***Table 3.2.3:*** *End-to-End Analysis Pipeline*

## 3.3 Scope, Limitations, and Reproducibility

This section defines the scope, limitations, and reproducibility controls used to evaluate whether retrieval-augmented LLM analysis can detect WCAG 2.2 accessibility issues in web source code. The study used a controlled, primarily quantitative, quasi-experimental design with fixed test files, prompts, scoring metrics, and documented inference presets. The evaluation was structured around four experimental conditions: `rag-think`, `rag-nothink`, `norag-think`, and `norag-nothink`. These conditions were used to compare retrieval augmentation and inference-preset behaviour within the test harness, rather than to describe final user-facing dropdown controls in the extension.

The scope of the evaluation was limited to static source-code analysis for HTML, CSS, JavaScript, and TypeScript/React files. These file types were selected because they represent the main front-end artefacts where accessibility-relevant structure, styling, interaction logic, ARIA relationships, and component-level behaviour are authored. Additional detail on test-file construction, error-density calibration, and deployment decisions is provided in Appendix B.

### *3.3.1 Research Philosophy and Design Strategy*

Following Creswell and Creswell's research design framework [55], this study adopts a post-positivist, primarily quantitative, quasi-experimental design. This is appropriate because the research evaluates measurable claims about accessibility detection accuracy under controlled conditions. Specifically, the study examines whether retrieval augmentation, inference-preset configuration, and model selection affect the ability of LLM-based analysis to detect known WCAG 2.2 accessibility issues in source-code test files.

The experiment used a 2×2 structure varying retrieval augmentation (enabled or disabled) and inference preset (`think` or `no-think`). The `think` preset used a reasoning-oriented configuration with higher temperature and extended context; the `no-think` preset used a more constrained balanced configuration. Because these presets also differed in associated inference settings, they are treated as controlled presets rather than a pure reasoning-mode manipulation. This produced four conditions: `rag-think`, `rag-nothink`, `norag-think`, and `norag-nothink`.

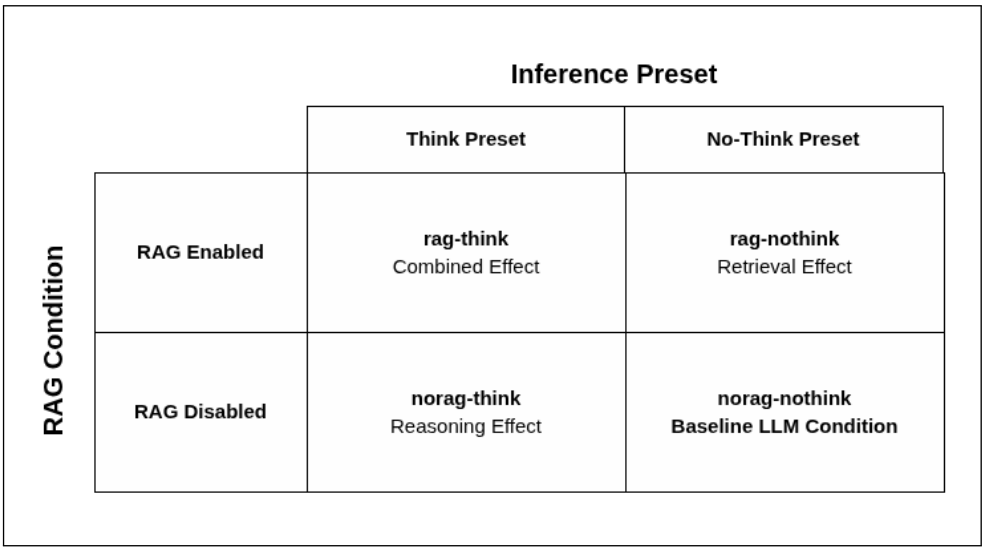



***Figure 3.3.1:*** *2×2 Factorial Experimental Design*

Model selection was treated as an additional comparison dimension so that different LLM families could be evaluated under the same condition structure. The detailed study sequence is discussed later in Section 3.4.

### *3.3.2 Domain Scope: EAA, WCAG 2.2, and File Type Coverage*

The study is motivated by the EAA, Directive 2019/882, and EN 301 549 [2, 3]. EN 301 549 V3.2.1 aligns web accessibility requirements with WCAG 2.1 Level AA. This study adopts WCAG 2.2 Level A and AA as a current technical evaluation standard because WCAG 2.2 extends WCAG 2.1 and remains backward-compatible with WCAG 2.1 conformance [1, 3].

The scope covers WCAG 2.2 criteria evidenceable from static source code. Runtime-dependent issues were excluded unless present in the analysed file; Level AAA criteria were also excluded. `prefers-reduced-motion` was included as a best-practice CSS indicator for motion sensitivity and cognitive accessibility.

Table 3.3.2a summarises the WCAG 2.2 criterion groups in scope.

| Category | Representative Success Criteria (SCs) Covered | WCAG Level | EAA/EN 301 549 Relevant |
|---|---|---|---|
| **Non-text Content** | 1.1.1 alt attributes on images, SVG alternatives | A | Yes |
| **Semantic Structure** | 1.3.1 Info and Relationships, table headers, form labels | A | Yes |
| **Input Purpose** | 1.3.5 Identify Input Purpose, autocomplete attributes | AA | Yes |
| **Visual Contrast** | 1.4.3 Contrast Minimum, statically resolvable colour values only | AA | Yes |
| **Focus Indicators** | 2.4.7 Focus Visible, 2.4.11 Focus Not Obscured Minimum | AA | Yes |
| **Bypass Blocks** | 2.4.1 Skip Links, landmark structure | A | Yes |
| **Page Title** | 2.4.2 Page Titled | A | Yes |
| **Link Purpose** | 2.4.4 Link Purpose in Context, non-descriptive link text | A | Yes |
| **Headings and Labels** | 2.4.6 Headings and Labels, heading level hierarchy | AA | Yes |
| **Name, Role, Value** | 4.1.2 ARIA attributes, roles, accessible names | A | Yes |
| **Status Messages** | 4.1.3 Status Messages, live regions | AA | Yes |
| **Label in Name** | 2.5.3 Label in Name | A | Yes |
| **Target Size** | 2.5.8 Target Size Minimum, statically resolvable dimensions only | AA | Yes |
| **Error Handling** | 3.3.1 Error Identification, 3.3.2 Labels or Instructions | A | Yes |
| **Keyboard Access** | 2.1.1 Keyboard, 2.1.2 No Keyboard Trap | A | Yes |
| **Animation** | 2.3.3 Animation from Interactions, `prefers-reduced-motion` | AAA* | No, AAA not mandated |

***Table 3.3.2a:*** *WCAG 2.2 Success Criterion Groups in Scope*

* *SC 2.3.3 is Level AAA and not required by the EAA; included as a best-practice indicator only.*

Table 3.3.2b summarises the test-file error-level calibration used to create clean, low, medium, and high accessibility-error conditions.

| File Tier | Error Count | Justification |
|---|---|---|
| **Clean** | 0 | Intentionally compliant baseline, reflecting pages without detectable barriers |
| **Low** | 10 | Minimal-violation condition |
| **Medium** | 30 | Aligned with reported average error density in large-scale accessibility studies [15] |
| **High** | 51 | Grounded in high-density accessibility findings from healthcare homepage studies [17] |

***Table 3.3.2b:*** *Test-File Error Level Calibration*

The ground-truth corpus contained 41 annotated test files: 16 regular test files and 25 adversarial test files. Adversarial test files contained valid-seeming but ambiguous accessibility patterns designed to test whether the system avoids spurious false positives on borderline constructs; they were used during model screening in Studies 1 and 2 to assess hallucination behaviour, and results are reported in Appendix E.2 and E.3. The file-size and error-density rationale is provided in Appendix B.1 and Appendix B.2.

### *3.3.3 Experimental Variables*

Table 3.3.3a summarises the independent variables.

| Variable | Description | Values |
|---|---|---|
| **Retrieval Augmentation** | Whether retrieved WCAG 2.2 knowledge chunks were injected into the model context at inference time | Enabled / Disabled |
| **Inference Preset** | Whether the model was run using the reasoning-oriented `think` preset or the more constrained `no-think` preset. These presets also differed in associated inference settings where supported. | Think / No-think |
| **Model Selection** | The specific LLM or model ensemble used to perform accessibility analysis | Multiple cloud-hosted models across families |

***Table 3.3.3a:*** *Independent Variables*

The first two variables formed the 2×2 factorial design shown in Figure 3.3.1.

Table 3.3.3b summarises the dependent variables.

| Metric | Description | Justification |
|---|---|---|
| **Precision** | Proportion of detected issues that matched a ground-truth concept | Measures false-positive control and output correctness |
| **Recall** | Proportion of ground-truth concepts that were detected | Measures detection completeness |
| **F1 Score** | Harmonic mean of precision and recall | Balances missed issues and false positives |
| **MCC** | Balanced classification metric accounting for all confusion-matrix quadrants | More informative when class distribution is uneven [51] |
| **Latency** | Time from prompt submission to complete response receipt | Operational performance indicator for developer workflows |
| **NASA-TLX-Informed Workload Estimate** | Computational workload proxy derived from the detected accessibility-barrier inventory and mapped to NASA-TLX dimensions | Provides a workload-informed estimate for comparison with participant-reported NASA-TLX scores, without claiming to directly measure cognitive load [12, 13] |

***Table 3.3.3b:*** *Dependent Variables*

Table 3.3.3c summarises the fixed variables used to support reproducibility.

| Variable | Fixed Value |
|---|---|
| **Prompt structure and template** | Identical structured prompt for all models and conditions |
| **Evaluation corpus** | Same annotated ground-truth test files for all runs |
| **Ground-truth matching method** | Keyword-match scoring function applied identically across all runs |
| **Scoring metrics** | Precision, recall, F1 score, MCC, latency, and TLX computed consistently |
| **RAG retrieval parameters** | Fixed retrieval parameters within each RAG-enabled condition |
| **Inference hyperparameters** | Fixed values within each experimental configuration |

***Table 3.3.3c:*** *Control Variables*

### *3.3.4 Boundary Conditions and Reproducibility Controls*

The main boundary conditions were static source-code analysis, single-file review, test-file-based ground truth, keyword-match scoring, and cloud inference non-determinism. These constraints supported experimental control but limited generalisation to fully rendered or runtime-dependent web applications.

The RAG layer used ChromaDB as the persistent vector store, `all-MiniLM-L6-v2` embeddings for dense retrieval, and BM25Okapi for sparse keyword retrieval [41–44]. LLM inference was routed through the Ollama HTTP API to maintain a consistent interface across model families. Additional boundary and deployment details are provided in Appendix B.3 and Appendix B.4.

The complete candidate model pool is provided in Appendix B.4. It included cloud-routed and locally executed models from multiple model families, allowing early screening to assess detection capability, inference stability, and deployment feasibility.

Inference settings were documented before experimentation; supported parameters varied between cloud and local routes, and the full parameter mapping is recorded in Appendix B.4.

Table 3.3.4 records the default inference settings used where no preset-specific override applied. Preset-specific differences are reported separately in Table 3.4.2.

| Parameter | Value | Scope | Justification |
|---|---|---|---|
| `temperature` | 0.1 | Cloud and local | Supports consistency in structured problem-solving tasks [48] |
| `top_p` | 0.9 | Cloud and local | Reduces stochastic noise in structured generation [49] |
| `num_predict` | 32,000 | Cloud and local | Prevents truncation on violation-dense test files |
| `num_ctx` | 32,768 | Local only | Supports full prompt and output within local context |
| `top_k` | 40 | Local only | Retains standard candidate-token filtering [50] |
| `repeat_penalty` | 1.05 | Local only | Reduces repeated output without suppressing recurring WCAG and ARIA terms [41, 50] |
| `repeat_last_n` | 64 | Local only | Reduces repetition checks across repeated issue blocks |
| `frequency_penalty` | 0.0 | Local only | Avoids suppressing recurring technical terms |
| `presence_penalty` | 0.0 | Local only | Same rationale as frequency penalty |
| `seed` | 42 | Local only | Supports reproducibility where local seeding is available [56] |

***Table 3.3.4:*** *Default Inference Settings Used Where No Preset-Specific Override Applied*

### *3.3.5 Experimental Design Structure and Workflow*

All experiments followed the same workflow: select model and condition, load test file, run baseline checks, apply the RAG setting and inference preset, submit the prompt, parse the response, and compare against ground truth. Metrics were then computed where applicable.

Figure 3.3.5 summarises the workflow.

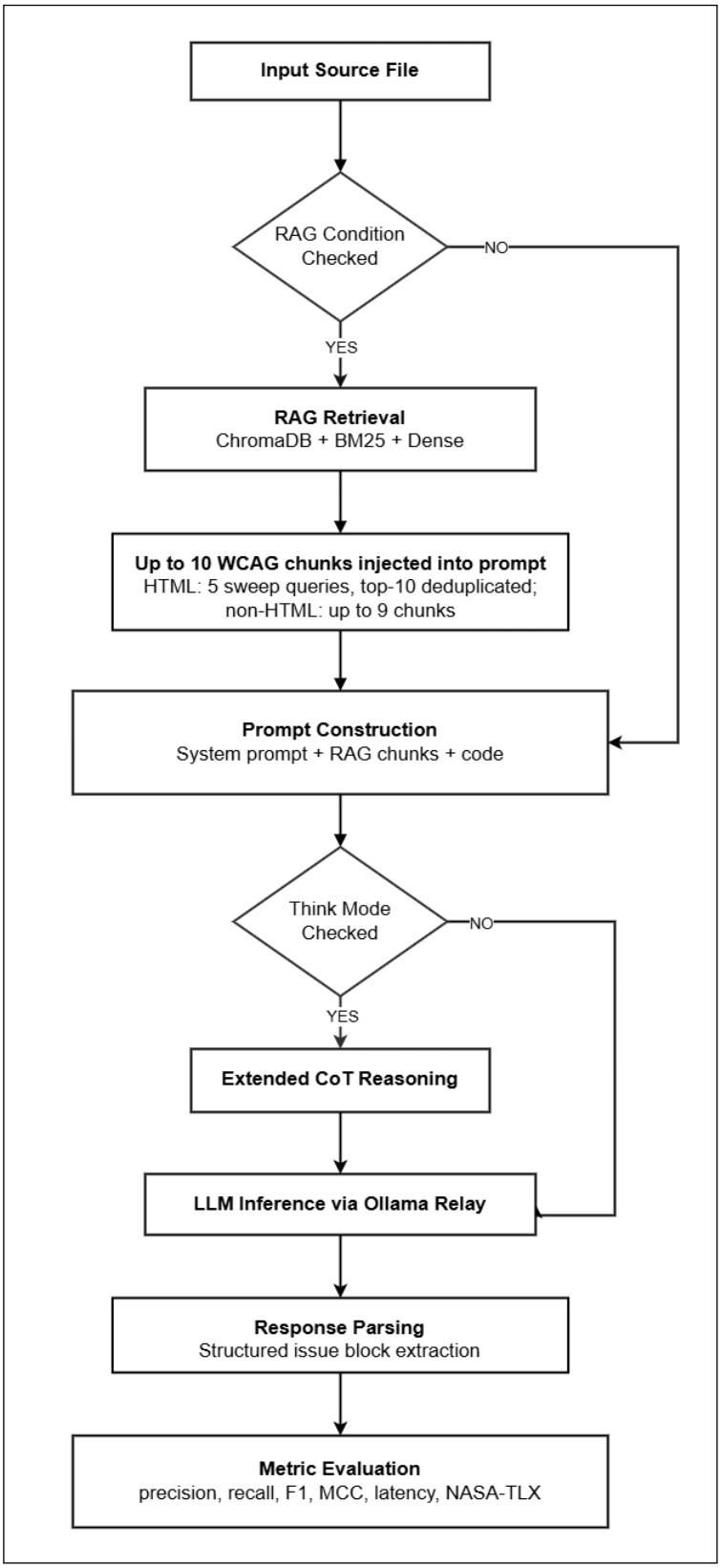



***Figure 3.3.5:*** *Experimental Workflow*

## 3.4 System Implementation

This section summarises the AI Accessibility Assistant implementation as a modular pipeline: VS Code extension, RAG service, LLM inference layer, and structured output layer. Further detail is in Appendix C.

### *3.4.1 Core Components*

The system comprised a VS Code extension, RAG service, LLM inference layer, and structured output layer. Figure 3.4.1 shows how these components connect.

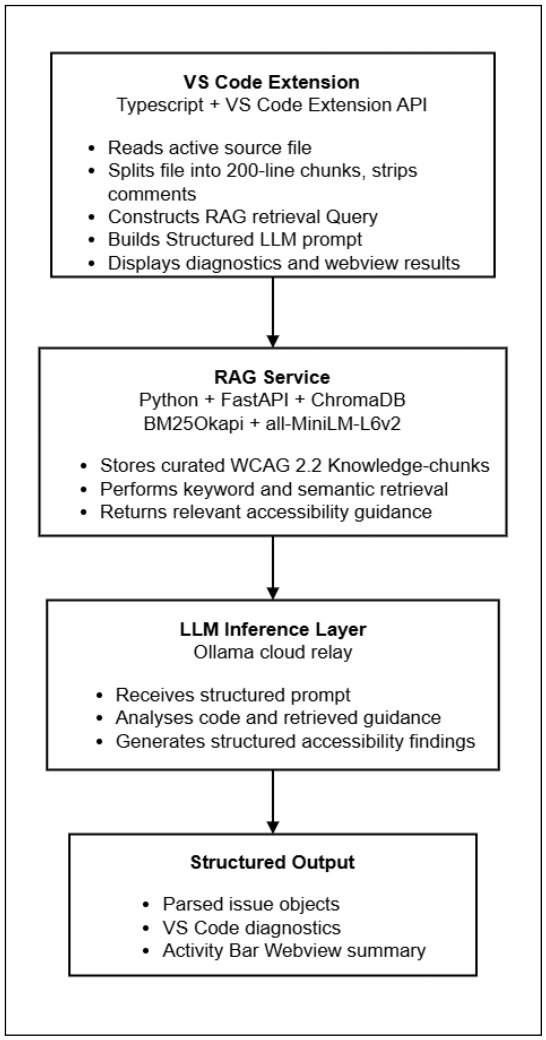



***Figure 3.4.1:*** *Schematic Diagram of the System Implementation Pipeline*

### *3.4.2 Functional Behaviour*

The extension detected file type via `document.languageId` [22], then ran deterministic baseline checks, split files into 200-line chunks with 30-line overlap, stripped comments (inline `//` and block `/* */` for JS/CSS/TSX; `<!-- -->` for HTML) to reduce token overhead while preserving line numbers, and built a RAG query. Baseline findings (missing alt text, unassociated inputs, suppressed focus) were passed as diagnostics and included in the model prompt.

The RAG service retrieved WCAG 2.2 and NASA-TLX guidance via hybrid BM25 and dense retrieval [41–44]. The prompt instructed the model to evaluate only source-file evidence and report findings consistently, drawing on structured prompting approaches [45, 46].

Table 3.4.2 summarises the inference presets. Unsupported parameters on cloud-routed models were recorded as not applicable.

| **Parameter** | **Balanced Preset: No-Think** | **Reasoning Preset: Think** |
|---|---|---|
| `temperature` | 0.30 (kimi-k2.5); 0.70 (qwen3.5) | 0.60 |
| `top_p` | 0.90 (kimi-k2.5); 0.80 (qwen3.5) | 0.95 |
| `top_k` | N/A (local Ollama only) | |
| `repeat_penalty` | N/A (local Ollama only) | |
| `num_predict / max tokens` | 32,000 | |
| `num_ctx / context window` | N/A (local Ollama only) | |
| `seed` | N/A (local Ollama only) | |

***Table 3.4.2:*** *Inference Parameter Presets by Experimental Condition*

Detailed baseline, retrieval, prompt, caching, and parsing behaviour is provided in Appendix C.2–C.7.

### *3.4.3 Core Algorithm*

The core algorithm followed a fixed sequence: detect file type, run baseline checks, split the source file into 200-line chunks with 30-line overlap, retrieve relevant guidance, assemble and submit the prompt to both models, deduplicate within-model outputs using title-only Jaccard (threshold 0.40), merge cross-model findings via Jaccard consensus (title-and-explanation score $\geq$0.30, line numbers within $\pm$5), admit single-model survivors that cite a specific WCAG success criterion with high or medium severity and a concrete line number, apply an accessibility-term filter, and publish the final issues.

Figure 3.4.3 presents the flowchart of the end-to-end analysis algorithm.

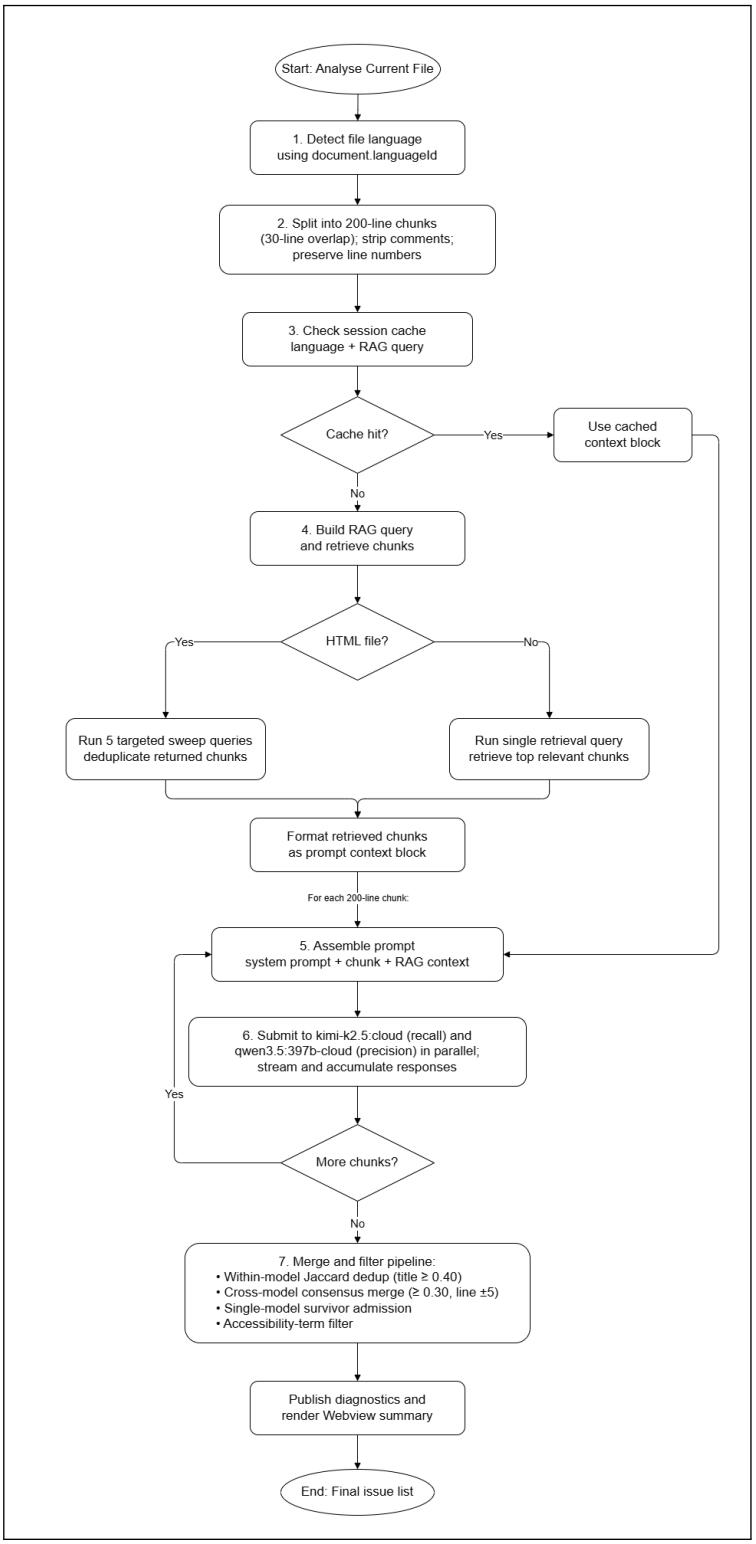



***Figure 3.4.3:*** *Flowchart of the End-to-End Analysis Algorithm*

Detailed explanation of each algorithmic stage is provided in Appendix C.2–C.7.

### *3.4.4 System Behaviour and Execution Conditions*

The implementation supported the four execution conditions from Section 3.2.3. RAG-enabled runs injected retrieved WCAG 2.2 guidance into the prompt; RAG-disabled runs used only the source excerpt, baseline findings, and system prompt. Think mode applied the reasoning preset; no-think mode applied the balanced preset. All conditions included deterministic baseline checks.

### *3.4.5 Reliability Testing*

Before formal benchmarking, the prototype was tested using an Input, Process, Expected Output, Actual Output, and Corrective Action framework. The checks covered extension activation, baseline diagnostics, RAG indexing, retrieval, prompt construction, streamed inference, response parsing, diagnostic publication, and Web-view rendering. Additional reliability notes are provided in Appendix D.

Table 3.4.5 summarises the prototype reliability verification checks carried out before formal benchmarking.

| **Input** | **Process** | **Expected Output** | **Actual Output** | **Corrective Action** |
|---|---|---|---|---|
| **TypeScript source file opened in VS Code** | `activate()` registers the extension commands and instantiates the sidebar Webview provider | Extension activates without error; available commands appear; sidebar renders | Extension activated successfully; commands were available; sidebar loaded correctly | None required |
| **HTML file analysed** | The baseline checking function applies image alt and form label rules | Diagnostics appear in Problems tab without network access | Diagnostic markers appeared at correct positions | None required |
| **CSS file analysed** | Focus-indicator rules are applied to interactive selectors | Selectors suppressing focus without replacement are flagged | Correct diagnostics were produced; no false positives for selectors with replacement styles | None required |

| **Input** | **Process** | **Expected Output** | **Actual Output** | **Corrective Action** |
|---|---|---|---|---|
| **Knowledge-base Markdown files available** | RAG service chunks, embeds, stores, and indexes documents | ChromaDB and BM25 indexes are populated | Indexing completed successfully | None required |
| **Retrieval query sent to `/retrieve`** | Hybrid BM25 and dense retrieval is performed | Relevant WCAG 2.2 chunks are returned | Relevant chunks were returned; off-topic chunks were filtered | Coarse character chunking was replaced with section-based 128-token chunking |
| **Extension assembles excerpt, baseline findings, and RAG chunks** | Prompt builder inserts system message, excerpt, and context block | Well-formed prompt is produced | Prompt assembled correctly with line numbers and bounded context | None required |

| **Input** | **Process** | **Expected Output** | **Actual Output** | **Corrective Action** |
|---|---|---|---|---|
| **Prompt submitted to Ollama cloud relay** | Stream opens to `/api/chat` | Streamed response is received and terminates correctly | Cloud stream worked; local Ollama failed with connection errors | Cloud relay retained as active inference route |
| **LLM response contains issue blocks** | Parser removes reasoning blocks and extracts issue fields | Structured issue objects are returned | Issues parsed correctly; speculative content filtered | Additional speculative-language filters were added |
| **Parsed issues available** | Issues are mapped to VS Code diagnostics | Issues are visible in editor, Problems tab, and Webview | Diagnostics and summary card rendered correctly | None required |

***Table 3.4.5:*** *Prototype Reliability Verification IPO Chart*

## 3.5 Benchmarking and Validation Studies

This section summarises the six-study evaluation programme. Studies 1–4 used controlled benchmark test files; Studies 5 and 6 provided real-world validation through expert review, user testing, and tool comparison. Full records are in Appendix E; findings for Studies 5 and 6 are in Chapter 4.

### *3.5.1 Study 1: Candidate Model Screening*

Study 1 screened the initial candidate model pool using the `html-high` test file (51 ground-truth violations), removing models with poor detection, excessive false positives, or instability.

This stage also tested local deployment viability and cloud-routed inference stability. Local models were explored, but hardware and stability constraints made them unsuitable for the main experimental pipeline. Cloud-routed inference through the Ollama Application Programming Interface (API) was therefore retained for later testing. Early RAG and prompt behaviour were also refined during this stage, including the move from broad character-based chunks to more precise section-based chunking, reduction of irrelevant retrieved material, and expansion of WCAG rule documents.

Table 3.5.1 summarises the methodological purpose of Study 1.

| **Study Focus** | **Methodological Purpose** |
|---|---|
| **Initial model screening** | Removed models with poor detection, excessive hallucination, or unstable output |
| **Local versus cloud inference** | Assessed whether local deployment was feasible for the VS Code extension workflow |
| **Early RAG testing** | Identified retrieval problems and supported chunking/prompt refinement |
| **`html-high` test file** | Provided a high-density accessibility benchmark using 51 ground-truth issues |

***Table 3.5.1:*** *Study 1 Model Screening Summary*

Ten models were retained for Study 2; the full list and elimination record are in Appendix E.2.

### *3.5.2 Study 2: Parametric Optimisation and Multi-File Evaluation*

Study 2 extended testing to `css-high`, `js-high`, and `tsx-high` to assess whether models generalised across source-code types. Model-specific parameters, language-specific prompt guidance, and three-run aggregation (union rule) were introduced to reduce stochastic variation; the aggregation rationale is discussed in Section 3.5.3.

Table 3.5.2 summarises the methodological scope of Study 2.

| **Study Focus** | **Methodological Purpose** |
|---|---|
| **Multi-file testing** | Tested generalisation across HTML, CSS, JavaScript, and TSX |
| **Model-specific parameters** | Improved fairness by allowing stronger candidate configurations |
| **Language-specific guidance** | Added file-type-specific accessibility prompts |
| **Three-run aggregation** | Reduced the effect of stochastic LLM variation |
| **Shortlisting** | Narrowed the model pool for advanced configuration testing |

***Table 3.5.2:*** *Study 2 Evaluation Scope*

Study 2 produced a reduced candidate shortlist for later advanced configuration testing. The detailed model elimination record, shortlist results, aggregation notes, language-specific sweep details, and supporting graphs are provided in Appendix E.3, where interpreted performance findings are also provided.

### *3.5.3 Study 3: Advanced Configuration and Ensemble Evaluation*

Study 3 examined advanced configurations on the narrowed candidate pool: RAG chunk-size ablation, model version comparison, and ensemble voting. The 128-token chunk size was retained for stronger retrieval; `kimi-k2.5` was retained over `kimi-k2.6` after version comparison showed weaker recall in the newer version. Dual-model and three-model ensemble configurations were compared.

Table 3.5.3 summarises the configuration areas examined in Study 3.

| **Configuration Area** | **Methodological Purpose** |
|---|---|
| **RAG chunk-size ablation** | Tested which chunking strategy supported more relevant retrieval |
| **Model version comparison** | Checked whether newer model versions improved or weakened detection |
| **Dual-model voting** | Tested whether two complementary models improved output quality |
| **Three-model voting** | Tested whether adding a third model improved recall or increased false positives |
| **Final configuration selection** | Selected the model configuration for later validation |

***Table 3.5.3:*** *Study 3 Configuration Summary*

Study 3 produced the final model configuration used for later validation. The detailed ensemble-selection notes, comparison tables, configuration outcomes, and supporting graphs are provided in Appendix E.4, where interpreted performance findings are also provided.

### *3.5.4 Study 4: NASA-TLX-Informed Workload Estimation Benchmark*

Study 4 evaluated the NASA-TLX-informed workload estimation pipeline [12, 13], examining whether the system's estimate responded appropriately to issue count, severity, difficulty, and language type before comparison with participant-reported scores in Study 5. Three benchmark runs covered ensemble comparison, severity-weight refinement, WCAG mapping, subscale caps, language difficulty modifiers, and JavaScript knowledge-base expansion. The estimate was treated as a heuristic proxy, not a direct cognitive-load measurement.

Table 3.5.4 summarises the NASA-TLX benchmark focus areas used in Study 4.

| **Benchmark Focus** | **Methodological Purpose** |
|---|---|
| **Ensemble comparison** | Compared NASA-TLX-informed workload-estimation behaviour across model configurations |
| **Severity weighting** | Increased the contribution of critical and high-severity issues |
| **WCAG-to-TLX mapping** | Linked accessibility barriers to NASA-TLX workload dimensions |
| **Subscale caps** | Prevented workload estimates from exceeding defined scale limits |
| **Language difficulty modifiers** | Reflected observed differences between HTML, CSS, JavaScript, and TSX |
| **JavaScript knowledge-base expansion** | Improved support for dynamic interaction patterns |

***Table 3.5.4:*** *Study 4 NASA-TLX Benchmark Focus*

Additional NASA-TLX refinement notes, test tables, workload-mapping details, and supporting graphs are provided in Appendix E.5 and Appendix F.3.

### *3.5.5 Study 5: Real-World Expert and User Validation*

Study 5 applied the selected AI assistant configuration to a defective website and used its recommendations to produce an AI-remediated version. This study introduced real-world validation beyond the synthetic benchmark test files by combining expert developer review with participant-facing usability and workload evaluation.

Developer reviewers assessed the AI-generated issue list and remediation decisions. Their comments were coded using the scheme in Appendix F.4: confirmed genuine barriers, confirmed false positives, missing issues, appropriate remediations, and remediation concerns, connecting expert feedback to precision, recall-gap identification, and fix-quality assessment.

Vision-impaired participants using screen readers completed comparable tasks on both the defective and improved website versions and completed NASA-TLX after each condition [12, 13]. Participants completed the defective version first to experience the original barriers before evaluating the remediated version; this fixed order is acknowledged as a possible order effect in Chapter 4. Participants used their preferred assistive technologies and interaction methods throughout.[1]

The study involved developer reviewers and vision-impaired participants who used screen readers; final numbers are reported in Chapter 4. No medical diagnosis or clinical information was required. Participant-facing materials, consent forms, task sheets, and NASA-TLX questionnaires are in Appendix G, without identifiable participant information.

[1] All participants were fully blind screen-reader users. The workload comparison therefore reflects barriers as they manifest in the rendered HTML/ARIA tree; CSS, JS, and TSX issues that do not surface at that layer are outside the scope of the workload comparison.

Table 3.5.5a summarises the validation artefacts used in Study 5.

| Artefact | Description | Purpose |
|---|---|---|
| **Defective website** | Original version containing accessibility barriers | Baseline condition |
| **AI-generated issue list** | Findings produced by the assistant | Used for expert review and remediation |
| **Improved website** | Version after applying reviewed AI recommendations | Comparison condition |
| **Expert review notes** | Developer assessment of issues and fixes | Evaluates precision, recall gaps, and fix quality |
| **NASA-TLX responses** | User workload ratings for both versions | Compares predicted and reported workload |

***Table 3.5.5a:*** *Study 5 Validation Artefacts*

Table 3.5.5b summarises the expert developer review focus used to validate the AI-generated findings and fixes.

| Review Focus | Purpose |
|---|---|
| **Whether each flagged issue was genuine** | Supports precision assessment |
| **Whether each applied fix resolved the issue** | Supports remediation quality assessment |
| **Whether significant issues were missed** | Supports recall-gap identification |

***Table 3.5.5b:*** *Expert Developer Review Focus*

Figure 3.5.5 summarises the real-world validation workflow used in Study 5.

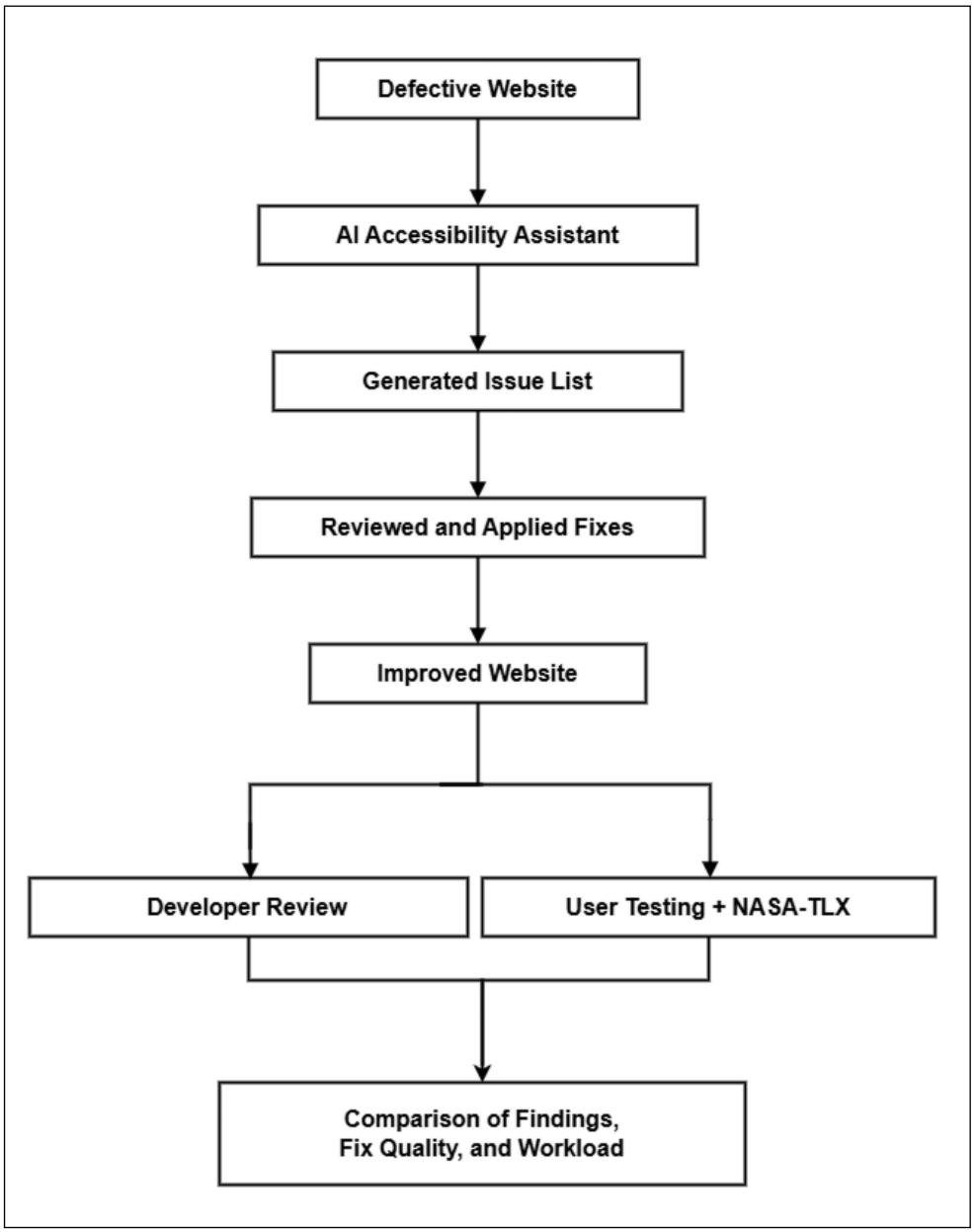



***Figure 3.5.5:*** *Study 5 Real-World Validation Workflow*

Full Study 5 findings are reported in Chapter 4. Additional validation materials, consent forms, task sheets, testing screenshots, NASA-TLX materials, and ethical safeguards are provided in Appendix E.6, Appendix F.3, Appendix F.4, and Appendix G.

### *3.5.6 Study 6: Supplementary Comparison*

Study 6 compared the AI Accessibility Assistant with established automated accessibility tools across HTML, CSS, JavaScript, and TypeScript/React test files, including Lighthouse, axe-core, Axe Linter, ESLint jsx-a11y, Webhint, and stylelint-a11y [4–10]. The comparison was descriptive rather than a direct benchmark, as these tools analyse different representations of web content (rendered DOM, source code, JSX/TSX, and CSS).

Findings were aligned by issue category; shared findings were treated as corroborated issues, AI-only findings as possible semantic detections, and tool-reported issues missed by the AI as recall gaps. Descriptive findings are reported in Chapter 4; additional tool-alignment categories and supplementary notes are in Appendix E.7.

Table 3.5.6 summarises the supplementary tool comparison scope.

| **Tool** | **Analysis Target** | **Purpose in Study** |
|---|---|---|
| **AI Accessibility Assistant** | Source files inside VS Code | Identifies source-level and semantic accessibility issues |
| **Lighthouse [4, 5]** | Rendered DOM in browser | Provides an established browser audit reference |
| **axe-core [6]** | Rendered DOM/rule engine | Provides rule-based accessibility issue comparison |
| **Axe Linter [7]** | Source code/editor workflow | Provides source-level linting comparison |
| **ESLint jsx-a11y [8]** | JSX/TSX source code | Provides React accessibility linting comparison |
| **Webhint [9]** | Web configuration and source/browser hints | Provides configurable web quality and accessibility checks |
| **stylelint-a11y [10]** | CSS source code | Provides CSS accessibility linting comparison |

***Table 3.5.6:*** *Supplementary Tool Comparison Scope*

Figure 3.5.6 summarises the supplementary tool comparison process.

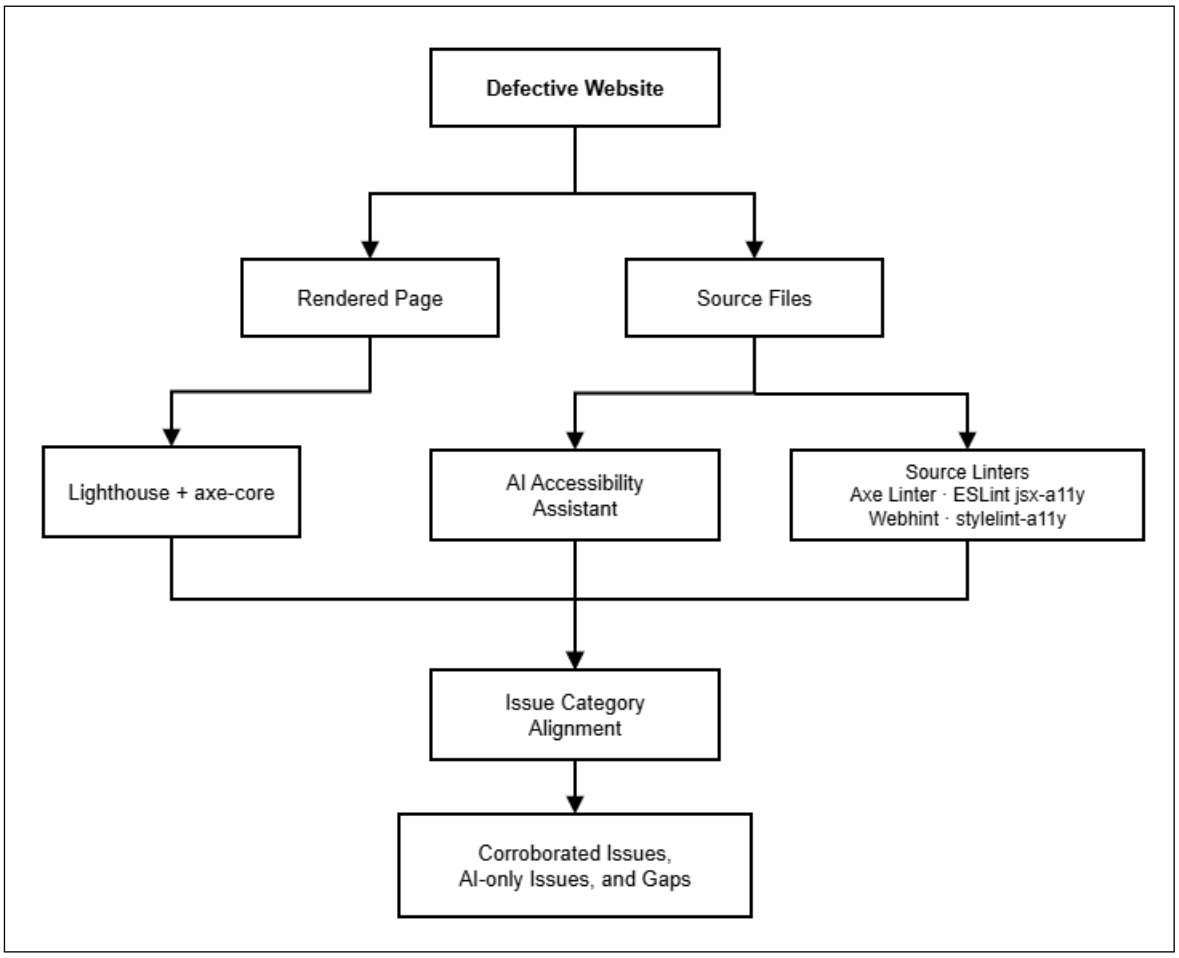



***Figure 3.5.6:*** *Supplementary Tool Comparison Process*

## 3.6 Evaluation Framework and Testing Matrix

This section defines how system performance was measured across the benchmarking and validation studies. The evaluation framework included a benchmark test file suite, classification metrics, acceptance criteria, NASA-TLX workload comparison, and expert review coding. Further detail on keyword matching, NASA-TLX barrier mapping, and expert review coding is provided in Appendix F.

### *3.6.1 Benchmark Test File Suite*

The evaluation corpus comprised 16 regular code test files organised by file language and violation density. The four file languages were HTML, CSS, JavaScript, and TypeScript/React. The four difficulty levels were clean, low, medium, and high.

Test files were cumulative within each language. The low test file extended the clean version with ten accessibility violations, the medium test file extended the low version, and the high test file contained the full violation set. This ensured that difficulty increased through violation density rather than unrelated changes in file content.

Each injected violation was recorded as an `IssueConcept` in the ground-truth registry. During evaluation, model output was matched against these ground-truth concepts using keyword matching. This allowed different model phrasings to be recognised while still requiring issue-specific evidence. The full keyword-matching approach is explained in Appendix F.1.

Table 3.6.1 summarises the benchmark test file matrix used during evaluation.

| File ID | Language | Difficulty | Ground-Truth Issues |
|---|---|---|---|
| `html-clean` | HTML | Clean | 0 |
| `html-low` | HTML | Low | 10 |
| `html-medium` | HTML | Medium | 30 |
| `html-high` | HTML | High | 51 |
| `css-clean` | CSS | Clean | 0 |
| `css-low` | CSS | Low | 10 |
| `css-medium` | CSS | Medium | 30 |
| `css-high` | CSS | High | 51 |
| `js-clean` | JavaScript | Clean | 0 |
| `js-low` | JavaScript | Low | 10 |
| `js-medium` | JavaScript | Medium | 30 |
| `js-high` | JavaScript | High | 51 |
| `tsx-clean` | TypeScript/React | Clean | 0 |
| `tsx-low` | TypeScript/React | Low | 10 |
| `tsx-medium` | TypeScript/React | Medium | 30 |
| `tsx-high` | TypeScript/React | High | 51 |
| **Totals** | 4 languages | 12 non-clean test files | 364 annotated issues |

***Table 3.6.1:*** *Benchmark Test File Matrix*

### *3.6.2 Classification Metrics*

Each ground-truth `IssueConcept` was treated as a binary classification slot, evaluated using True Positive (TP), False Positive (FP), False Negative (FN), and True Negative (TN) outcomes. The primary metric was F1 score (harmonic mean of precision and recall); MCC was used as a secondary metric for uneven class distributions [51]. Latency was recorded as an operational indicator; fix quality was assessed through expert review in Study 5. Full metric definitions, formulas, and the confusion matrix are in Appendix F.1.

### *3.6.3 Composite Scoring and Acceptance Criteria*

During Study 1, candidate models were ranked using a composite score combining F1 score and speed.

$$\text{Composite} = 0.8\,F_1 + 0.2\,S \tag{3.1}$$

$$S = 1 - \frac{t_{\text{model}} - t_{\text{min}}}{t_{\text{max}} - t_{\text{min}}} \tag{3.2}$$

where $S$ represents the normalised latency score, $t_{\text{model}}$ represents the model's observed latency, $t_{\text{min}}$ represents the fastest observed latency, and $t_{\text{max}}$ represents the slowest observed latency. Higher $S$ values therefore represented faster output. The composite formula weighted F1 score more heavily than speed because detection reliability was more important than response time during initial model screening.

From Study 2 onwards, the composite score was replaced with an 80.0% overall accuracy threshold per model-condition combination. A model could remain under consideration only if it met this threshold or if failure was caused by an addressable system issue, such as missing prompt guidance or retrieval configuration problems.

From the multi-file evaluation phase of Study 2 onwards, each model-condition-test-file combination was run three times. A union rule was applied during model-screening and aggregation analysis, where a ground-truth issue was counted as detected if it appeared in at least one of the three runs. This reduced the effect of stochastic variation in LLM output and helped avoid eliminating a model because

of a single weak run. However, this aggregation approach may overestimate performance compared with a single real-world execution. For this reason, aggregated results were treated as benchmarking evidence rather than as a direct representation of single-run end-user performance, and Chapter 4 distinguishes between single-run and aggregated results where relevant.

### *3.6.4 NASA-TLX as Dual-Role Evaluation Instrument*

NASA-TLX was used in two ways. First, the system used NASA-TLX as a framework for generating a workload-informed estimate from detected accessibility barriers. Each violation category was mapped to one or more NASA-TLX dimensions, such as Mental Demand, Effort, or Frustration [12, 13]. This output was treated as a heuristic workload proxy rather than as a direct measurement or prediction of cognitive load.

Second, NASA-TLX was used as a self-report questionnaire in Study 5. Participants completed tasks on both the defective website and the AI-remediated version, then rated their workload across the six NASA-TLX dimensions. These participant-reported scores were compared with the system's NASA-TLX-informed workload estimates to examine whether the accessibility-issue-derived estimate aligned with, approximated, or correlated with user-reported workload. The full barrier-to-TLX mapping is provided in Appendix F.3. Per the standard NASA-TLX convention, higher Performance scores indicate greater workload from perceived task failure [12].

### *3.6.5 Statistical Analysis Plan*

The statistical analysis compared the system's NASA-TLX-informed workload estimates with participant-reported NASA-TLX scores. Pearson correlation (or Spearman as a non-parametric alternative) was used to compare AI-derived and participant-reported NASA-TLX scores. A Wilcoxon signed-rank test assessed whether remediation reduced reported workload. The expert-review coding scheme is in Appendix F.4.

## 3.7 Ethical Considerations

Ethical procedures applied mainly to Study 5, as Studies 1 to 4 used synthetic code test files and did not involve human participants or personal data. Study 5 involved expert developer review and user testing with vision-impaired participants using screen readers. Supporting participant materials, including information sheets, consent forms, task sheets, and testing materials, are provided in Appendix G.

The study was conducted in accordance with Malta College of Arts, Science and Technology (MCAST) and Institute of Information and Communication Technology (IICT) ethical requirements. Vision-impaired participants using screen readers and developer reviewers were recruited voluntarily, received a plain-language information sheet, and provided written informed consent before participation. Participant-facing materials were provided in accessible formats where required.

Participant data were processed in line with the General Data Protection Regulation (GDPR), Regulation (EU) 2016/679 [57] and MCAST data manage-

ment requirements. Participants were assigned numeric codes, all analysis used pseudonymised data, and raw data were stored on an encrypted local device. No medical diagnoses, clinical details, or unnecessary personal information were collected, analysed, or reported.

The study presented minimal risk because tasks involved ordinary website interaction using participants' preferred devices and assistive technologies. Sessions were kept short, rest breaks were offered, and participants could stop at any time without giving a reason. Participants and expert reviewers were informed that AI-generated issue lists and recommendations could be incomplete, incorrect, or contextually inappropriate. The AI system was evaluated only as the research artefact and was not used for recruitment, screening, participant selection, coding, statistical analysis, or processing participant responses.

## 3.8 Chapter Summary

This chapter established the methodological and technical foundation for evaluating the AI Accessibility Assistant. It defined the study's primarily quantitative quasi-experimental design, system implementation, benchmark corpus, evaluation metrics, NASA-TLX integration, validation studies, and ethical procedures. Together, these elements provided a structured basis for assessing retrieval-augmented LLM accessibility analysis within a developer workflow. The next chapter presents the results generated through this methodology.

# Chapter 4: Analysis of Results and Discussion

## 4.1 Overview

This chapter presents and analyses the results of the two evaluation studies conducted to assess the AI Accessibility Assistant. Section 4.2 summarises the evaluation dataset and the metrics used throughout the chapter. Section 4.3 reports findings from Study 5, the main expert review and user validation study, covering detection performance, re-run results, NASA-TLX workload data, and participant survey responses. Section 4.4 presents findings from Study 6, the supplementary tool comparison study. Section 4.5 synthesises results across both studies, followed by analytical answers to each research question in Section 4.6, a discussion in relation to existing literature in Section 4.7, and a review of threats to validity in Section 4.8. The chapter concludes with a summary in Section 4.9.

## 4.2 Evaluation Dataset and Reporting Approach

Results throughout this chapter are reported using a consistent set of metrics. Precision, recall, and F1 score are the primary classification metrics. Precision measures the proportion of AI-reported issues that correspond to genuine known issues in the ground truth ($\text{Precision} = \text{TP}/(\text{TP}+\text{FP})$). Recall measures the proportion of ground-truth issues that the AI detected ($\text{Recall} = \text{TP}/(\text{TP}+\text{FN})$). The F1 score is their harmonic mean. Where applicable, false-positive and false-negative counts are reported separately to allow interpretation of over- and under-detection. Latency is

reported in milliseconds where captured. For the NASA-TLX workload analysis, scores are reported on a 0–100 scale per dimension following Hart and Staveland's original instrument [12], with an unweighted overall workload calculated as the arithmetic mean of the six subscales [13].

The Study 5 evaluation dataset consisted of four test files (HTML, CSS, JavaScript, and TypeScript/React), each containing 30 planted accessibility issues selected from the WCAG 2.2 Level A and AA criterion groups described in Section 3.3.2. Each test file was validated by five developer participants. A clean counterpart test file was run once per participant who chose the corresponding file type, serving as a true-negative baseline to test for spurious detections. A re-run was conducted after participants applied fixes suggested by the AI, allowing the study to measure whether detected issues were subsequently resolved and whether new issues or false positives appeared after remediation. This design is consistent with shift-left evaluation methodology and allows both detection accuracy and remediation effectiveness to be assessed in a single session [58].

Where the AI reported more findings than the 30 planted issues, each additional finding was audited against the source code. Findings confirmed as genuine accessibility issues beyond the planted set are counted as additional true positives; only findings with no corresponding code evidence are treated as false positives. This distinction is reported explicitly in Study 6, where raw finding counts exceed the planted set on some file types.

## 4.3 Study 5: Expert Review and User Validation Results

Study 5 was the primary real-world validation study for the AI Accessibility Assistant. It combined a developer participant evaluation with an expert accessibility review conducted by assistive technology (AT) users. Ten developer participants (DEV001–DEV010) validated the AI's detection output, explanation quality, and fix appropriateness against ground-truth accessibility test files from 19 May to 2 June 2026. In parallel, two AT expert users evaluated both the defective and improved versions of the validation website to provide participant-reported NASA-TLX workload scores for comparison against AI-predicted estimates. Full participant information and consent documentation is provided in Appendix G.

Table 4.3 presents the demographic profile of the ten developer participants.

| ID | Experience | Accessibility Level | Primary Area | Test File(s) |
|---|---|---|---|---|
| DEV001 | 6–10 yrs | Advanced | Full-stack | HTML, CSS, JS, TSX |
| DEV002 | 6–10 yrs | Basic | Backend | HTML |
| DEV003 | >10 yrs | Professional | User Interface (UI) / User Experience (UX) | CSS |
| DEV004 | 3–5 yrs | Intermediate | UI/UX | CSS |
| DEV005 | >10 yrs | Intermediate | Full-stack | HTML, CSS, JS, TSX |
| DEV006 | 3–5 yrs | Advanced | Backend | HTML, CSS, JS, TSX |
| DEV007 | 6–10 yrs | Advanced | Full-stack | TSX |
| DEV008 | >10 yrs | Professional | Full-stack | JS, TSX |
| DEV009 | 3–5 yrs | Basic | Full-stack | HTML |
| DEV010 | 1–2 yrs | None | Backend | JS |

***Table 4.3:*** *Study 5 Developer Participant Demographics*

The pool spanned one to ten-plus years of experience, multiple development backgrounds, and accessibility experience from none to professional level, representing developers likely to encounter EAA compliance requirements [2].

### *4.3.1 Expert Review Findings*

The expert review component of Study 5 used a multimodal usability protocol (Annex 3) in which two AT users evaluated both the defective and improved versions of the study website. Both participants used screen readers as their primary access technology. Participant A used JAWS on Microsoft Edge; Participant B used VoiceOver on Safari. Both evaluated the defective version first, followed by the improved version, in a fixed single-order design. Seven standardised tasks were completed for each website version, covering pricing information retrieval, form submission, team identification, FAQ navigation, blog reading, button activation, and carousel interaction. Table 4.3.1 summarises the task structure and relevant observations from each session.

| No. | Task Description | Expert Observations (Defective Site) |
|---|---|---|
| 1 | Identify the monthly price and user limit of the Pro plan in the Pricing section | Participant A: first pricing section did not render with screen reader; Participant B: completed with difficulty |
| 2 | Fill in the Contact Us form (email, name, message) and submit | Participant A: form option had no text description or ARIA labels; options announced without context |
| 3 | Identify the Chief Technology Officer from the Meet the Team section | Both participants: team card images repeated the graphic name rather than the person's role |
| 4 | Open one FAQ item and read its answer | Participant B: unable to locate FAQ section without assistance; required help navigating to it |
| 5 | Read the Blog article related to accessibility or low-vision users | Both participants: completed; no significant barrier reported |
| 6 | Activate the "Get Started Today" button | Both participants: completed successfully |
| 7 | Advance to the next testimonial using the carousel buttons | Participant A: menuButton `aria-expanded` was reset to `false` on initialisation and never updated on click, causing it to always announce the menu as "collapsed"; pressing Space silenced the screen reader |

***Table 4.3.1:*** *Study 5 Expert Review: Task Observations on the Defective Website*

Several of the barriers observed during the expert review correspond directly to accessibility issues that the AI Accessibility Assistant detected in the source-code test files. Specifically, the `aria-expanded` not updating on click (Task 7) maps to the AI-detected issue in the JS test file at line 18, and the missing ARIA labels on form options (Task 2) aligns with the form-label issues detected in the HTML test file. This convergence between AI source-code detections and AT-user barriers supports the relevance of the detected issues to real assistive technology usage, consistent with the finding of Petrie and Kheir [11] that barriers reported by users with disabilities frequently correspond to violations detectable through guideline analysis.

On the improved website, both participants reported a substantially better experience, noting improved heading structure, clearer organisation, and easier navigation. Both completed all seven tasks without assistance, in contrast to the defective version.

### *4.3.2 Detection and Re-run Results*

Each participant ran the AI Accessibility Assistant on one or more test files, reviewed the detected issues, applied suggested fixes where appropriate, and conducted a re-run to verify resolution. All results are compared against the 30 planted ground-truth issues per file.

#### *4.3.2.1 Per-Participant Detection Results*

Table 4.3.2a presents the per-participant detection statistics.

| ID | File | Detected /30 | FP | FN | Recall | Precision | F1 | New Detections on Re-run |
|---|---|---|---|---|---|---|---|---|
| DEV001 | HTML | 14 | 0 | 16 | 46.7% | 100% | 0.64 | 4 |
| DEV001 | CSS | 5 | 0 | 25 | 16.7% | 100% | 0.29 | 2 |
| DEV001 | JS | 7 | 8 | 23 | 23.3% | 46.7% | 0.31 | 2 |
| DEV001 | TSX | 5 | 0 | 25 | 16.7% | 100% | 0.29 | 4 |
| DEV002 | HTML | 13 | 0 | 17 | 43.3% | 100% | 0.61 | 4 |
| DEV003 | CSS | 6 | 0 | 24 | 20.0% | 100% | 0.33 | 0 |
| DEV004 | CSS | 6 | 0 | 24 | 20.0% | 100% | 0.33 | 3 |
| DEV005 | HTML | 14 | 0 | 16 | 46.7% | 100% | 0.64 | 3 |
| DEV005 | CSS | 5 | 0 | 25 | 16.7% | 100% | 0.29 | 0 |
| DEV005 | JS | 14 | 3 | 16 | 46.7% | 82.4% | 0.60 | 0 |
| DEV005 | TSX | 3 | 6 | 27 | 10.0% | 33.3% | 0.15 | 2 |
| DEV006 | HTML | 15 | 0 | 15 | 50.0% | 100% | 0.67 | 3 |
| DEV006 | CSS | 0 | 0 | 30 | 0.0% | n/a | 0.00 | 2 |
| DEV006 | JS | 5 | 6 | 25 | 16.7% | 45.5% | 0.24 | 3 |
| DEV006 | TSX | 7 | 6 | 23 | 23.3% | 53.8% | 0.33 | 5 |
| DEV007 | TSX | 3 | 6 | 27 | 10.0% | 33.3% | 0.15 | 3 |
| DEV008 | JS | 7 | 6 | 23 | 23.3% | 53.8% | 0.33 | 2 |
| DEV008 | TSX | 1 | 1 | 29 | 3.3% | 50.0% | 0.06 | 0 |
| DEV009 | HTML | 15 | 0 | 15 | 50.0% | 100% | 0.67 | 0 |
| DEV010 | JS | 7 | 8 | 23 | 23.3% | 46.7% | 0.31 | 1 |

***Table 4.3.2a:*** *Study 5: Per-Participant Detection Results by File Type*

Detection rates varied considerably across participants and file types. HTML detection ranged from 43.3% (DEV002) to 50.0% (DEV006, DEV009). JavaScript detection ranged from 16.7% (DEV006) to 46.7% (DEV005). TSX detection was notably lower, ranging from 3.3% (DEV008) to 23.3% (DEV006). CSS detection was the weakest file type, with individual recall values between 0.0% (DEV006)

and 20.0% (DEV003, DEV004).

Precision varied considerably across participants on JavaScript and TSX file types. False positives were recorded for DEV001 (JS: 8 FP), DEV005 (JS: 3 FP; TSX: 6 FP), DEV006 (JS: 6 FP; TSX: 6 FP), DEV007 (TSX: 6 FP), DEV008 (JS: 6 FP; TSX: 1 FP), and DEV010 (JS: 8 FP). The predominant pattern across JS false positives was the model flagging ARIA attributes or event-handler patterns that were already correctly implemented or not required by WCAG at those locations. TSX false positives similarly involved ARIA state attributes that were conditionally managed at runtime in ways not resolvable through static analysis alone. HTML and CSS participants recorded zero false positives.

#### *4.3.2.2 Aggregate Detection Metrics by File Type*

Table 4.3.2b presents the aggregate detection metrics across all participants for each file type, and Figure 4.3.2a visualises these results.

| File Type | Participants | TP | FP | FN | Recall | Precision | F1 |
|---|---|---|---|---|---|---|---|
| HTML | 5 | 85 | 0 | 65 | 56.7% | 100.0% | 0.724 |
| CSS | 5 | 29 | 0 | 121 | 19.3% | 100.0% | 0.324 |
| JS | 5 | 48 | 31 | 102 | 32.0% | 60.8% | 0.419 |
| TSX | 5 | 33 | 19 | 117 | 22.0% | 63.5% | 0.327 |
| **Overall** | **10** | **195** | **50** | **405** | **32.5%** | **79.6%** | **0.462** |

***Table 4.3.2b:*** *Study 5: Aggregate Detection Metrics by File Type (combined session: first run and re-run)*

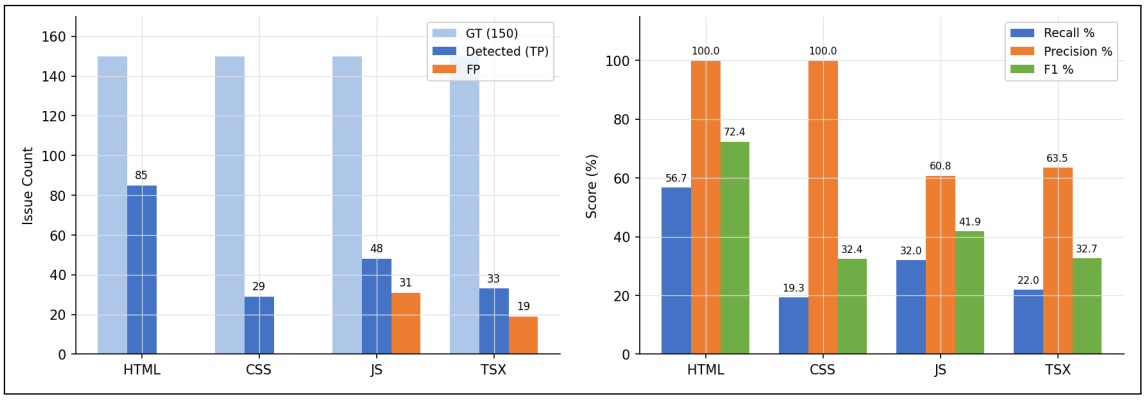



***Figure 4.3.2a:*** *Study 5: Detection Counts (left) and Precision, Recall, and F1 (right) by File Type*

Across 600 ground-truth issue instances (150 per file type, five participants each), the AI detected 195 issues across both the first run and the re-run (32.5% combined-session recall) with 50 false positives (79.6% precision) and an overall F1 score of 0.462. HTML achieved the highest recall (56.7%, F1=0.724), suggesting that the AI's semantic understanding is most effective for HTML structure, where WCAG-relevant patterns such as missing `alt` attributes, absent `<label>` associations, and non-descriptive link text have clear syntactic signatures. CSS recall was the lowest (19.3%, F1=0.324), reflecting the difficulty of detecting accessibility-relevant style rules such as focus indicators using only `:focus` instead of `:focus-visible`, colour-contrast failures at statically resolvable values, and layout patterns that affect AT behaviour.

#### *4.3.2.3 Clean Test file Results*

Clean counterpart files were run alongside defective files by each participant for their chosen file types. Participants testing only HTML (DEV002, DEV009) and CSS only (DEV004) recorded zero findings on their clean files. However, a number of participants received findings on clean JS, CSS, and TSX files: DEV001 (JS: 3 FP), DEV003 (CSS: 1 FP), DEV005 (CSS: 1 FP), DEV006 (JS: 4 FP; TSX: 2 FP), DEV007 (TSX: 1 FP), DEV008 (JS: 6 FP; TSX: 5 FP), and DEV010 (JS: 1 FP). All 24 clean-file findings were individually audited and confirmed as false positives: the model flagged ARIA attribute patterns and event-handler configurations that were already correctly implemented in the clean code or not required by WCAG at those locations. Clean-file HTML recorded zero false positives across all participants. These clean-file FPs represent a specificity concern distinct from the defective-file precision figures reported in Section 4.3.2.2: they reflect the model's tendency to produce spurious ARIA and event-handler detections in compliant JS and TSX code, consistent with the false-positive pattern observed on the defective test files [34].

#### *4.3.2.4 Re-run Results*

After applying the AI's suggested fixes, participants re-ran the analysis on the same file to confirm whether the fixed issues re-appeared and whether any additional issues were detected. Table 4.3.2c summarises the first-run detections and the number of new issues detected on re-run per file type.

| File Type | Detected (1st Run) | Fixed Issues Re-appeared | New Detections on Re-run |
|---|---|---|---|
| HTML | 71 | 0 | 14 |
| CSS | 22 | 0 | 7 |
| JS | 40 | 0 | 8 |
| TSX | 19 | 0 | 14 |
| **Total** | **152** | **0** | **43** |

***Table 4.3.2c:*** *Study 5: Re-run Summary: Fixed Issue Re-appearances and New Detections*

No issue that had been fixed after the first run re-appeared on re-run across any file type, confirming that the applied fixes held. The 43 new detections on re-run were ground-truth issues from the same file that the AI had not flagged in the first run; these are included in the combined-session figures in Table 4.3.2b.

#### *4.3.2.5 Survey Responses and Qualitative Feedback*

Upon completing their session, each participant submitted a structured survey rating the AI's detection accuracy, explanation quality, fix appropriateness, and usefulness. Table 4.3.2d summarises the Likert-scale responses. Figure 4.3.2b visualises the distribution of responses.

| ID | Accuracy | Explanation | Fix Quality | Resolution | Would Use |
|---|---|---|---|---|---|
| DEV001 | Accurate | Accurate | Very accurate | All/nearly all | Strongly agree |
| DEV002 | Accurate | Very accurate | Very accurate | All/nearly all | Strongly agree |
| DEV003 | Accurate | Accurate | Very accurate | Most | Strongly disagree |
| DEV004 | Accurate | Accurate | Accurate | All/nearly all | Strongly agree |
| DEV005 | Accurate | Accurate | Neutral | Some | Agree |
| DEV006 | Accurate | Accurate | Very accurate | All/nearly all | Strongly agree |
| DEV007 | Accurate | Accurate | Accurate | Few | Agree |
| DEV008 | Very accurate | Very accurate | Very accurate | All/nearly all | Strongly agree |
| DEV009 | Accurate | Accurate | Neutral | All/nearly all | Agree |
| DEV010 | Neutral | Very accurate | Very accurate | All/nearly all | Strongly agree |

***Table 4.3.2d:*** *Study 5: Participant Survey Responses*

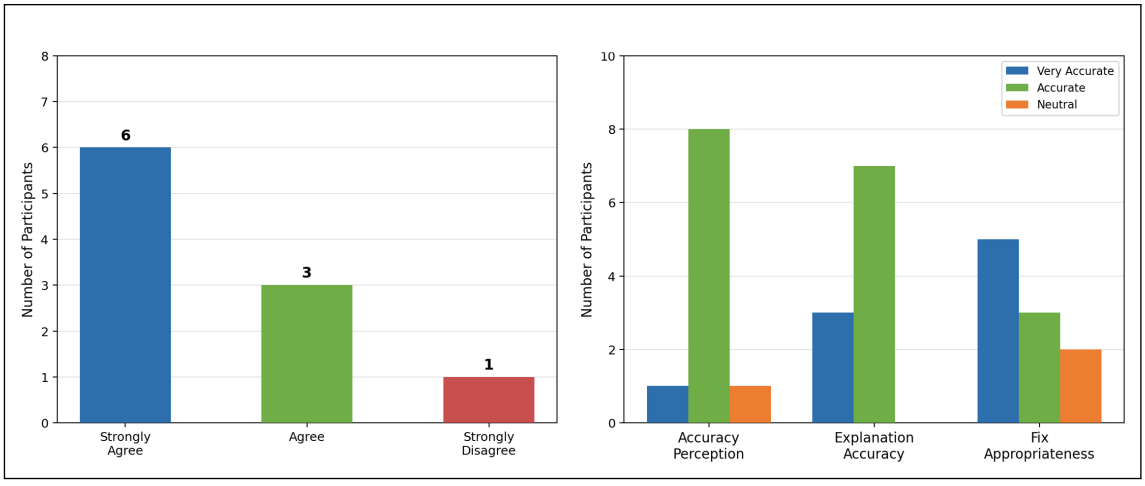



***Figure 4.3.2b:*** *Study 5: Survey Response Distribution: Workflow Usefulness (left) and Quality Ratings (right)*

Nine of the ten participants (90%) rated detection accuracy as accurate or very accurate; all ten (100%) rated explanation quality as accurate or very accurate. Fix appropriateness was very accurate for six participants, accurate for two, and neutral for two. Workflow usefulness gave a 90% agreement rate (six strongly agreed, three agreed, one strongly disagreed).

The sole strongly-disagreeing participant (DEV003) preferred error-block highlighting over per-line underlining and noted that code editing can shift detected error positions, a known VS Code static-analysis limitation, but one that highlights a usability gap for expert users. Commonly noted weaknesses were latency and incomplete coverage on complex files; several participants suggested pairing the tool with a rule-based parser, consistent with Study 6 complementarity findings [36].

Figure 4.3.2c shows the per-participant detection rate across each participant's chosen file types.

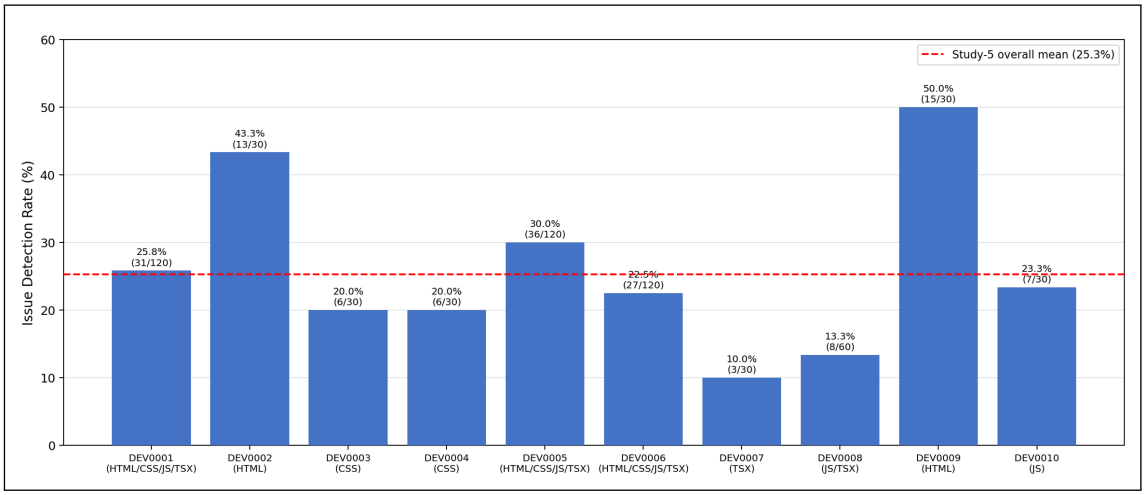



***Figure 4.3.2c:*** *Study 5: Per-Participant Issue Detection Rate Across Chosen File Types*

### *4.3.3 Participant NASA-TLX Results*

NASA-TLX workload scores were collected from the two AT expert users (Participants A and B) immediately after each website condition, allowing a direct within-participant workload comparison. Table 4.3.3 presents the subscale and overall scores for each participant and condition.

| **Dimension** | **Part. A (Defective)** | **Part. A (Improved)** | **Part. B (Defective)** | **Part. B (Improved)** |
|---|---|---|---|---|
| Mental Demand | 70 | 25 | 85 | 15 |
| Physical Demand | 10 | 10 | 10 | 10 |
| Temporal Demand | 25 | 10 | 25 | 10 |
| Performance | 65 | 5 | 55 | 0 |
| Effort | 50 | 10 | 60 | 20 |
| Frustration | 80 | 10 | 85 | 20 |
| **Overall (mean)** | **50.0** | **11.7** | **53.3** | **12.5** |

***Table 4.3.3:*** *Study 5: Participant-Reported NASA-TLX Scores by Condition and Subscale*

*Note: Higher scores indicate greater workload across all subscales. For Performance, higher scores reflect greater perceived task failure; scores of 5 and 0 on the improved website indicate near-perfect task success.*

Both participants reported substantially higher workload on the defective website than on the improved website across all subscales. The largest differences were observed on Mental Demand (Participant A: $70 \rightarrow 25$; Participant B: $85 \rightarrow 15$), Frustration (A: $80 \rightarrow 10$; B: $85 \rightarrow 20$), and Effort (A: $50 \rightarrow 10$; B: $60 \rightarrow 20$). Physical Demand was consistently low across both conditions (10/100), as screen reader navigation does not impose substantially different physical demands between accessible and inaccessible content.

### *4.3.4 Predicted versus Reported Workload*

A central research objective of this study was to examine whether the AI-derived NASA-TLX workload estimate, generated by the model from source-code analysis of the defective and improved HTML test file, aligns with participant-reported workload scores. Tables 4.3.4a and 4.3.4b present the AI-predicted scores alongside the participant-reported scores for the defective and improved conditions respectively.

| **Dimension** | **AI Predicted** | **Part. Mean** | **Difference** |
|---|---|---|---|
| Mental Demand | 80 | 77.5 | +2.5 |
| Physical Demand | 35 | 10.0 | +25.0 |
| Temporal Demand | 15 | 25.0 | −10.0 |
| Performance | 75 | 60.0 | +15.0 |
| Effort | 78 | 55.0 | +23.0 |
| Frustration | 82 | 82.5 | −0.5 |
| **Overall** | **60.8** | **51.65** | **+9.15** |

***Table 4.3.4a:*** *Study 5: AI-Predicted vs Participant-Reported NASA-TLX Scores: Defective Website Condition*

| **Dimension** | **AI Predicted** | **Part. Mean** | **Difference** |
|---|---|---|---|
| Mental Demand | 30 | 20.0 | +10.0 |
| Physical Demand | 10 | 10.0 | 0.0 |
| Temporal Demand | 5 | 10.0 | −5.0 |
| Performance | 10 | 2.5 | +7.5 |
| Effort | 15 | 15.0 | 0.0 |
| Frustration | 10 | 15.0 | −5.0 |
| **Overall** | **13.3** | **12.1** | **+1.2** |

***Table 4.3.4b:*** *Study 5: AI-Predicted vs Participant-Reported NASA-TLX Scores: Improved Website Condition*

Figures 4.3.4a and 4.3.4b show the dimension-by-dimension and overall workload comparisons; Figures 4.3.4c and 4.3.4d show the actual AI-generated NASA-TLX output.

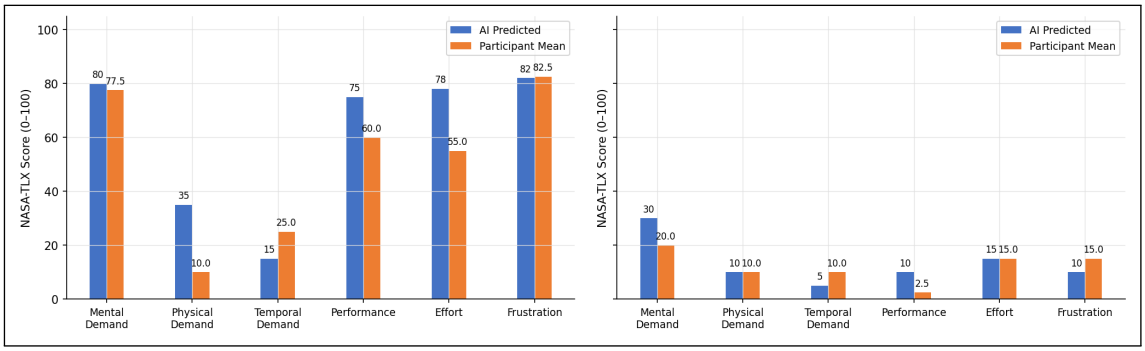



***Figure 4.3.4a:*** *Study 5: NASA-TLX AI Predicted vs Participant Reported Scores by Dimension and Condition*

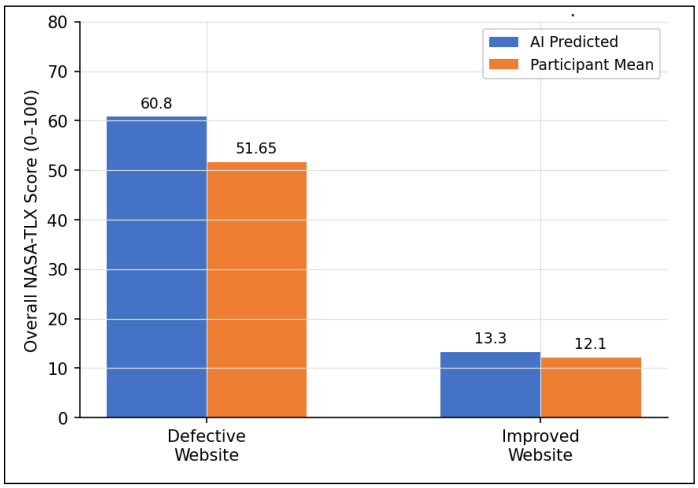



***Figure 4.3.4b:*** *Study 5: Overall NASA-TLX Workload: AI Predicted vs Participant Mean*

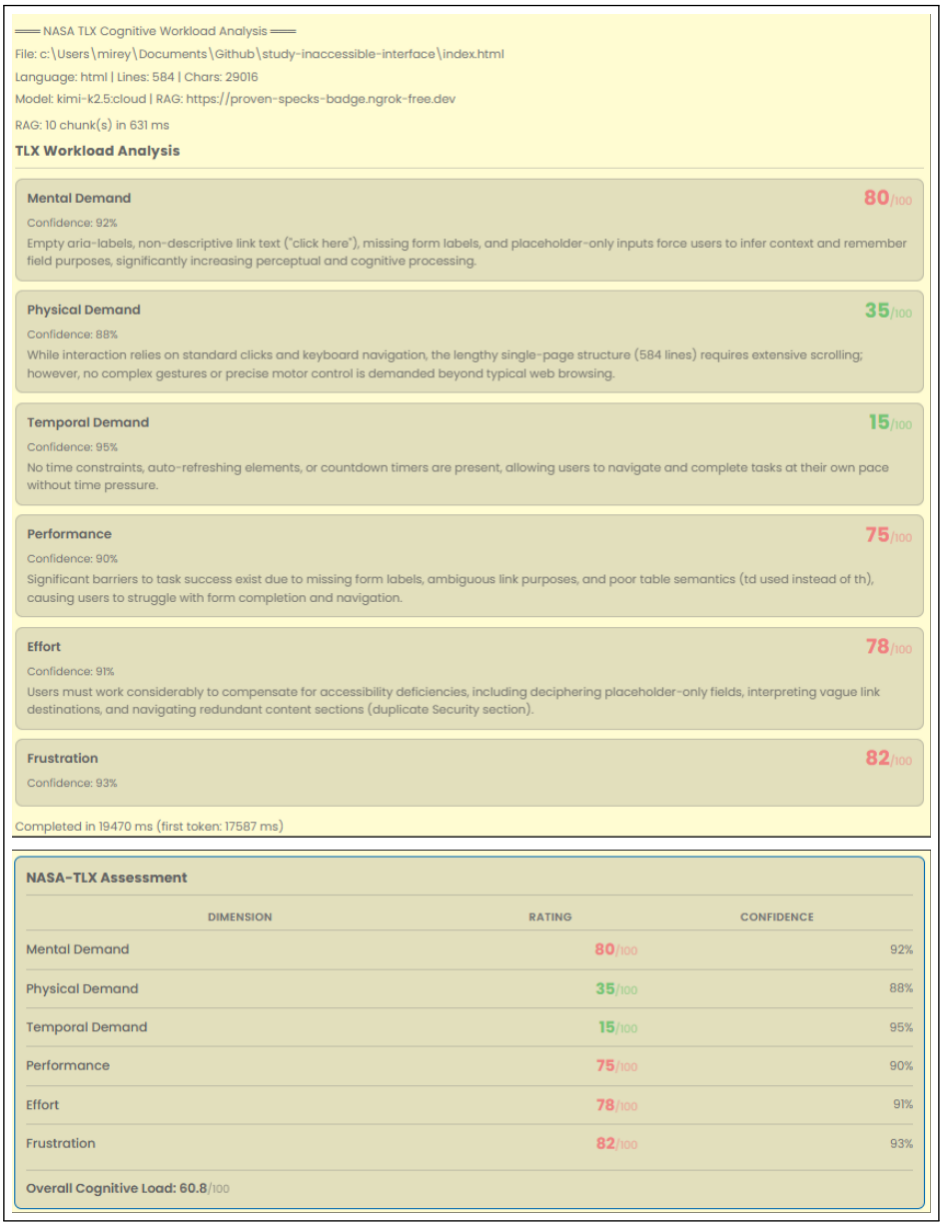



***Figure 4.3.4c:*** *Study 5: AI-Generated NASA-TLX Output: Inaccessible Website (AI-estimated NASA-TLX workload: 60.8/100). Top: per-dimension workload analysis with reasoning; bottom: summary assessment table.*

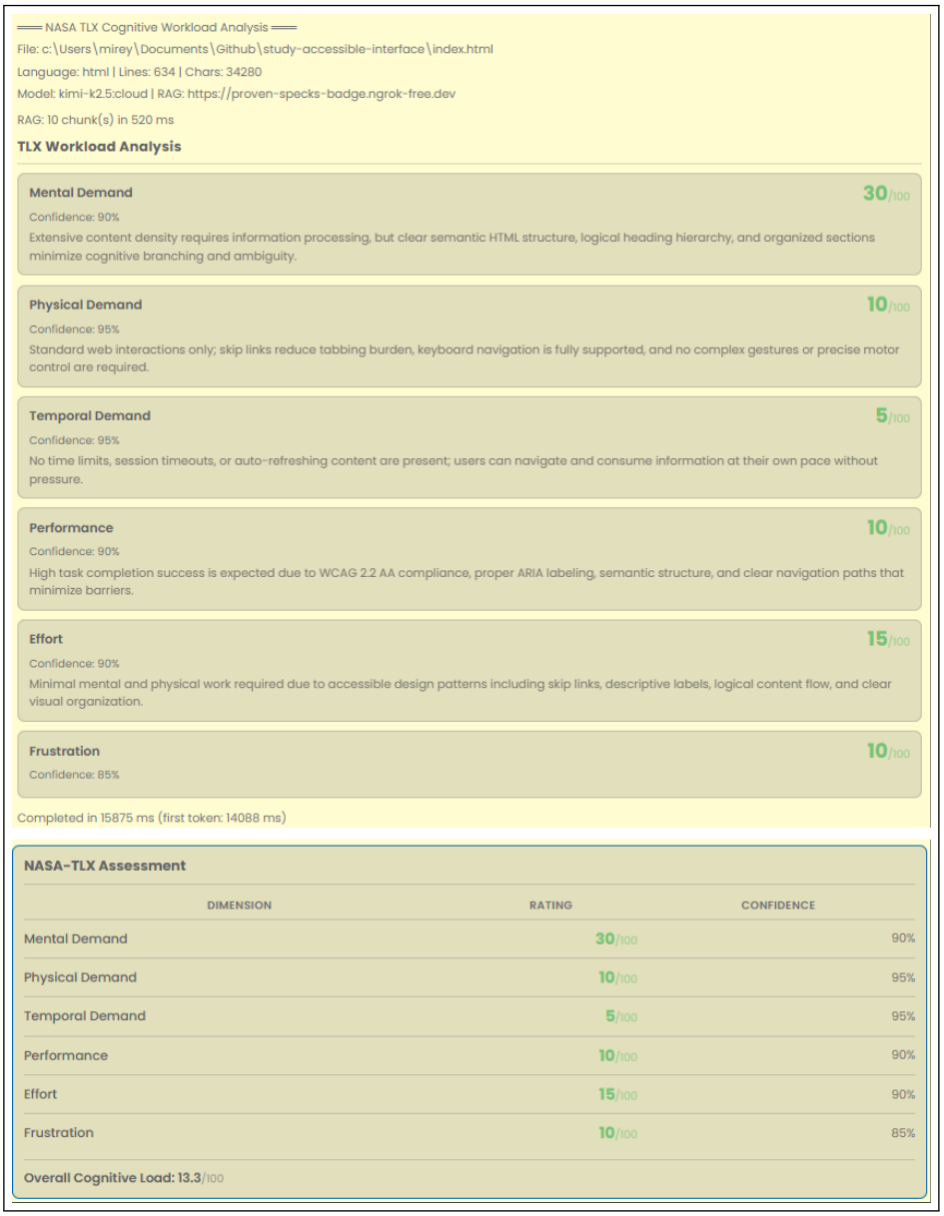



***Figure 4.3.4d:*** *Study 5: AI-Generated NASA-TLX Output: Accessible Website (AI-estimated NASA-TLX workload: 13.3/100). Top: per-dimension workload analysis with reasoning; bottom: summary assessment table.*

On the defective condition, the AI predicted an overall workload of 60.8/100, compared to a participant-reported mean of 51.65/100, an overestimate of 9.15 points (17.7%). On the improved condition, the AI predicted 13.3/100, compared to a participant-reported mean of 12.1/100, an overestimate of 1.2 points (9.9%). Both conditions show the correct directional relationship: the AI assigned higher

workload to the defective site than to the improved site, consistent with the direction of participant-reported scores ($51.65 > 12.1$). The magnitude and distribution of scores differed across subscales.

The most accurate subscale predictions were Frustration (AI=82 vs 82.5) and Mental Demand (AI=80 vs 77.5). Physical Demand was overestimated by +25 points, as the model over-inferred physical burden from page complexity without accounting for screen reader interaction modality. Performance on the improved condition showed close alignment (AI=10 vs participant mean=2.5): both the model and participants returned near-zero scores, indicating near-perfect task success (low workload from performance failure) on the improved site. Consistent with Hart and Staveland [12], the estimate is best interpreted as a directional proxy rather than a precise measurement.

## 4.4 Study 6: Supplementary Tool Comparison Results

Study 6 compared the AI Accessibility Assistant with six established automated accessibility tools across four purpose-built test script files: HTML, CSS, JavaScript, and TSX. Each test script file contained 30 intentionally planted accessibility defects. The 30-defect-per-file target was selected to reflect the empirical accessibility-error density reported by Martins and Duarte in their large-scale analysis of web pages [15]. Results are from the 2 June 2026 run using a dual-model consensus configuration: two LLMs (`kimi-k2.5` and `qwen3.5`) independently analysed each 200-line code chunk; within-model outputs were first deduplicated using title-only Jaccard (threshold 0.40), then cross-model findings were merged using a

title-and-explanation Jaccard score of at least 0.30 with line numbers within ±5. A finding is included if both models agree at that cross-model threshold; single-model survivors are admitted only when they cite a specific WCAG success criterion, carry high or medium severity, and provide a concrete line number. This consensus step suppresses findings that only one model flags, reducing noise at the cost of some recall. Comment stripping was enabled to reduce token overhead.

The purpose was not to produce a single absolute ranking, but to examine how tool coverage varied by artefact type, analysis surface, and evaluation method. This distinction is important because accessibility tools inspect different representations of an interface: source-level linters examine authored code, while rendered-DOM tools such as Lighthouse and axe-core inspect browser-rendered output, explaining the absence of finding-level overlap between tool categories.

A distinction is drawn between raw findings and ground-truth recall. Raw findings refer to the total number of issues reported by each tool. Ground-truth recall refers to how many of the 30 planted defects were detected. Automated tools may report more than 30 findings when they count repeated element-level violations, incidental issues, or overlapping rules for the same defect category; high raw finding counts therefore do not automatically indicate stronger accuracy.

For classification metrics, each tool's output was evaluated against the 30 planted Ground Truth (GT) defects per test script file. Findings were classified into: (1) **GT-matched TP**: the finding matches a planted GT defect; (2) **additional genuine TP**: a real accessibility issue confirmed by source inspection that was not planted; (3) **false positive (FP)**: no corresponding real accessibility issue in the

code. GT-matched TPs and additional genuine TPs are tracked separately because they measure different things: GT-matched TPs measure coverage of a known, controlled defect set and allow direct cross-tool comparison; additional genuine TPs measure whether the tool surfaces real problems beyond the planted set, which matters for practical utility. Merging them would inflate GT Recall figures and obscure the distinction. **Accuracy** is Total TP / Raw Findings. **GT Recall** is GT-matched TP / 30. **F1** is the harmonic mean of Accuracy and GT Recall.

### *4.4.1 Findings by File Type*

Table 4.4.1a summarises the raw findings reported by each tool across the four file types. Because not every tool was applicable to every test script file, results are presented by file type. Lighthouse and axe-core are rendered-DOM audit tools; Axe Linter applies a subset of axe-core rules statically to source files; ESLint jsx-a11y handles JSX accessibility linting; Webhint provides configuration and accessibility hinting; stylelint-a11y handles CSS accessibility linting.

| File | AI Asst. | Lighthouse | axe-core | Axe Linter | ESLint jsx-a11y | Webhint | stylelint-a11y |
|---|---|---|---|---|---|---|---|
| HTML | 11 | 3 | 6 | 6 | N/A | 2 | N/A |
| CSS | 6 | 0 | 0 | 0 | N/A | 0 | 29 |
| JavaScript | 60 | 0 | 0 | 0 | N/A | 1 | N/A |
| TSX | 32 | N/A | N/A | 1 | 1 | 1 | N/A |

***Table 4.4.1a:*** *Study 6: Raw Findings Per Test Script File (2 June 2026 run)*

*Note: Raw counts include all findings regardless of ground-truth match. axe-core and Axe Linter returned 6 HTML findings each, of which 5 were true positives against ground truth. stylelint-a11y returned 29 CSS findings, of which 6 matched planted GT issues.*

Table 4.4.1b shows which tools were applicable to each file type. The AI assistant is the only tool applicable to all four artefact types.

| **Tool** | **HTML** | **CSS** | **JS** | **TSX** |
|---|---|---|---|---|
| AI Assistant | Yes | Yes | Yes | Yes |
| Lighthouse | Yes | Yes (harness) | Yes (harness) | No |
| axe-core | Yes | Yes (harness) | Yes (harness) | No |
| Axe Linter | Yes | Yes | Yes | Yes |
| ESLint jsx-a11y | No | No | No | Yes |
| Webhint | Yes | Yes | Yes | Yes |
| stylelint-a11y | No | Yes | No | No |

***Table 4.4.1b:*** *Study 6: Tool Applicability Matrix for Test Script Files*

#### *4.4.1.1 HTML*

HTML was the only file type for which multiple tools produced non-zero findings; full per-tool metrics are in Table 4.4.1c. The AI assistant reported 11 findings (6 GT-TP, 2 additional genuine, 3 FP; F1=0.314), Lighthouse 3 (GT-TP=3, FP=0; F1=0.182), axe-core and Axe Linter 6 each (GT-TP=5, FP=1; F1=0.280), and Webhint 2 (GT-TP=1, FP=1; F1=0.062).

| **Tool** | **GT-TP** | **Extra TP** | **Total TP** | **FP** | **Raw** | **Accuracy** | **GT Recall** | **F1** |
|---|---|---|---|---|---|---|---|---|
| AI Assistant | 6 | 2 | 8 | 3 | 11 | 0.727 | 0.200 | 0.314 |
| Lighthouse | 3 | 0 | 3 | 0 | 3 | 1.000 | 0.100 | 0.182 |
| axe-core | 5 | 0 | 5 | 1 | 6 | 0.833 | 0.167 | 0.280 |
| Axe Linter | 5 | 0 | 5 | 1 | 6 | 0.833 | 0.167 | 0.280 |
| ESLint jsx-a11y | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |
| Webhint | 1 | 0 | 1 | 1 | 2 | 0.500 | 0.033 | 0.062 |
| stylelint-a11y | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |

***Table 4.4.1c:*** *Study 6: HTML Test Script File Classification Metrics (30 GT defects)*

*Note: The 1 axe-core and 1 Axe Linter FP was a duplicate rule firing (*`landmark-no-duplicate-banner`* on the same planted defect covered by* `landmark-unique`*).*

*The 3 AI FPs were carousel navigation buttons flagged as missing* `aria-controls`*, a toggle button missing* `aria-expanded`*, and a broken* `aria-labelledby` *reference, none of which were evidenced in the source on inspection. Webhint's 1 FP was an autocomplete suggestion not corresponding to a planted defect.*

Despite multiple tools producing findings on HTML, zero finding-level overlap was observed: no two tools detected the same individual planted issue. This is consistent with Vigo et al.'s finding that different automated tools detect non-overlapping subsets of accessibility issues [58] and Frazão and Duarte's observation that tool agreement rates are below 15% on identical pages [59].

#### *4.4.1.2 CSS*

In the CSS test script file, the AI assistant reported 6 findings. stylelint-a11y returned 29 findings. Lighthouse, axe-core, Axe Linter, and Webhint all returned 0. ESLint jsx-a11y is not applicable to CSS.

Of the AI's 6 findings, 1 matched a planted GT issue (GT#11: the backwards `prefers-reduced-motion` logic). Upon source inspection, 2 additional non-GT AI findings were confirmed as genuine accessibility issues in the code: link text distinguished only by colour removal (a WCAG 1.4.1 violation at the base link state), and `color: #fff !important` on social links overriding user stylesheets. Three findings were false positives: a forced-colors media query override that already exists correctly in the source, a `.search-icon` touch target at 16×16 px that meets the WCAG 2.5.8 minimum, and a focus indicator inside `@media (prefers-reduced-motion: reduce)` that is correctly preserved. Total TP = 3, FP = 3, Accuracy = 0.500, GT-Recall = 0.033, F1 = 0.062.

stylelint-a11y returned 29 raw findings: 6 GT-matched and 23 rule-scope over-fires across `a11y/no-display-none` and `a11y/media-prefers-reduced-motion` patterns (Accuracy = 0.207, GT-Recall = 0.200, F1 = 0.203). The CSS result is the starkest finding of Study 6: no tool achieved GT Recall above 20% or Accuracy above 21% against the planted CSS defect set. The planted CSS issues targeted context-dependent composite patterns (`:focus` versus `:focus-visible`, font-size insufficiency at specific breakpoints, backwards reduced-motion logic, `user-select` blocking, and CSS-only state management without ARIA synchronisation) rather than single-property violations detectable by static rules. The AI's failure to reach higher recall under the strict dual-model consensus reflects that many CSS accessibility issues require visual rendering context to assess reliably. The stylelint-a11y false-positive rate (23/29 = 79%) is consistent with prior observations that rule-based tools generate high false-positive rates when their rules fire beyond their calibrated scope [34].

| **Tool** | **GT-TP** | **Extra TP** | **Total TP** | **FP** | **Raw** | **Accuracy** | **GT Recall** | **F1** |
|---|---|---|---|---|---|---|---|---|
| AI Assistant | 1 | 2 | 3 | 3 | 6 | 0.500 | 0.033 | 0.062 |
| Lighthouse | 0 | 0 | 0 | 0 | 0 | N/A | 0.000 | N/A |
| axe-core | 0 | 0 | 0 | 0 | 0 | N/A | 0.000 | N/A |
| Axe Linter | 0 | 0 | 0 | 0 | 0 | N/A | 0.000 | N/A |
| ESLint jsx-a11y | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |
| Webhint | 0 | 0 | 0 | 0 | 0 | N/A | 0.000 | N/A |
| stylelint-a11y | 6 | 0 | 6 | 23 | 29 | 0.207 | 0.200 | 0.203 |

***Table 4.4.1d:*** *Study 6: CSS Test Script File Classification Metrics (30 GT defects)*

*Note: stylelint-a11y Extra TP shows 0 because non-GT findings were not individually source-audited; the 23 non-GT findings are treated as FP by default.*

#### *4.4.1.3 JavaScript*

In the JavaScript test script file, the AI assistant returned 60 findings, by far the highest raw output of any tool on any test script file. Webhint returned 1 finding (a colour contrast check, not matching any planted GT defect). Lighthouse, axe-core, Axe Linter, and stylelint-a11y all returned 0. ESLint jsx-a11y is not applicable to plain JavaScript.

Of the 60 AI findings, 15 matched planted ground-truth defects. Source-code audit of the remaining 45 found that approximately 36 were genuine accessibility issues present in the code (unintentional errors in the test script file beyond the planted set) and approximately 9 were actual false positives with no corresponding code evidence. The 15 GT-matched TPs included ARIA live-region omissions, `aria-expanded` state mismanagement, focus-trap and keyboard accessibility gaps, reduced-motion non-compliance, and unannounced dynamic state changes. Total TP = 51, FP = 9, Accuracy = 0.850, GT-Recall = 0.500, F1 = 0.630.

| **Tool** | **GT-TP** | **Extra TP** | **Total TP** | **FP** | **Raw** | **Accuracy** | **GT Recall** | **F1** |
|---|---|---|---|---|---|---|---|---|
| AI Assistant | 15 | 36 | 51 | 9 | 60 | 0.850 | 0.500 | 0.630 |
| Lighthouse | 0 | 0 | 0 | 0 | 0 | N/A | 0.000 | N/A |
| axe-core | 0 | 0 | 0 | 0 | 0 | N/A | 0.000 | N/A |
| Axe Linter | 0 | 0 | 0 | 0 | 0 | N/A | 0.000 | N/A |
| ESLint jsx-a11y | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |
| Webhint | 0 | 0 | 0 | 1 | 1 | 0.000 | 0.000 | N/A |
| stylelint-a11y | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |

***Table 4.4.1e:*** *Study 6: JavaScript Test Script File Classification Metrics (30 GT defects)*

The AI assistant achieved 85% Accuracy (51/60 findings genuine) and 50% GT Recall, the highest of any metric for any tool on any test script file in

Study 6. This supports the finding that LLM-based tools can surface behavioural and state-management accessibility issues that were not detected by the deterministic tools in this setup [29, 38].

#### *4.4.1.4 TSX*

In the TSX test script file, the AI assistant returned 32 findings. ESLint jsx-a11y returned 1. Axe Linter returned 1. Webhint returned 1. Lighthouse and axe-core were excluded because TSX is source-level code and no rendered browser runtime was used for this test script file.

Of the AI's 32 findings, 17 matched planted ground-truth defects. Source-code audit of the remaining 15 found approximately 12 to be genuine accessibility issues and approximately 3 to be actual false positives. The 17 GT-matched TPs covered div-as-button and keyboard-semantic patterns, `aria-expanded`/`aria-haspopup` omissions, focus-trap gaps in DatePicker and modal components, live-region omissions in stepper and progress bar, combobox state-announcement gaps, and `prefers-reduced-motion` non-compliance. The 12 additional genuine findings included tooltip, modal, autocomplete, and listbox ARIA gaps. Total TP = 29, FP = 3, Accuracy = 0.906, GT-Recall = 0.567, F1 = 0.697.

ESLint jsx-a11y, Axe Linter, and Webhint each returned 1 finding; metrics are reported in Table 4.4.1f.

| Tool | GT-TP | Extra TP | Total TP | FP | Raw | Accuracy | GT Recall | F1 |
|---|---|---|---|---|---|---|---|---|
| AI Assistant | 17 | 12 | 29 | 3 | 32 | 0.906 | 0.567 | 0.697 |
| Lighthouse | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |
| axe-core | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |
| Axe Linter | 1 | 0 | 1 | 0 | 1 | 1.000 | 0.033 | 0.064 |
| ESLint jsx-a11y | 1 | 0 | 1 | 0 | 1 | 1.000 | 0.033 | 0.064 |
| Webhint | 0 | 0 | 0 | 1 | 1 | 0.000 | 0.000 | N/A |
| stylelint-a11y | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |

***Table 4.4.1f:*** *Study 6: TSX Test Script File Classification Metrics (30 GT defects)*

The TSX result is the AI assistant's strongest file by Accuracy (0.906), with GT Recall of 56.7%, the highest among non-JS files. Rule-based linters' 3.3% GT Recall versus the AI's 56.7% confirms the limited scope of deterministic tools for ARIA relationship errors and state-linked focus management [29].

### *4.4.2 Aggregate Results*

Table 4.4.2a reports total findings by tool across all applicable test script files.

| Tool | Total Raw | Applicable Files | Avg per File |
|---|---|---|---|
| AI Assistant | 109 | 4 | 27.25 |
| Lighthouse | 3 | 3 | 1.00 |
| axe-core | 6 | 3 | 2.00 |
| Axe Linter | 7 | 4 | 1.75 |
| ESLint jsx-a11y | 1 | 1 | 1.00 |
| Webhint | 4 | 4 | 1.00 |
| stylelint-a11y | 29 | 1 | 29.00 |

***Table 4.4.2a:*** *Study 6: Total Raw Findings by Tool Across All Test Script Files*

Table 4.4.2b provides full classification metrics for every tool on every applicable test script file. AI findings were individually source-audited and split into GT-TP, Extra TP, and FP. Other tools show GT-TP only; Extra TP for non-AI

tools was not separately audited. Accuracy = Total TP / Raw. GT Recall = GT-TP / 30. F1 = harmonic mean of Accuracy and GT Recall.

| File | Tool | GT-TP | Extra TP | Total TP | FP | Raw | Accuracy | GT Recall | F1 |
|---|---|---|---|---|---|---|---|---|---|
| HTML | AI Assistant | 6 | 2 | 8 | 3 | 11 | 0.727 | 0.200 | 0.314 |
| HTML | Lighthouse | 3 | 0 | 3 | 0 | 3 | 1.000 | 0.100 | 0.182 |
| HTML | axe-core | 5 | 0 | 5 | 1 | 6 | 0.833 | 0.167 | 0.280 |
| HTML | Axe Linter | 5 | 0 | 5 | 1 | 6 | 0.833 | 0.167 | 0.280 |
| HTML | ESLint | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |
| HTML | Webhint | 1 | 0 | 1 | 1 | 2 | 0.500 | 0.033 | 0.062 |
| HTML | stylelint | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |
| CSS | AI Assistant | 1 | 2 | 3 | 3 | 6 | 0.500 | 0.033 | 0.062 |
| CSS | Lighthouse | 0 | 0 | 0 | 0 | 0 | N/A | 0.000 | N/A |
| CSS | axe-core | 0 | 0 | 0 | 0 | 0 | N/A | 0.000 | N/A |
| CSS | Axe Linter | 0 | 0 | 0 | 0 | 0 | N/A | 0.000 | N/A |
| CSS | ESLint | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |
| CSS | Webhint | 0 | 0 | 0 | 0 | 0 | N/A | 0.000 | N/A |
| CSS | stylelint | 6 | 0 | 6 | 23 | 29 | 0.207 | 0.200 | 0.203 |
| JS | AI Assistant | 15 | 36 | 51 | 9 | 60 | 0.850 | 0.500 | 0.630 |
| JS | Lighthouse | 0 | 0 | 0 | 0 | 0 | N/A | 0.000 | N/A |
| JS | axe-core | 0 | 0 | 0 | 0 | 0 | N/A | 0.000 | N/A |
| JS | Axe Linter | 0 | 0 | 0 | 0 | 0 | N/A | 0.000 | N/A |
| JS | ESLint | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |
| JS | Webhint | 0 | 0 | 0 | 1 | 1 | 0.000 | 0.000 | N/A |
| JS | stylelint | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |
| TSX | AI Assistant | 17 | 12 | 29 | 3 | 32 | 0.906 | 0.567 | 0.697 |
| TSX | Lighthouse | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |
| TSX | axe-core | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |
| TSX | Axe Linter | 1 | 0 | 1 | 0 | 1 | 1.000 | 0.033 | 0.064 |
| TSX | ESLint | 1 | 0 | 1 | 0 | 1 | 1.000 | 0.033 | 0.064 |
| TSX | Webhint | 0 | 0 | 0 | 1 | 1 | 0.000 | 0.000 | N/A |
| TSX | stylelint | N/A | N/A | N/A | N/A | N/A | N/A | N/A | N/A |

***Table 4.4.2b:*** *Study 6: Full Classification Metrics Per Tool Per Test Script File (AI source-audited; other tools GT-TP only)*

Table 4.4.2c summarises the AI assistant's aggregate performance across all four test script files combined (120 total GT issues, source-audited).

| **Metric** | **Value** |
|---|---|
| Total GT-TP | 39 |
| Total Extra TP (genuine unplanted) | 52 |
| Total TP (GT + genuine) | 91 |
| Total FP (actual hallucinations) | 18 |
| Total Raw Findings | 109 |
| Aggregate Accuracy | 0.835 |
| Aggregate GT Recall | 0.325 |
| Aggregate F1 | 0.468 |

***Table 4.4.2c:*** *Study 6: AI Assistant Aggregate Metrics Across All Four Test Script Files (120 GT issues, source-audited)*

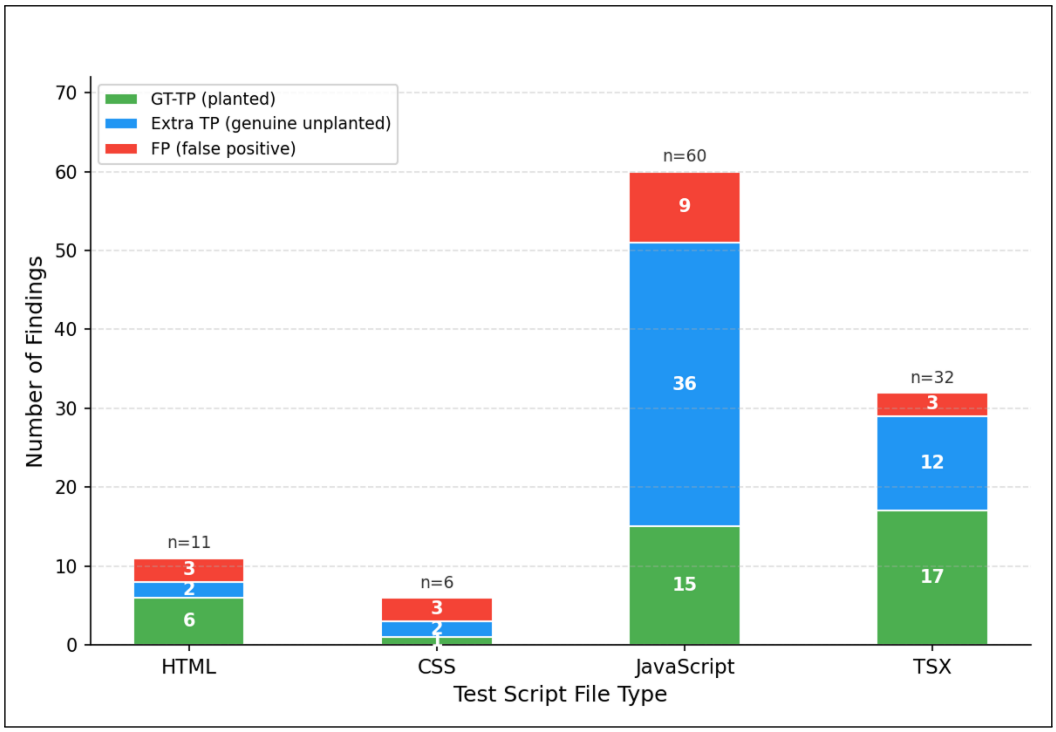



***Figure 4.4.2:*** *Study 6: AI Assistant Findings Breakdown per File Type. Each bar shows the composition of AI findings into GT-TP (planted ground-truth matches, green), Extra TP (genuine unplanted issues, blue), and FP (false positives with no code evidence, red). The total finding count per file is shown above each bar.*

Figure 4.4.2 shows the breakdown of AI findings by file type. Across all 109 findings, 83.5% were genuine (GT or unplanted); only 18 (16.5%) were

actual false positives. The aggregate GT Recall of 32.5% is at the upper end of published baselines for automated accessibility tools [58], and the F1 of 0.468 better characterises real-world utility than raw-findings volume.

### *4.4.3 Ground-Truth Recall Analysis*

Tables 4.4.3a and 4.4.3b summarise GT-TP counts and recall percentages across all tools and file types. Because each test script file contained 30 intentionally planted defects, GT Recall is directly comparable across tools and file types.

| File | AI Asst. | Lighthouse | axe-core | Axe Linter | ESLint jsx-a11y | Webhint | stylelint-a11y |
|---|---|---|---|---|---|---|---|
| HTML | 6 | 3 | 5 | 5 | N/A | 1 | N/A |
| CSS | 1 | 0 | 0 | 0 | N/A | 0 | 6 |
| JavaScript | 15 | 0 | 0 | 0 | N/A | 0 | N/A |
| TSX | 17 | N/A | N/A | 1 | 1 | 0 | N/A |

***Table 4.4.3a:*** *Study 6: True Positive Counts (GT Matches) Out of 30 Planted Defects*

| File | AI Asst. | Lighthouse | axe-core | Axe Linter | ESLint jsx-a11y | Webhint | stylelint-a11y |
|---|---|---|---|---|---|---|---|
| HTML | **20%** | 10% | 17% | 17% | N/A | 3% | N/A |
| CSS | **3%** | 0% | 0% | 0% | N/A | 0% | 20% |
| JavaScript | **50%** | 0% | 0% | 0% | N/A | 0% | N/A |
| TSX | **57%** | N/A | N/A | 3% | 3% | 0% | N/A |

***Table 4.4.3b:*** *Study 6: GT Recall Percentage Out of 30 Planted GT Defects*

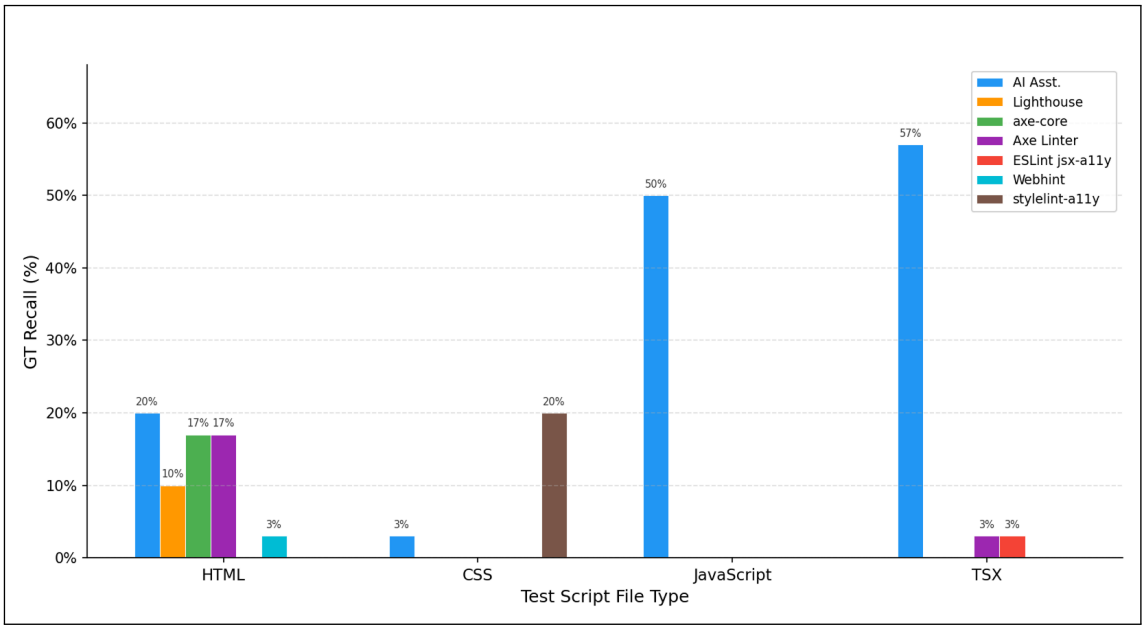



***Figure 4.4.3:*** *Study 6: Ground-Truth Recall by Tool per File Type. Bars show the percentage of 30 planted defects recalled by each tool. Tools not applicable to a given file type are shown as 0% (N/A). The AI assistant provided the broadest cross-artefact coverage in this test setup; deterministic tools remained valuable within their intended operating scopes.*

Figure 4.4.3 plots recall across tools and file types. The results reveal a clear file-by-file pattern. On HTML, the AI (20%) led recall while Lighthouse (10%), axe-core (17%), and Axe Linter (17%) contributed substantial complementary coverage within their rendered-DOM rule sets. On CSS, all tools effectively failed: stylelint-a11y achieved the highest CSS recall (20%) but with a 79% false-positive rate (23/29 findings were FP). On JavaScript, the AI achieved 50% recall while every other applicable tool returned 0%. On TSX, the AI achieved 57% recall versus ESLint jsx-a11y's 3% and Axe Linter's 3%.

The recall gap is most acute for CSS. The planted CSS issues targeted context-dependent composite patterns: `:focus` versus `:focus-visible`, font-size insufficiency at specific breakpoints, backwards reduced-motion logic, `user-select` blocking, and CSS-only state management without ARIA synchronisation, rather

than single-property violations detectable by static rules. All tools combined covered only 6–7 of 30 planted CSS defects, and no approach achieved Accuracy above 21% on this test script file. CSS accessibility checking at source level remains largely a human review task beyond the simplest single-property violations.

These recall figures place the AI assistant at or above the upper end of published baselines. Vigo et al. found that leading automated tools achieved 20–36% recall against comprehensive WCAG 2.x defect sets [58]; the AI assistant's JavaScript recall of 50% and TSX recall of 57% substantially exceed those baselines. These figures must be contextualised against the test script file design: they were constructed to target issue types where deterministic tools have no coverage.

The absence of finding-level overlap across all test script files (0 shared findings) confirms high complementarity between methods. This reinforces a layered evaluation strategy rather than reliance on a single checker, consistent with WCAG-EM guidance on structured multi-method accessibility evaluation [26–28]. GT Recall was highest for the AI assistant in TSX (57%), JavaScript (50%), and HTML (20%). The AI assistant's aggregate Accuracy was 83.5% (91 total TP out of 109 raw findings: 39 GT-matched + 52 genuine unplanted), with aggregate GT Recall of 32.5% and aggregate F1 of 0.468. The most defensible conclusion is complementary use: AI for source-level semantic and behavioural discovery across all artefact types, deterministic rendered-DOM tools for HTML confirmation, specialist linters for file-type-specific rule coverage, and reviewer adjudication before treating findings as confirmed violations.

### *4.4.4 Discussion of Tool Comparison*

#### *4.4.4.1 Raw Counts Versus Ground-Truth Recall*

Raw count alone is a poor proxy for tool quality [58]. The AI assistant's high raw count (109 findings) reflects broad coverage across all source artefact types; 83.5% of those findings are genuine. Full per-file metrics are in Table 4.4.2b.

#### *4.4.4.2 AI Performance Compared to Prior Tool Studies*

The AI assistant's GT Recall of 50% on JavaScript and 57% on TSX exceeds published baselines of 20–36% for leading automated tools [58], with the aggregate Accuracy of 83.5% well above figures from recent LLM-based tool studies [37–39]. The comparison is contextualised in Appendix E.7.

#### *4.4.4.3 Deterministic Tool Performance in Context*

Zero JavaScript and TSX recall from Lighthouse, axe-core, and Axe Linter is consistent with their design scope: these tools cannot analyse behavioural logic or component source without a browser runtime. For HTML rendered-DOM checking they are competitive with the AI (GT-Recall 17% vs 20%). A detailed breakdown is in Appendix E.7.

#### *4.4.4.4 Practical Implications*

The findings support a layered evaluation pipeline: the AI assistant as the primary source-level analyser for JavaScript and TSX, supplemented by Axe Linter and ESLint jsx-a11y for JSX/HTML source checks, stylelint-a11y for CSS, and

Lighthouse/axe-core for rendered-DOM auditing. This strategy aligns with WCAG-EM guidance [26, 28, 58]. A full per-tool breakdown is in Appendix E.7.

#### *4.4.4.5 Threats to Validity*

Key threats include: synthetic error-seeded test files may not reflect real-world distributions; keyword-based GT matching is conservative; Lighthouse and axe-core were not run against TSX; and the dual-model consensus requires both models to produce valid output. A full six-point threat analysis is in Appendix E.7.

Study 6 shows that multi-tool triangulation is stronger than reliance on a single accessibility checker, confirming prior findings from tool evaluation literature [28, 58–60]. The AI assistant achieved the highest GT Recall in JavaScript (50%) (where no other applicable tool returned any GT matches) and TSX (57%), where the nearest alternatives (Axe Linter and ESLint jsx-a11y) each returned a single GT match (3.3% recall). CSS GT Recall was near zero across all tools (0–20%), indicating a systematic gap that no tool adequately addresses. The aggregate AI performance (Accuracy=0.835, GT-Recall=0.325, F1=0.468: 91 total TP from 109 raw findings, of which 39 were planted-GT matches and 52 were genuine unplanted issues) situates the system at the upper end of published baselines for automated tools on complex multi-artefact accessibility evaluation. The main conclusion is that different tools expose different accessibility surfaces, and robust evaluation requires a layered strategy combining AI source-level analysis, rendered-DOM auditing, specialised source linters, and reviewer adjudication, consistent with WCAG-aligned testing principles, ACT-rule consistency, and WCAG-EM guidance.

## 4.5 Cross-Study Synthesis

Three patterns are consistent across all six studies. First, precision is moderate to high: approaching 100% in benchmarking, 79.6% in Study 5 (driven by false positives on JS and TSX), and 83.5% aggregate Accuracy in Study 6. Second, recall is strongly file-type-dependent: HTML consistently highest, CSS near zero for all tools, JavaScript-exclusive in Study 6 (no other tool returned any GT matches in JS); TSX was near-exclusive to the AI assistant, with Axe Linter and ESLint jsx-a11y each returning a single GT match (3.3% recall) against the AI's 56.7%. Third, the NASA-TLX estimation demonstrates consistent directional accuracy across Studies 4 and 5, correctly assigning higher overall workload to more barrier-dense conditions and achieving directional agreement on five of six subscales (Physical Demand overestimated, as the model has no mechanism for inferring screen-reader interaction modality from source code alone). The transition from benchmark aggregation (F1 = 70.5%, `rag-think`, Studies 1–3) to live combined-session participant testing (recall = 32.5%) reflects the structural difference between union-rule multi-run scoring and normal developer workflow execution rather than a model capability failure. Study 6 confirmed that zero finding-level overlap exists across tools, supporting complementary use rather than substitution.

## 4.6 Answers to the Research Questions

*Note: This section provides analytical answers to the three research questions based on the evidence gathered in Studies 1–6. Conclusions, implications, and recommendations are deferred to Chapter 5.*

### *4.6.1 RQ1: Alignment Between AI-Derived and Participant-Reported NASA-TLX Workload Estimates*

**Research Question 1 asks:** To what extent does the accessibility-issue-derived NASA-TLX workload estimate align with participant-reported NASA-TLX workload scores?

The Study 5 comparison shows directional agreement on five of six subscales in both conditions: the AI correctly assigned higher overall workload to the defective site (AI=60.8 vs participant mean=51.65) than to the improved site (AI=13.3 vs 12.1). The closest subscale predictions were Frustration (AI=82 vs 82.5) and Mental Demand (AI=80 vs 77.5), both well-evidenced in source-code patterns. Physical Demand was overestimated by +25 points (failing directional agreement, as participants rated both conditions equally at 10/100), reflecting the limits of static estimation without actual task execution. Consistent with Hart and Staveland [12], the estimate functions as a credible directional proxy but not as a precise predictor of participant-reported workload magnitude.

### *4.6.2 RQ2: Effectiveness of AI-Assisted Source-Code Analysis for WCAG 2.2 Detection and Remediation*

**Research Question 2 asks:** How effectively can AI-assisted source-code analysis support the identification and remediation of WCAG 2.2 accessibility issues?

Study 5 recorded 195 GT detections across 600 instances over both runs: 32.5% combined-session recall, 79.6% precision, F1 = 0.462. HTML achieved the highest recall (56.7%), CSS the lowest (19.3%). Study 6 recorded 32.5% aggregate GT Recall across 120 planted defects with 83.5% Accuracy. Under benchmark multi-run aggregation (Studies 1–3), F1 reached 70.5%, representing the system's detection ceiling. A portion of the Study 5 recall gap is structural: issues requiring DOM or runtime evidence (e.g. `aria-expanded` state, live-region timing) are definitionally resistant to static analysis. Clean-file HTML recorded zero findings across all participants; however, clean JS, CSS, and TSX files returned 24 findings across seven participants, all individually audited and confirmed as false positives on correctly implemented code (see Section 4.3.2.3). AI-assisted source-code analysis is effective within static analysis constraints; absolute recall is limited by runtime-dependent issue types, and clean-file specificity on JS, CSS, and TSX remains a limitation.

### *4.6.3 RQ3: Actionability and Quality of Developer-Facing Accessibility Feedback*

**Research Question 3 asks:** How effectively can a developer-facing AI assistant provide actionable accessibility feedback within a source-code authoring environment?

All ten participants (100%) rated explanation quality as accurate or very accurate; 90% rated detection accuracy similarly; and 90% agreed the tool is useful in real workflows. One participant (DEV003) strongly disagreed, citing a preference for error-block highlighting and line-position drift after editing, a limitation of the VS Code diagnostic model rather than the analysis quality. The two AT expert participants completed all seven tasks without assistance on the improved website, with Frustration scores dropping from 80–85 to 10–20, suggesting that the AI-guided source-code fixes improved usability for the two AT participants in this exploratory validation. The principal limitations are analysis latency and the recall constraints identified under RQ2.

## 4.7 Discussion in Relation to Existing Literature

The Study 5 and Study 6 results are broadly consistent with the tool benchmarking literature. Vigo et al. [58] found leading tools achieve 20–36% recall against WCAG 2.x defect sets; the AI assistant's combined-session recall of 32.5% (Study 5) and GT Recall of 32.5% (Study 6) fall within that range. Fischer et al. [28] confirmed that CSS issues are among the least reliably detected by any automated approach, replicated here with near-zero CSS recall across all tools. Pool et al. [26] and Frazão and Duarte [59] documented zero finding-level overlap between tools, reproduced in Study 6. Andruccioli et al. [35] and Huang et al. [38] found that LLM tools surface ARIA state-management issues undetectable by rule-based tools; this is confirmed by the Study 6 JavaScript findings where only the AI returned GT matches. López-Gil et al. [34] identified false-positive fatigue as a barrier to adoption; the 83.5% aggregate Accuracy in Study 6 directly addresses this concern; however, the 24 clean-file false positives on JS, CSS, and TSX in Study 5 (all confirmed via source audit) indicate that false-positive fatigue remains a risk on compliant JS, CSS, and TSX code, and further calibration of the consensus threshold is warranted. Hart and Staveland [12] established that NASA-TLX scores are experiential; the subscale discrepancies in Study 5 are consistent with this fundamental constraint. Petrie and Kheir [11] found AT barriers correspond to guideline violations, confirmed by the convergence between AI detections and expert-review observations in Study 5.

## 4.8 Threats to Validity and Result Limitations

Several threats should be noted. Internally, test files were synthetic and intentionally seeded, keyword-based scoring may credit terminologically matching but incomplete findings, Study 5 participants were self-selected, the fixed presentation order (defective website first, improved website second) in Study 5 may have introduced a carry-over or order effect on the NASA-TLX ratings, and Study 6 test files were designed to favour AI coverage on JavaScript and TSX. Externally, results are specific to the Kimi–Qwen configuration, single-file static analysis, WCAG 2.2 Level A/AA, and a single study website. Regarding construct validity, the NASA-TLX proxy captures directional burden but is not equivalent to experienced workload; Extra TP findings outside GT Recall may underestimate practical utility. Statistically, the NASA-TLX comparison rests on $N = 2$ AT participants and the developer survey on $N = 10$, neither sufficient for hypothesis testing; benchmark F1 reflects multi-run aggregation, overestimating single-run performance. Documented test files, inference presets, and scoring procedures are provided in the appendices to support replication [56].

## 4.9 Chapter Summary

Study 5 demonstrated good-precision (79.6%), moderate-recall (32.5%) combined-session detection with 90% participant workflow agreement. Precision was lower on JavaScript (60.8%) and TSX (63.5%) due to false positives on ARIA and event-handler patterns not resolvable through static analysis; HTML and CSS recorded 100% precision. The AI-derived NASA-TLX estimate correctly predicted workload direction at the overall level and across five of six subscales, supported by AT-expert workload reductions. Study 6 showed the AI assistant as the tool with the broadest cross-artefact coverage in this test setup (other tools scored as 0%/N/A where not applicable to a given file type), while deterministic tools remained valuable within their intended operating scopes, with 83.5% aggregate Accuracy and zero finding-level overlap with deterministic tools. The three research questions are answered: RQ1 with qualified support for the workload estimate as a directional proxy; RQ2 with evidence of effective detection within static analysis constraints; RQ3 with consistent evidence of high-quality, actionable feedback. Chapter 5 presents conclusions and recommendations.

# Chapter 5: Conclusions and Recommendations

## 5.1 Overview

This chapter draws together the findings of the six evaluation studies reported in Chapters 3 and 4 and provides final answers within the evaluated scope to the three research questions and the overarching research hypothesis. Section 5.2 summarises the research. Section 5.3 provides final answers to each research question. Section 5.4 states the overall conclusion on the hypothesis. Section 5.5 identifies the technical, methodological, and practical contributions. Section 5.6 acknowledges the study's limitations. Sections 5.7 and 5.8 offer recommendations and directions for future work. The chapter closes with a final statement in Section 5.9.

## 5.2 Summary of the Research

This research designed, implemented, and evaluated an AI Accessibility Assistant: a VS Code extension that analyses web source-code files for WCAG 2.2 Level A and AA accessibility issues using a RAG-based LLM pipeline and produces a NASA-TLX-informed workload estimate from the detected barriers. The system addressed a gap in the literature: existing automated accessibility tools are restricted to rule-encoded pattern violations on rendered HTML, leaving JavaScript, TypeScript/React, and CSS behavioural accessibility largely undetected, with no mechanism for estimating the user burden associated with identified barriers [29, 34, 58].

The evaluation programme comprised six studies. Studies 1 to 3 screened and shortlisted candidate models, optimised configuration parameters, and selected a final dual-model consensus ensemble (kimi-k2.5 and qwen3.5) across four experimental conditions (`rag-think`, `rag-nothink`, `norag-think`, `norag-nothink`), arriving at `rag-think` as the best-performing configuration with F1 = 70.5% under three-run union aggregation. Study 4 benchmarked the NASA-TLX workload estimation pipeline, calibrating severity weighting, WCAG-to-TLX mapping, subscale caps, and language difficulty modifiers. Study 5 conducted real-world validation with ten developer participants and two AT expert users, recording 32.5% combined-session recall, 79.6% precision, F1 = 0.462, and correct directional workload assignment at the overall level (AI: 60.8 defective vs 13.3 improved; participants: 51.65 vs 12.1) with subscale-level agreement on five of six dimensions. Precision was 100% on HTML and CSS; lower precision on JavaScript (60.8%) and TSX (63.5%) reflects false positives on ARIA and event-handler patterns not resolvable through static analysis alone. Study 6 compared the AI assistant against six established tools across four source-code test files, recording an aggregate AI Accuracy of 83.5%, GT Recall of 32.5%, and F1 of 0.468, with zero finding-level overlap between any pair of tools confirming high complementarity. CSS accessibility checking remained near-zero across all tools. JavaScript recall was exclusive to the AI assistant; TSX recall was near-exclusive (Axe Linter and ESLint jsx-a11y each returned one GT match, 3.3%, versus the AI's 56.7%).

## 5.3 Final Answers to the Research Questions

### *5.3.1 RQ1: Alignment Between the AI-Derived and Participant-Reported NASA-TLX Workload Estimate*

**RQ1 asks:** To what extent does the accessibility-issue-derived NASA-TLX workload estimate align with, approximate, or correlate with participant-reported NASA-TLX workload scores?

**The answer is: yes, with qualification.** The AI-derived estimate aligned directionally with participant-reported scores at the overall workload level and across five of six NASA-TLX subscales in both the defective and improved website conditions. The AI correctly assigned higher overall workload to the defective site (60.8/100) than to the improved site (13.3/100), consistent with the participant-reported direction (51.65 vs 12.1). The closest subscale predictions were Frustration (AI = 82 vs reported = 82.5) and Mental Demand (AI = 80 vs reported = 77.5), both grounded in source-code patterns directly detectable by static analysis. Physical Demand was overestimated by 25 points because the model had no mechanism for inferring the screen-reader interaction modality from source code alone. Performance on the improved condition showed close alignment (AI = 10 vs reported = 2.5): both the model and participants returned near-zero scores, reflecting near-perfect task success on the improved site. These subscale discrepancies are consistent with Hart and Staveland's finding that NASA-TLX scores are experiential and situation-specific, and therefore not fully predictable from structural analysis alone [12]. The AI-derived estimate is a credible directional proxy for accessibility workload burden,

suitable for signalling which codebase version is likely to impose greater cognitive demands on AT users, but not a substitute for participant-reported measurement.

### *5.3.2 RQ2: Effectiveness of AI-Assisted Source-Code Analysis for WCAG 2.2 Detection and Remediation*

**RQ2 asks:** How effectively can AI-assisted source-code analysis support the identification and remediation of WCAG 2.2 accessibility issues in HTML, CSS, JavaScript, and TypeScript/React files?

**The answer is: effectively within the constraints of static analysis.** Under real-world single-run conditions in Study 5, the system detected 195 of 600 ground-truth issues across both runs (32.5% combined-session recall) with 79.6% overall precision (100% on HTML and CSS; 60.8% on JavaScript and 63.5% on TSX). HTML achieved the highest recall (56.7%), followed by JavaScript (32.0%), TSX (22.0%), and CSS (19.3%). Under controlled benchmark three-run union aggregation, the system reached F1 = 70.5%, representing its detection ceiling under optimal conditions. Study 6 confirmed an aggregate GT Recall of 32.5% across 120 planted defects with 83.5% Accuracy, placing the system at or above Vigo et al.'s published baseline range of 20–36% for leading automated accessibility tools [58]. The recall gap between the benchmark ceiling (70.5%) and combined-session performance (32.5%) is partly structural: issues requiring DOM rendering, runtime state, or live-region timing evidence are definitionally resistant to static analysis. The AI assistant is the only tool evaluated that returned any GT-matched findings on JavaScript source files; on TSX it was the dominant tool (56.7% GT Recall), with the nearest alternatives (Axe Linter and ESLint jsx-a11y) each

returning a single GT match at 3.3%, confirming that ARIA state-management, dynamic announcements, and component-level interaction patterns that rule-based tools cannot detect are largely exclusive to the AI assistant [29, 38].

### *5.3.3 RQ3: Actionability and Quality of Developer-Facing Accessibility Feedback*

**RQ3 asks:** How effectively can a developer-facing AI assistant provide actionable accessibility feedback within a source-code authoring environment?

**The answer is: highly effectively, with one usability caveat.** All ten developer participants (100%) rated explanation quality as accurate or very accurate. Nine participants (90%) rated detection accuracy as accurate or very accurate, and nine (90%) agreed or strongly agreed that the tool is useful in real development workflows. The two AT expert users completed all seven tasks without assistance on the AI-improved website, contrasted with multiple tasks requiring assistance on the defective version, and reported Frustration dropping from 80–85 to 10–20. This suggests the AI-guided source-code fixes produced measurable usability improvements for the two screen-reader users, consistent with Petrie and Kheir's finding that guideline violations correspond to barriers experienced by users with disabilities [11]. The one usability concern raised (that detected error positions shift when code is edited after analysis) is a limitation of the VS Code diagnostic model rather than the analysis quality, and does not diminish the accuracy or actionability of the generated guidance.

## 5.4 Overall Conclusion on the Hypothesis

The research hypothesis states: *AI-based techniques that combine source-code accessibility analysis with NASA-TLX-informed workload estimation can provide developer-facing feedback that better reflects accessibility barriers and likely user burden than approaches focused only on rule-based accessibility checking.*

**The hypothesis is supported within the constraints of the implemented benchmark, expert review, and exploratory AT-user validation.** First, the AI assistant detected accessibility barriers in JavaScript and TSX source code that zero rule-based tools detected, including ARIA state-management failures, dynamic live-region deficiencies, and component-level focus management gaps, precisely the issues AT-expert users encountered as real barriers. Second, the NASA-TLX-informed workload estimate correctly distinguished the defective from the improved website in overall workload direction and on five of six subscales, providing a structured, quantitative workload signal no rule-based tool produced. Third, developer participants confirmed the feedback was understandable and actionable at a 90–100% agreement rate, and AT participant results supported improved usability following AI-guided remediation. Rule-based tools (Lighthouse, axe-core, Axe Linter, Webhint, ESLint jsx-a11y, stylelint-a11y), while precise within their rule categories, achieved 0% recall on JavaScript and 3% on TSX, and produced no workload estimation capability. The AI assistant's combination of source-level semantic analysis, retrieved accessibility knowledge, workload-aware interpretation, and actionable developer-facing output represents a qualitatively different and complementary form of developer-facing accessibility feedback within the evaluated scope.

## 5.5 Contribution of the Study

### *5.5.1 Technical Contribution*

The primary technical contribution is the AI Accessibility Assistant itself: a VS Code extension that integrates a retrieval-augmented dual-LLM pipeline for WCAG 2.2 accessibility analysis with a NASA-TLX-informed workload estimation module. The extension operates on HTML, CSS, JavaScript, and TypeScript/React source files directly in the authoring environment, presenting per-line diagnostics with structured remediation guidance. Technical contributions include: (1) a 200-line chunk-based analysis pipeline that gives each code segment the full attention of the language model, preventing the attention-degradation failure observed in full-file prompting; (2) a dual-model consensus merge mechanism using Jaccard similarity (title-and-explanation score $\geq$ 0.30) that substantially reduces false positives compared with single-model output, while admitting quality-gated single-model findings that cite a WCAG SC, carry high or medium severity, and include a concrete line number; (3) a RAG subsystem backed by a ChromaDB vector store with accessibility knowledge-base documents spanning ARIA, WCAG criterion groups, and NASA-TLX workload mappings; and (4) a NASA-TLX estimation module that maps detected accessibility issues to workload dimensions using severity-weighted, subscale-capped scores calibrated across four file types. The full extension source, evaluation scripts, benchmark test files, and scoring procedures are provided in the accompanying code repository, consistent with reproducibility standards [56].

### *5.5.2 Methodological Contribution*

The study introduces an original evaluation methodology for AI-assisted accessibility tools that integrates three distinct lines of evidence: controlled benchmark scoring against planted ground-truth defects, real-world developer validation with structured survey and re-run confirmation, and participant-reported NASA-TLX workload comparison against AI-derived estimates. This triangulated design distinguishes between the system's detection ceiling under ideal multi-run conditions and its operational single-run performance, and connects source-code analysis outcomes directly to AT user experience through the workload proxy. The WCAG-to-NASA-TLX mapping framework (linking barrier severity, issue type, and file-type difficulty modifiers to the six subscale dimensions of the Task Load Index [12, 13]) is a novel methodological contribution that may be applicable to other accessibility tool evaluations seeking to connect technical findings to user burden. The four-condition experimental structure (`rag-think`, `rag-nothink`, `norag-think`, `norag-nothink`) provides a reusable ablation framework for evaluating retrieval augmentation and chain-of-thought reasoning independently in LLM-based code analysis tools.

### *5.5.3 Practical Contribution*

The practical contribution of this study is evidence-based guidance for practitioners and organisations subject to accessibility compliance requirements, particularly under the EAA [2]. The study demonstrates that AI-assisted source-code analysis can shift accessibility evaluation left into the authoring environment, allowing

developers to receive structured, line-level guidance before code is deployed, without requiring a separate audit tool or post-deployment browser testing step. The Study 6 results establish a practical tool-selection framework for organisations assembling accessibility evaluation pipelines: the AI assistant provides the only coverage for JavaScript and TSX behavioural patterns, deterministic rendered-DOM tools (Lighthouse, axe-core) provide high-precision HTML confirmation, and specialist linters (stylelint-a11y for CSS, ESLint jsx-a11y for JSX) contribute targeted zero-FP rule coverage within their defined scopes. The zero finding-level overlap across all tools confirms that no single checker is sufficient and that a layered strategy is required [26, 58]. Developers and accessibility evaluators can use these findings to construct defensible, WCAG-EM-aligned evaluation pipelines proportionate to their codebase's artefact types.

## 5.6 Limitations of the Study

Several limitations must be acknowledged. First, the Study 5 and Study 6 test files were synthetic and intentionally seeded with known defects; real-world codebases have different issue distributions and densities. Second, the study involved ten developer participants and two AT users, sample sizes that are sufficient for exploratory validation but not for statistical generalisation. Third, the AI assistant was evaluated in a single model configuration (kimi-k2.5 and qwen3.5 at the selected consensus threshold); different model families or newer model versions may produce different performance profiles. Fourth, the dual-model consensus requirement reduces recall relative to a pure union: agreed findings (Jaccard $\geq$ 0.30)

are always included, but single-model findings are only admitted if they meet all three quality gates (WCAG SC cited, high or medium severity, concrete line number), meaning lower-confidence single-model detections are discarded even when correct. Fifth, the NASA-TLX comparison was based on $N = 2$ AT participants, which provides directional signal but is insufficient for statistical workload analysis. Sixth, the benchmark F1 of 70.5% was achieved under three-run union aggregation, which overestimates single-run performance; the operational recall in live use is approximately 32.5% across a combined two-run session. Finally, the study's scope was limited to WCAG 2.2 Level A and AA, a single study website, and the four source-code file types examined; coverage and performance may differ on other websites, frameworks, or accessibility standards. A further practical limitation is analysis latency: each 200-line chunk requires approximately five minutes to process through the dual-model pipeline, meaning a 600-line file takes around fifteen minutes end-to-end. This makes the tool unsuitable for on-save or keystroke-triggered analysis and restricts its use to deliberate milestone-based runs. Related to this, the RAG knowledge base and inference parameter configuration were calibrated during the benchmarking studies but were not exhaustively optimised; it is likely that further parameter tuning and richer knowledge-base coverage would improve both recall and false-positive rates beyond the values reported here. The false-positive rate, while low in aggregate (16.5% across Study 6), was non-negligible in individual file runs (particularly on CSS and TSX), and some real issues were suppressed by the dual-model consensus requirement, meaning the system discards certain genuine findings in order to control noise. A related

specificity concern was identified in Study 5: 24 findings on clean counterpart JS, CSS, and TSX files were individually audited and confirmed as false positives on correctly implemented code, indicating that the model's tendency to flag ARIA and event-handler patterns spuriously is not confined to defective files.

## 5.7 Recommendations

### *5.7.1 Recommendations for Developers*

Developers should use the AI Accessibility Assistant as a primary source-level analysis tool during implementation, particularly for JavaScript and TypeScript/React components where ARIA state management, live-region configuration, and focus management issues require semantic understanding that rule-based linters cannot provide. Analysis should be run on each file at a meaningful implementation milestone, not solely as a pre-deployment check, to embed accessibility verification into the normal development cycle. AI-detected issues should be treated as credible until reviewed but not as exhaustive; developers should supplement AI findings with rendered-DOM tool output (Lighthouse or axe-core) for HTML confirmation and with ESLint jsx-a11y for basic JSX compliance. Developers should apply suggested fixes and re-run the analysis to confirm resolution within the same session. Where latency is a concern, the balanced inference preset (`no-think` configuration) may be applied for rapid initial scanning, with the reasoning preset (`think` configuration) reserved for thorough review of complex files.

### *5.7.2 Recommendations for Accessibility Evaluators*

Accessibility evaluators assembling evaluation pipelines for WCAG 2.2 compliance should apply a layered strategy: AI source-level analysis for all four artefact types, rendered-DOM auditing (Lighthouse, axe-core) for HTML, and specialist linters (ESLint jsx-a11y, stylelint-a11y) for JSX and CSS respectively. No single tool should be relied upon as comprehensive; the zero overlap in Study 6 confirms that each tool surfaces a distinct, non-redundant subset of accessibility issues [58, 59]. The AI-derived NASA-TLX workload estimate should be used as a triage signal for prioritising remediation: conditions where the AI assigns high Frustration or Mental Demand scores warrant earlier user testing with AT users. All AI findings should be adjudicated by a practitioner before being treated as confirmed violations, consistent with WCAG-EM guidance on multi-method evaluation [24]. CSS accessibility evaluation should not rely on any automated tool alone; the near-zero recall across all tools on CSS confirms that composite style-pattern violations require human review.

### *5.7.3 Recommendations for Future Researchers*

Future researchers extending this line of work should address the NASA-TLX subscale underestimation at high issue counts. The current linear mapping model is directionally correct but underpowered at the magnitude level; non-linear scaling or learned regression models trained on larger AT participant datasets would improve absolute prediction accuracy. Researchers should also investigate the Jaccard consensus threshold empirically against a labelled held-out dataset, rather than setting it by inspection, to determine the precision–recall trade-off curve for different threshold values. Longitudinal evaluation studies examining how developers' accessibility knowledge and practice change over sustained use of the tool would strengthen the case for shift-left accessibility tooling. Studies covering Level AAA criteria, WCAG 3.0 emerging guidelines, and non-WCAG frameworks such as EN 301 549 [3] would extend the coverage scope. Finally, multi-model ensemble studies comparing a wider range of LLM families, including models specifically fine-tuned on accessibility corpora, would clarify whether the performance improvements observed from kimi-qwen consensus generalise to other model pairings.

## 5.8 Future Work

Several directions for extending this research are identified based on the technical limitations and open questions encountered during the study. The most immediate is multi-file analysis: the current extension analyses one file at a time, which prevents detection of cross-file accessibility relationships such as an ARIA label defined in one component and consumed in another. A session-level analysis mode that ingests a full component tree would address this gap. A second direction is runtime-aware analysis: integrating a browser execution step before the AI analysis would allow the model to inspect the rendered accessibility tree and identify issues that manifest only after JavaScript execution, substantially increasing recall for dynamic ARIA patterns currently outside the reach of static analysis. A third direction is adaptive consensus thresholding: replacing the fixed Jaccard threshold with a model-confidence-aware threshold that adjusts based on the model's self-reported certainty would reduce the blind-spot risk from uniform intersection rules. Fourth, the NASA-TLX estimation module would benefit from validation against a larger AT participant sample across diverse disability profiles, websites, and assistive technologies to calibrate the mapping coefficients more reliably. Fifth, extending the RAG knowledge base to cover WCAG 3.0 outcome criteria, mobile accessibility guidelines, cognitive accessibility patterns under Cognitive Accessibility (COGA) guidance, and EN 301 549 clauses beyond WCAG would broaden the tool's regulatory coverage. A sixth priority is improving recall and reducing false positives through RAG knowledge-base enrichment and systematic parameter tuning. The current knowledge base covers core WCAG criterion groups, ARIA patterns,

and NASA-TLX mappings but does not include detailed remediation examples, component-level interaction patterns, or WCAG 3.0 outcome criteria. Expanding the knowledge base with richer, example-driven accessibility documents (particularly for CSS composite patterns and JavaScript ARIA state management) would give the retrieval step more targeted material to ground model reasoning, likely reducing hallucinations and increasing true-positive rate. In parallel, a systematic grid search over inference parameters (temperature, top-p, Jaccard consensus threshold, chunk overlap) against a held-out labelled dataset would allow the precision–recall trade-off to be tuned empirically rather than by inspection, potentially recovering genuine findings currently discarded by the quality-gating requirements. A seventh direction is addressing the latency constraint: the approximately five-minute processing time per 200-line chunk is the most significant barrier to practical adoption in an interactive IDE workflow. Approaches worth investigating include running chunk analysis in parallel background threads so the developer can continue editing while results accumulate incrementally, caching results per chunk and invalidating only changed regions on re-run, and distilling the dual-model pipeline into a single smaller model fine-tuned on labelled outputs of the full ensemble. Finally, integrating the tool into CI/CD pipelines, triggering analysis automatically on pull request and reporting accessibility regressions as blocking checks, would operationalise shift-left accessibility evaluation at the organisational level, consistent with the EAA timeline requiring compliance by 2025 for products and services across both public and private sectors [2].

## 5.9 Final Closing Statement

Web accessibility fails at scale not primarily because standards are unclear or tools are absent, but because the gap between authoring and auditing remains too wide for individual developers to bridge reliably in their normal workflow [14, 34]. This research demonstrates that the gap can be narrowed by embedding retrieval-augmented AI analysis directly into the source-code authoring environment. The AI Accessibility Assistant makes WCAG 2.2 accessibility knowledge available at the line level, in the editor, while the developer still has the context and capability to act on it. The system is not a replacement for user testing, expert review, or deterministic rule engines; it is a complementary layer that addresses the coverage gap those approaches leave in JavaScript, TypeScript/React, and CSS source code. The convergence of AI source-code analysis, workload-informed interpretation, and real-world AT user validation in a single evaluation framework is the contribution this study offers to the research community and to the practitioners working to make the web genuinely accessible to all users.

# List of References

[1] W3C, “Web content accessibility guidelines (wcag) 2.2,” 2023, w3C Recommendation. [Online]. Available: https://www.w3.org/TR/WCAG22/. Accessed: Apr. 27, 2026.

[2] European Parliament and Council, “Directive (eu) 2019/882 of the european parliament and of the council of 17 april 2019 on the accessibility requirements for products and services,” 2019, official Journal of the European Union, L 151, pp. 70–115. [Online]. Available: https://eur-lex.europa.eu/eli/dir/2019/882/oj. Accessed: Apr. 27, 2026.

[3] European Telecommunications Standards Institute, “En 301 549 v3.2.1: Accessibility requirements for ict products and services,” 2021, eTSI, Sophia Antipolis, France. [Online]. Available: https://www.etsi.org/deliver/etsi_en/301500_301599/301549/03.02.01_60/en_301549v030201p.pdf. Accessed: Apr. 27, 2026.

[4] Google, “Introduction to lighthouse,” 2025, chrome for Developers. [Online]. Available: https://developer.chrome.com/docs/lighthouse/overview. Accessed: Apr. 27, 2026.

[5] ——, “Lighthouse accessibility score,” 2026, chrome for Developers. [Online]. Available: https://developer.chrome.com/docs/lighthouse/accessibility/scoring. Accessed: May 16, 2026.

[6] Deque Systems, “axe-core: The reliable accessibility testing engine,” 2024, gitHub Repository. [Online]. Available: https://github.com/dequelabs/axe-core. Accessed: Apr. 27, 2026.

[7] ——, “axe accessibility linter,” 2026, visual Studio Marketplace. [Online]. Available: https://marketplace.visualstudio.com/items?itemName=deque-systems.vscode-axe-linter. Accessed: May 16, 2026.

[8] jsx-eslint, “eslint-plugin-jsx-a11y,” 2026, gitHub Repository. [Online]. Available: https://github.com/jsx-eslint/eslint-plugin-jsx-a11y. Accessed: May 16, 2026.

[9] webhint, “webhint’s accessibility configuration,” 2026, webhint Documentation. [Online]. Available: https://webhint.io/docs/user-guide/configurations/configuration-accessibility/. Accessed: May 16, 2026.

[10] YozhikM, “stylelint-a11y,” 2026, gitHub Repository. [Online]. Available: https://github.com/YozhikM/stylelint-a11y. Accessed: May 16, 2026.

[11] H. Petrie and O. Kheir, “The relationship between accessibility and usability of websites,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI ’07)*, San Jose, CA, USA, 2007, pp. 397–406.

[12] S. G. Hart and L. E. Staveland, “Development of nasa-tlx: Results of empirical and theoretical research,” in *Human Mental Workload*, P. A. Hancock and N. Meshkati, Eds. Amsterdam, Netherlands: North-Holland, 1988, pp. 139–183.

[13] S. G. Hart, "Nasa task load index (nasa-tlx): 20 years later," in *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 50, no. 9, 2006, pp. 904–908.

[14] WebAIM, "The webaim million: The 2024 report on the accessibility of the top 1,000,000 home pages," 2024, [Online]. Available: https://webaim.org/projects/million/2024. Accessed: Apr. 27, 2026.

[15] B. Martins and C. Duarte, "A large-scale web accessibility analysis considering technology adoption," *Universal Access in the Information Society*, vol. 23, pp. 1857–1872, 2024.

[16] World Health Organisation, "Disability and health," 2023, wHO Fact Sheet. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/disability-and-health. Accessed: Apr. 27, 2026.

[17] K. Fernandes, S. Paramananthan, L. Cockburn, and J. Nganji, "Readily available but how accessible? an analysis of the web accessibility of healthcare-related resources," *Journal of Accessibility and Design for All*, vol. 13, no. 2, pp. 188–215, 2023.

[18] W3C, "Understanding success criterion 1.3.1: Info and relationships," 2026, w3C WAI Supporting Document. [Online]. Available: https://www.w3.org/WAI/WCAG22/Understanding/info-and-relationships. Accessed: Apr. 27, 2026.

[19] ——, "H91: Using html form controls and links," 2026, techniques for WCAG 2.2. [Online]. Available:

https://www.w3.org/WAI/WCAG22/Techniques/html/H91. Accessed: Apr. 27, 2026.

[20] ——, "Accessible rich internet applications (wai-aria) 1.1," 2017, w3C Recommendation. [Online]. Available: https://www.w3.org/TR/wai-aria-1.1/. Accessed: Apr. 27, 2026.

[21] ——, "Aria authoring practices guide," 2026, w3C Web Accessibility Initiative. [Online]. Available: https://www.w3.org/WAI/ARIA/apg/. Accessed: Apr. 27, 2026.

[22] Microsoft, "Visual studio code extension api," 2026, microsoft Documentation. [Online]. Available: https://code.visualstudio.com/api. Accessed: Apr. 27, 2026.

[23] W3C, "Accessibility conformance testing (act) rules format 1.1," 2026, w3C Recommendation. [Online]. Available: https://www.w3.org/TR/act-rules-format/. Accessed: Apr. 27, 2026.

[24] ——, "Website accessibility conformance evaluation methodology (wcag-em) 1.0," 2014, w3C Working Group Note. [Online]. Available: https://www.w3.org/TR/WCAG-EM/. Accessed: May 16, 2026.

[25] J. Dias, D. Carvalho, H. Paredes, P. Martins, T. Rocha, and J. Barroso, "Automated evaluation tools for web and mobile accessibility: A systematic literature review," in *Innovations in Bio-Inspired Computing and Applications*, 2022, pp. 447–456.

[26] J. R. Pool, "Accessibility metatesting: Comparing nine testing tools," in *Proceedings of the 20th International Web for All Conference (W4A '23)*, Austin, TX, USA, 2023, pp. 1–4.

[27] A. Lempola, T. Poranen, and Z. Zhang, "Comparing automatic accessibility testing tools," in *Proceedings of the Annual Doctoral Symposium of Computer Science 2024*, Vaasa, Finland, 2024, pp. 43–53, cEUR Workshop Proceedings, vol. 3776.

[28] T. Fischer, B. Lundell, and J. Gamalielsson, "Coverage of web accessibility guidelines provided by automated checking tools," *Universal Access in the Information Society*, vol. 24, pp. 3615–3637, 2025.

[29] J. Abascal, M. Arrue, and X. Valencia, "Tools for web accessibility evaluation," in *Web Accessibility: A Foundation for Research*, ser. Human–Computer Interaction Series, Y. Yesilada and S. Harper, Eds. London, UK: Springer, 2019.

[30] A. Vaswani *et al.*, "Attention is all you need," in *Proceedings of NeurIPS 2017*, Long Beach, CA, USA, 2017, pp. 5998–6008.

[31] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT 2019*, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[32] T. Brown *et al.*, "Language models are few-shot learners," in *Proceedings of NeurIPS 2020*, Virtual, 2020, pp. 1877–1901.

[33] M. Chen *et al.*, "Evaluating large language models trained on code," 2021, arXiv preprint arXiv:2107.03374.

[34] J. M. López-Gil and J. Pereira, "Turning manual web accessibility success criteria into automatic: An llm-based approach," *Universal Access in the Information Society*, 2025.

[35] M. Andruccioli, B. Bassi, G. Delnevo, and P. Salomoni, "Leveraging large language models for sustainable and inclusive web accessibility," *Big Data and Cognitive Computing*, vol. 9, no. 10, p. 247, 2025.

[36] N. Fathallah, D. Hernández, and S. Staab, "Accessguru: Leveraging llms to detect and correct web accessibility violations in html code," in *Proceedings of the 27th ACM SIGACCESS Conference on Computers and Accessibility (ASSETS '25)*, 2025.

[37] Z. He, S. F. Huq, and S. Malek, "Enhancing web accessibility: Automated detection of issues with generative ai," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 2264–2287, 2025.

[38] C. Huang, A. Ma, S. Vyasamudri, E. Puype, S. Kamal, J. B. Garcia, S. Cheema, and M. Lutz, "Access: Prompt engineering for automated web accessibility violation corrections," 2024, arXiv preprint arXiv:2401.16450.

[39] F. Paternò, M. Vinci, M. Manca, and N. Iannuzzi, "How an llm can improve automatic web accessibility validation?" in *Proceedings of the 16th Biannual Conference of the Italian SIGCHI Chapter (CHItaly '25)*, 2025, pp. 1–8.

[40] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Proceedings of NeurIPS 2020*, Virtual, 2020, pp. 9459–9474.

[41] Chroma, "Chroma: Search infrastructure for ai," 2026, gitHub Repository. [Online]. Available: https://github.com/chroma-core/chroma. Accessed: Apr. 27, 2026.

[42] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: Bm25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[43] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of EMNLP-IJCNLP 2019*, Hong Kong, China, 2019, pp. 3982–3992.

[44] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proceedings of NeurIPS 2020*, Virtual, 2020.

[45] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Proceedings of NeurIPS 2022*, New Orleans, LA, USA, 2022.

[46] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Proceedings of NeurIPS 2022*, New Orleans, LA, USA, 2022.

[47] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *Proceedings of ICLR 2023*, Kigali, Rwanda, 2023.

[48] M. Renze and E. Guven, "The effect of sampling temperature on problem solving in large language models," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, Miami, FL, USA, 2024, pp. 7346–7356.

[49] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *Proceedings of ICLR 2020*, Addis Ababa, Ethiopia, 2020.

[50] S. Welleck, I. Kulikov, S. Roller, E. Dinan, K. Cho, and J. Weston, "Neural text generation with unlikelihood training," in *Proceedings of ICLR 2020*, Addis Ababa, Ethiopia, 2020.

[51] D. Chicco and G. Jurman, "The advantages of the matthews correlation coefficient (mcc) over f1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, no. 6, 2020.

[52] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988.

[53] F. Paas, A. Renkl, and J. Sweller, "Cognitive load theory and instructional design: Recent developments," *Educational Psychologist*, vol. 38, no. 1, pp. 1–4, 2003.

[54] A. Darejeh, N. Marcus, G. Mohammadi, and J. Sweller, "Cognitive load measurement methods for usability testing: A critical analysis and framework for interface evaluation," *Human Factors*, 2026.

[55] J. W. Creswell and J. D. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 5th ed. Thousand Oaks, CA, USA: SAGE Publications, 2017.

[56] J. Pineau *et al.*, "Improving reproducibility in machine learning research," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.

[57] European Parliament and Council, "Regulation (eu) 2016/679 of the european parliament and of the council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data," 2016, official Journal of the European Union, L 119, pp. 1–88. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2016/679/oj. Accessed: Apr. 27, 2026.

[58] M. Vigo, J. Brown, and V. Conway, "Benchmarking web accessibility evaluation tools: Measuring the harm of sole reliance on automated tests," in *Proceedings of the 10th International Cross-Disciplinary Conference on Web Accessibility*, ser. W4A '13. New York, NY, USA: Association for Computing Machinery, 2013.

[59] T. Frazão and C. Duarte, "Comparing accessibility evaluation plug-ins," in *Proceedings of the 17th International Web for All Conference*, ser. W4A '20. New York, NY, USA: Association for Computing Machinery, 2020, pp. 1–11.

[60] G. Brajnik, “Comparing accessibility evaluation tools: A method for tool effectiveness,” *Universal Access in the Information Society*, vol. 3, no. 3–4, pp. 252–263, 2004.

[61] HTTP Archive, “Web almanac 2024: Page weight,” 2024, [Online]. Available: https://almanac.httparchive.org/en/2024/page-weight. Accessed: Jun. 2026.

[62] ——, “Web almanac 2022: Page weight,” 2022, [Online]. Available: https://almanac.httparchive.org/en/2022/page-weight. Accessed: Jun. 2026.

# Chapter A: Extended System Architecture Notes

## A.1 Purpose of the Architecture

The AI Accessibility Assistant was designed as a modular research artefact rather than a single fixed tool. This made it possible to vary the selected model, retrieval condition, and reasoning mode while keeping the rest of the analysis pipeline stable. This supported reproducibility because the same input files, prompt structure, response parser, and scoring method could be reused across all experimental conditions.

## A.2 Component Responsibilities

The system consisted of three primary components: the VS Code extension, the RAG service, and the LLM inference layer.

**VS Code Extension.** The VS Code extension acted as the developer-facing interface and experimental controller. It read the active source file, detected the file type, executed deterministic baseline checks, constructed RAG queries, assembled prompts, sent inference requests, parsed model responses, and displayed results through the Activity Bar Webview and VS Code diagnostics. TypeScript was used because the VS Code Extension API supports TypeScript-based extension development [22].

**RAG Service.** The RAG service stored WCAG 2.2 Level A and AA guidance, together with NASA-TLX material, as indexed Markdown chunks. Retrieval

combined BM25Okapi keyword matching [42] with `all-MiniLM-L6-v2` dense semantic embeddings [44]. This allowed the system to retrieve exact technical identifiers, such as WCAG success criterion numbers, as well as semantically related guidance written in different terminology. ChromaDB was used as the persistent vector store for retrieved knowledge chunks [41].

**LLM Inference Layer.** The LLM inference layer was accessed through the Ollama Hypertext Transfer Protocol (HTTP) interface. The selected model received the structured prompt and returned accessibility findings as structured issue blocks. Using the same inference interface across model families supported controlled comparison between models and reduced variation caused by changing the surrounding pipeline.

### A.3 Rationale for the Three-Layer Design

The three-layer design was selected because the study required both technical flexibility and controlled experimentation. Separating the extension, retrieval service, and inference layer made it possible to test the contribution of each major variable without changing the full system. For example, RAG could be enabled or disabled by including or excluding retrieved knowledge from the prompt, while reasoning mode could be changed through the selected inference preset.

The RAG service was kept separate from the VS Code extension so that the knowledge base could be indexed and queried independently. The LLM layer was accessed through a standard HTTP interface so that multiple models could be evaluated using the same prompt and response-handling pipeline.

## A.4 Failure-Tolerant Behaviour

The system was designed to return partial results where possible. Baseline checks were always executed before RAG or LLM calls. This meant that common accessibility issues could still be reported even if the RAG service or inference layer was unavailable. Table A.4 summarises the failure-tolerant behaviour implemented in the system.

| Failure Point | System Behaviour |
|---|---|
| **RAG service unavailable** | Baseline checks could still run, but retrieved guidance was omitted |
| **LLM inference unavailable** | Baseline diagnostics could still be displayed |
| **Response parsing incomplete** | Raw or partially parsed output could still be shown in the Webview |
| **Line mapping unreliable** | Issue was shown in the Webview rather than being placed as a misleading editor diagnostic |

***Table A.4:*** *Failure-Tolerant Behaviour*

## A.5 Relationship to the Experimental Conditions

The architecture supported the four experimental conditions by keeping the source files, test-file design, and scoring framework constant while varying only the RAG setting and inference preset. The specific execution conditions and their implementation are described in Section 3.2.1 and Section 3.3.4.

# Chapter B: Extended Domain Scope and Reproducibility Notes

This appendix provides additional detail for the domain scope, test-file design, boundary conditions, and deployment decisions summarised in Section 3.2.

## B.1 Test-File Size Rationale

To ensure that the benchmark test files were representative of real-world web content rather than artificially simplified prototypes, file sizes were informed by the HTTP Archive Web Almanac series. HTML test files were designed around the 32–33 kB median transferred document size reported in the 2024 Web Almanac [61]. CSS and JavaScript test file sizes were informed by the 2022 Web Almanac, which reported median desktop page resource sizes of 72 kB for CSS and 509 kB for JavaScript [62]. These values were used as reference points to keep the benchmark corpus closer to realistic front-end files rather than minimal test examples.

## B.2 Extended Error-Level Calibration Rationale

The number of injected violations per tier was informed by published accessibility studies. Martins and Duarte [15] reported recurring accessibility errors across a large-scale sample of web pages, supporting the use of a medium-density violation tier. Fernandes et al. [17] reported high-density accessibility failures in healthcare-related web resources, which informed the high difficulty tier. The clean and low

tiers provided control and minimal-violation conditions.

### B.3 Extended Boundary Conditions

The study was limited to static source-code analysis. The system did not execute JavaScript, inspect a rendered DOM, or evaluate browser-computed styles. This means that runtime-only issues, such as dynamically inserted ARIA attributes or interaction states created after execution, could not always be detected.

The analysis was also single-file. Cross-file relationships, such as CSS declarations affecting HTML elements defined in another file, were outside full detection scope unless the relevant evidence appeared in the analysed file.

The benchmark used test-file-based ground truth. This increased repeatability and allowed precision and recall to be calculated consistently, but performance on production code must be interpreted cautiously. The adversarial test files were included to test edge cases beyond the standard difficulty tiers.

Keyword matching was used for evaluation because LLMs often describe the same issue using different wording. This made scoring more flexible, but it may credit outputs that contain the correct keyword while still providing an incomplete explanation. For this reason, expert review was included later in the study to assess practical remediation quality.

## B.4 Deployment Notes

Local model deployment was explored before the main benchmarking phase. Several local models were attempted, but service instability and hardware limitations made local execution unsuitable for the main experiment. Cloud-routed inference through the Ollama HTTP API was therefore retained because it provided a consistent interface across the evaluated model families.

Some model variants were excluded due to infrastructure or compatibility failures. For example, MiniMax variants repeatedly returned HTTP 500 errors through the cloud relay. These failures were treated as deployment limitations rather than evidence of model capability.

# Chapter C: Detailed System Implementation

This appendix expands the implementation details summarised in Section 3.3, including extension behaviour, baseline checks, RAG retrieval, prompt construction, response parsing, and diagnostic rendering.

## C.1 VS Code Extension Implementation

The VS Code extension was implemented in TypeScript using the VS Code Extension API [22]. It acted as the developer-facing layer of the system and controlled the process from active-file reading to diagnostic publication. The extension was registered as a persistent sidebar `WebviewView` provider, with `retainContextWhenHidden` enabled so that panel state remained available when the user changed editor focus.

The extension contributed two commands. Table C.1 summarises the contributed commands and additional panel-level functionality.

| **Functionality** | **Status in prototype** | **Purpose** |
|---|---|---|
| `analyseFile` | Contributed command | Triggered the full accessibility analysis pipeline |
| `tlxAnalysis` | Contributed command | Invoked the computational NASA-TLX workload prediction feature |
| Model configuration (`selectModelPanel`) | Panel-level source module (not a separately contributed command) | Supported development-time preset management; formal model benchmarking used terminal scripts rather than the UI |

***Table C.1:*** *Extension Commands and Panel Functionality*

The extension supported four source-file types: HTML, CSS, JavaScript, and

TypeScript/React TSX. File type detection was performed using the VS Code `document.languageId` property. This identifier determined which deterministic baseline rules, retrieval queries, and prompt instructions were applied.

## C.2 Baseline Accessibility Checks

Baseline checks were designed as synchronous regex-based rules intended to execute before any RAG or LLM call. This design allowed the tool to return immediate accessibility feedback even if the retrieval service or model inference layer was unavailable. The baseline rule module (`baseline.ts`) was implemented and is present in the repository source.

In the public prototype snapshot, the `analyseFileForPanel` pipeline function does not explicitly call the baseline module before model inference. The baseline-only fallback path in the error-handling block is reached only when the LLM call fails, not as a guaranteed first-stage execution. The baseline module therefore represents a designed and implemented rule layer that was not fully wired into the active analysis pipeline in the current prototype. This does not affect the benchmarking or evaluation results, which were produced through the LLM/RAG pipeline, but it means the baseline should be described as a supporting rule layer rather than a guaranteed first-stage execution path in the developer-facing workflow.

For HTML files, the baseline rules detected images without non-empty `alt` attributes and form-related elements without clear associations. These checks supported WCAG 1.1.1 and WCAG 1.3.1. For CSS files, the baseline rules detected

cases where focus indicators were removed using declarations such as `outline: none`, `outline: 0`, or `outline: transparent` without a visible replacement such as a non-zero `box-shadow` or `border`. These checks supported WCAG 2.4.7.

For JavaScript and TSX files, baseline checks were limited to source-visible accessibility patterns where deterministic detection was possible. These included evidence of missing ARIA state updates, weak event-handling patterns, inaccessible interactive elements, or missing relationships where the relevant code was present in the analysed file. More contextual JavaScript and TSX issues were passed to the LLM-mediated analysis stage because they often required interpretation of component logic, state changes, or DOM-related relationships.

Baseline findings were displayed as VS Code diagnostics and embedded into the LLM prompt. This allowed deterministic findings to act as both direct feedback and contextual evidence for later AI-based analysis.

### C.3 RAG Service and Knowledge Base Indexing

The RAG service was implemented as a Python FastAPI application exposing a `/retrieve` endpoint. The knowledge base consisted of curated Markdown documentation covering WCAG 2.2 Level A and AA success criteria, together with supporting NASA-TLX material.

Source documents were indexed using a section-aware chunking strategy. Each `##` heading block was treated as a candidate section and then subdivided into 128-token windows with a 12-token overlap. The `sentence-transformers/all-`

`MiniLM-L6-v2` tokeniser was used to count tokens precisely [44]. Chunks shorter than 80 characters after stripping were discarded.

Each chunk was assigned a stable identifier in the form:

```
{filepath}::sec{i}
```

The chunks were stored in a ChromaDB persistent collection named `accessibility_docs` [41]. Dense embeddings were generated using `all-MiniLM-L6-v2` [44]. In parallel, BM25Okapi indexes were built in memory at service start-up to support sparse keyword retrieval [42].

### C.4 Hybrid Retrieval and Caching

At retrieval time, the system combined BM25 keyword scores with ChromaDB cosine-distance similarity scores. This hybrid approach allowed the RAG service to retrieve exact technical identifiers, such as WCAG success criterion numbers, while also retrieving conceptually related accessibility guidance written in different terminology.

Chunks whose cosine distance exceeded the configured threshold were filtered out. The default retrieval returned the most relevant chunks for the generated query. For HTML files, targeted sweep queries were used to retrieve guidance across different accessibility categories, including ARIA, form labelling, image alternatives, heading structure, and broken references.

For CSS files, retrieval focused on static styling evidence, including focus visibility, target size, reduced motion, colour contrast where values were available, and removed visual cues. For JavaScript and TSX files, retrieval focused on

dynamic interaction patterns, including ARIA state management, keyboard support, live regions, form validation, visibility toggling, and accessible names.

The extension used a session-level in-memory cache for RAG responses. Cache keys were based on language identifier, retrieval query, and excerpt length. A 60-second time-to-live was used so that repeated analysis of the same unchanged file did not trigger unnecessary retrieval requests.

## C.5 Prompt Construction

The prompt was assembled from three main parts. Table C.5 summarises the prompt components used by the system.

| Prompt Component | Description |
|---|---|
| **System message** | Established the model as a senior WCAG 2.2 accessibility auditor and defined output constraints |
| **Code excerpt** | Provided a line-numbered source-code excerpt with file metadata |
| **RAG context block** | Supplied retrieved WCAG 2.2 and NASA-TLX guidance chunks |

***Table C.5:*** *Prompt Components*

The system prompt used a structured two-phase audit format. In Phase 1, the model was instructed to examine the available source-code evidence and identify relevant accessibility relationships, including elements with `id` attributes, ARIA references, accessible names, landmarks, focus behaviour, and heading hierarchy. In Phase 2, the model was instructed to report only violations that could be confirmed from the available source code and retrieved guidance.

Language-specific sweep guidance was appended where relevant. HTML guidance covered image alternatives, form labelling, headings, landmarks, ARIA references, accessible names, link purpose, and table structure. CSS guidance covered

focus indicators, touch targets, visually hidden utilities, unreduced motion, spacing, statically available colour contrast values, forced-colours support, and removed link underlines. JavaScript and TSX guidance covered ARIA state management, toggle functions, press-state updates, live region announcements, form validation propagation, initial expansion state, keyboard interaction, and visibility toggling.

### C.6 Response Handling and Deduplication

Before inference, the source file was divided into 200-line chunks using a natural-boundary strategy that scanned forward from the target cut point for a blank line after a closing brace, so that no chunk ended mid-method or mid-block. Adjacent chunks shared a 30-line overlap so that issues spanning a chunk boundary remained visible in both adjacent chunks. Inline comments () and block comments ( , <!- - - ->) were stripped from each chunk before the prompt was assembled; the stripping preserved the original line-number array so that all reported line numbers remained accurate relative to the full file. Each chunk was submitted independently to both the primary model (`kimi-k2.5`) and the secondary model (`qwen3.5:397b`), producing separate issue lists for each model per chunk.

The LLM response for each chunk was streamed and accumulated during generation. After completion, reasoning blocks enclosed in `<think>...</think>` tags were removed because they did not form part of the structured issue output.

The remaining output was split using `Issue N:` headers. Each issue block was parsed for fields including severity, line, problem, and fix. These fields were converted into structured issue objects.

After all chunks had been processed, within-model deduplication was applied separately to each model's combined output. This stage used an exact-title pass followed by a Jaccard similarity pass at a threshold of 0.40 (title-only, computed over stop-word-filtered unigrams). Findings from different chunks that described the same issue were collapsed into a single entry with merged line hints. This prevented the same issue from being counted multiple times when visible in both a primary chunk and its adjacent overlap.

The two models' deduplicated outputs were then merged using a consensus strategy. If both models reported an issue with a combined title-and-explanation Jaccard similarity of at least 0.30 (computed over the first 200 characters of each explanation) and their cited line numbers were within five lines of each other, the two reports were merged into a single finding using the primary model's wording with combined line hints. Unmatched single-model findings were additionally admitted as quality-gated survivors if they cited a specific WCAG success criterion, carried high or medium severity, and included a concrete line number; findings that failed all three gates were discarded.

Findings containing speculative or unsupported language were removed during deduplication. Table C.6 summarises examples of filtered speculative-language patterns.

| **Filtered Phrase Type** | **Example** |
|---|---|
| **Uncertainty** | "may be" |
| **Conditional evidence** | "if present" |
| **Verification failure** | "cannot verify" |
| **Weak observation** | "appears to be" |

***Table C.6:*** *Speculative-Language Filters*

A second filter was applied after the speculative-language pass. Any finding whose combined title, explanation, and fix fields contained no recognised accessibility vocabulary was discarded. This accessibility-vocabulary filter matched terms including `aria`, `role`, `focus`, `keyboard`, `wcag`, `label`, `alt`, `tabindex`, `contrast`, `accessible`, `landmark`, `heading`, `screen reader`, and related equivalents. The purpose of this filter was to remove model outputs that described implementation details or general code quality issues without any accessibility relevance, before the findings were surfaced to the developer.

The retained issues were merged with baseline findings and published to the VS Code `DiagnosticCollection`.

### C.7 Diagnostic Rendering

Parsed issues were rendered through two output mechanisms. First, issues with reliable line information were published as VS Code diagnostics so that they appeared in the Problems panel and editor view. Second, all retained issues were displayed in the Activity Bar Webview summary, including severity, description, and suggested remediation.

This dual-output design supported both immediate code-level feedback and a more explanatory summary view. It also made the tool suitable for evaluation because issues could be exported, reviewed, and compared against the ground-truth test-file registry.

# Chapter D: Prototype Reliability Verification Notes

This appendix provides context for the reliability observations and corrective actions recorded in Table 3.4.5 in Section 3.3.5.

All major pipeline components, from extension activation through to Webview rendering, functioned as expected. The two corrective actions had a direct impact on benchmarking quality. The move to section-based 128-token chunking improved retrieval precision by keeping related WCAG criteria within the same chunk rather than splitting them across retrieval boundaries. Speculative-language filters addressed uncertain phrases such as "if present" and "cannot verify" that generated findings unsupported by the source-code evidence.

Local model inference was removed after connection failures during testing. Cloud-routed inference through the Ollama relay was retained for its stability, and these changes standardised the pipeline before formal benchmarking began.

A final note on prototype scope: the public repository represents the research prototype source snapshot rather than a polished marketplace-ready VS Code package. Some features, including the baseline rule module and model-configuration panel, were implemented but not fully wired into every execution path; details are noted in Appendix C.2. These differences do not affect the evaluation results, which were produced through the LLM/RAG pipeline using the dual-model consensus configuration throughout.

# Chapter E: Supplementary Benchmarking and Validation Notes

This appendix provides supplementary evidence for the benchmarking and validation studies described in Section 3.4. It records additional information on model elimination, prompt refinement, retrieval-augmented generation changes, ensemble testing, computational NASA-TLX refinement, real-world validation materials, and supplementary tool-comparison procedures.

The detailed benchmarking tables and graphs removed from the main methodology chapter are placed in this appendix so that Chapter 3 remains readable while still preserving the full audit trail for the experimental evaluation. Appendix E therefore acts as the supporting record for Studies 1 to 6, including interpreted performance findings for Studies 1 to 4, while Chapter 4 presents the interpreted findings and discussion for Studies 5 and 6.

The model-screening and condition-comparison studies were executed through terminal-based benchmarking scripts rather than manual interaction with the Visual Studio Code interface. This allowed the same test files, prompts, RAG settings, inference-preset definitions, supported inference parameters, and scoring procedures to be applied consistently across repeated model runs. The VS Code extension remained the developer-facing artefact, while the controlled model comparisons were carried out programmatically to support reproducibility.

## E.1 Study 1 Supplementary Notes

Study 1 involved broad model screening on the `html-high` test file. During this stage, the RAG and prompt pipeline were refined alongside model elimination. The aim was to identify models that were technically stable enough for later evaluation and capable of detecting known accessibility issues without excessive hallucination or repeated failure.

Table E.1 summarises the prompt and RAG refinements made during Study 1.

| Refinement | Reason |
|---|---|
| **Character-based chunking replaced with section-based 128-token chunking** | Original chunks were too broad and retrieved unrelated guidance |
| **`top_k` reduced from 6 to 3** | Reduced irrelevant retrieved content |
| **Cosine distance threshold introduced** | Filtered off-topic retrieval chunks |
| **Knowledge base expanded with rule files** | Improved retrieval coverage for images, forms, controls, ARIA, navigation, and tables |
| **Targeted sweep queries introduced** | Improved retrieval specificity for HTML accessibility patterns |

***Table E.1:*** *Study 1 Prompt and RAG Refinements*

Key failure patterns included persistent cloud relay failures, excessive false positives, detection avoidance, instability across repeated runs, and over-ranking of fast but inaccurate models by the early composite score. Local model testing was also explored during this stage, but local execution was not retained for the main benchmarking pipeline because of hardware and stability limitations.

### *E.1.1 Study 1 Model Elimination Record*

Table E.1.1 presents the Study 1 model elimination record.

| Test | Model | Elimination Reason | Approx. Avg F1 |
|---|---|---|---|
| **T1** | All local models, e.g. `llama3`, `mistral-local` variants | Local deployment produced lower composite scores and required hardware resources unsuitable for a lightweight VS Code extension. Cloud relay through the Ollama API was adopted for later tests. | ¡30% |
| **Pre-T9** | `minimax-m2`, `minimax-m2.5` | HTTP 500 errors occurred on every no-RAG call due to cloud relay infrastructure failure. | N/A |
| **T12** | `gemma3:4b` | Average F1 = 13.6% across three consecutive tests with extreme instability. | 13.6% |
| **T12** | `nemotron-3-nano:30b` | Average F1 = 17.3%, with no recovery trend. | 17.3% |
| **T12** | `devstral-2:123b` | Average F1 = 18.7%, with flat performance across tests. | 18.7% |
| **T14** | `ministral-3:3b` | Chronic hallucination pattern and below-median composite performance. | ¡20% |
| **T14** | `qwen3-coder-next` | Consistently below-median composite score with no improvement trajectory. | ¡20% |
| **T14** | `gpt-oss:20b` | Hallucinated element identifiers and was outperformed by `gpt-oss:120b`. | ¡25% |
| **T15** | `ministral-3:14b` | Produced 129 false positives across three runs, averaging 43 false positives per run. | High FP |
| **T15** | `devstral-small-2:24b` | Complete RAG collapse, with F1 = 7.5% in both RAG conditions. | 7.5% |
| **T15** | `gemma3:27b` | Average F1 = 16.2%, with no measurable improvement across four consecutive tests. | 16.2% |

***Table E.1.1:*** *Study 1 Model Elimination Record*

### *E.1.2 Study 1 Terminal and Graph Evidence*

Figure E.1.2a shows the benchmarking script configuration for the 20-model screening run, including the test file, preset, scoring formula, and run timestamp. Figure E.1.2b presents the composite score ranking for all 20 candidate models, reporting TP, FP, FN, F1, precision, recall, and average response time per model. Figure E.1.2c shows the false-positive analysis identifying models that hallucinated accessibility issues on clean test files where no real problems exist.

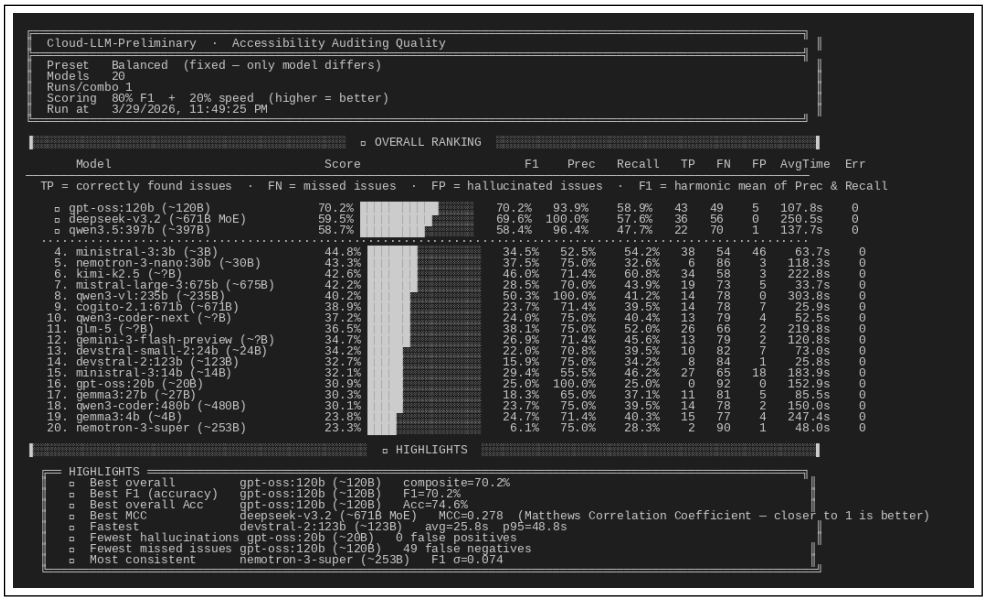



***Figure E.1.2a:*** *Study 1 Terminal-Based Model Screening Evidence*

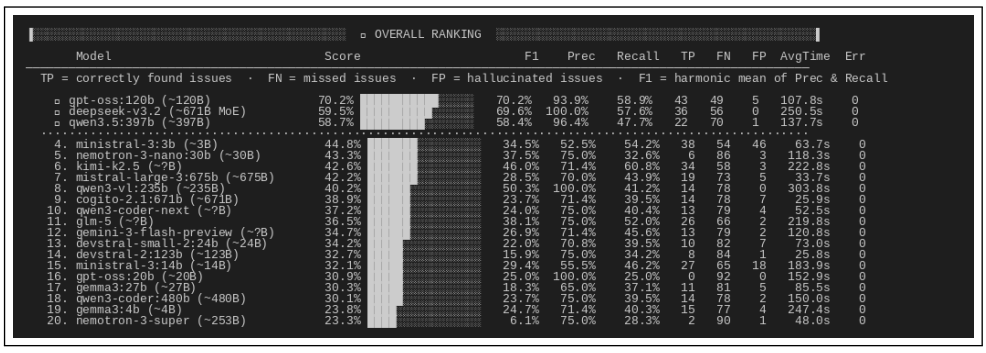



***Figure E.1.2b:*** *Study 1 Candidate Model Screening Results*

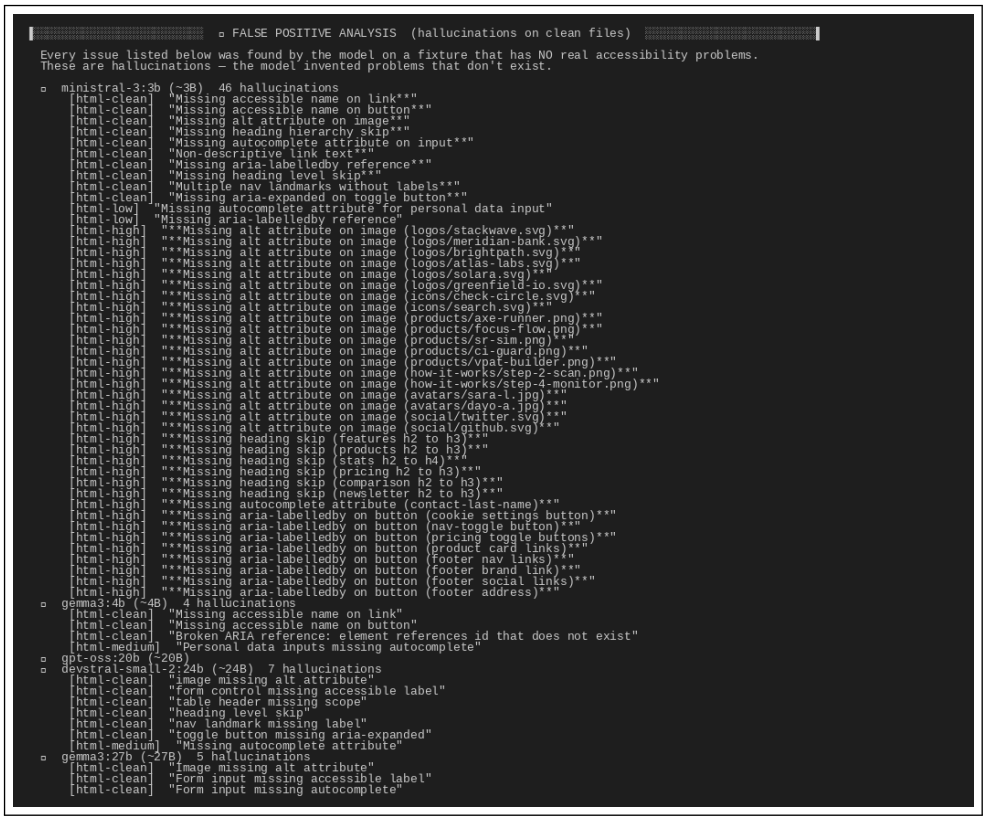



***Figure E.1.2c:*** *Study 1 False-Positive and Instability Pattern Summary*

## E.2 Study 2 Supplementary Notes

Study 2 introduced multi-file testing and model-specific parameters. The expansion to CSS, JavaScript, and TypeScript/React TSX showed that generic HTML-focused prompting was insufficient for other file types. As a result, language-specific guidance was added to better reflect the different accessibility evidence available in markup, styling, scripting, and component-based files.

Table E.2a summarises the language-specific sweep coverage introduced during Study 2.

| Sweep Group | Coverage |
|---|---|
| **CSS-A to CSS-H** | Focus removal, touch target sizing, visually hidden utilities, motion, spacing, contrast, forced-colours support, and link underlines |
| **JS-A to JS-G** | ARIA state management, toggle functions, press-state updates, live regions, form validation, initial expansion state, and visibility management |
| **TSX-specific guidance** | Component-level accessible names, ARIA relationships, state-driven visibility, keyboard interaction, and form validation behaviour |

***Table E.2a:*** *Study 2 Language-Specific Sweep Coverage*

Multi-run aggregation was introduced because single-run model output varied across repeated runs. The union rule counted a ground-truth issue as detected if it appeared in at least one of three runs. This reduced the effect of stochastic variation in model output and helped prevent a model from being removed solely because of a single weak run.

### *E.2.1 Study 2 Model Elimination Record*

Table E.2.1a presents the Study 2 model elimination record.

| Test | Model | Elimination Reason |
|---|---|---|
| **T16** | `cogito-2.1:671b` | F1 = 24.1%; systematic detection avoidance, predicting no issues on most inputs. |
| **T16** | `qwen3-vl:235b` | F1 = 25.5%; vision-language architecture performed poorly on plain text source code. |
| **T16** | `qwen3-coder:480b` | F1 = 20.9%; continued decline across tests with no recovery. |
| **T25 to T28** | `glm-5` | Sustained below-threshold performance across all four file types. |
| **T25 to T28** | `mistral-large-3:675b` | Sustained below-threshold performance, with JS F1 ceiling around 15%. |
| **T30** | `gemini-3-flash-preview` | 1/4 conditions passing; JS F1 near zero due to static-analysis limitations. |
| **T30** | `gemma4:31b trial` | 0/4 conditions passing on first and only test. |
| **T30 to T33** | `deepseek-v3.2` | JS F1 ceiling around 9%; token budget problems in no-think mode. |

***Table E.2.1a:*** *Study 2 Model Elimination Record*

Table E.2.1b summarises the final Study 2 shortlist standing.

| Model | Conditions Passing, T34, 16 Test Files | Best Single-Condition F1 | Notes |
|---|---|---|---|
| **`kimi-k2.5`** | 4 / 4 | 70%+ `norag-think` | Most consistent across file types and density levels |
| **`qwen3.5:397b`** | 4 / 4 | Comparable to Kimi | Stronger on CSS and TSX test files |
| **`gpt-oss:120b`** | 3 / 4 | 61.6% `rag-nothink`, `html-high`, Study 1 | Failed `rag-think` narrowly at 79.9% |

***Table E.2.1b:*** *Study 2 Final Shortlist Standing*

### *E.2.2 Study 2 Terminal and Graph Evidence*

Figure E.2.2a shows the Study 2 benchmarking configuration with three short-listed models evaluated across three runs per condition. Figure E.2.2b presents the composite score ranking for the three models under the `rag-think` condition. Figure E.2.2c shows the overall ranking stacked across all four experimental conditions (`rag-think`, `rag-nothink`, `norag-think`, `norag-nothink`), illustrating how each model performed under different RAG and reasoning-mode configurations. Figure E.2.2d presents the three-run consistency data, showing F1 standard deviation and the minimum and maximum F1 achieved per model.

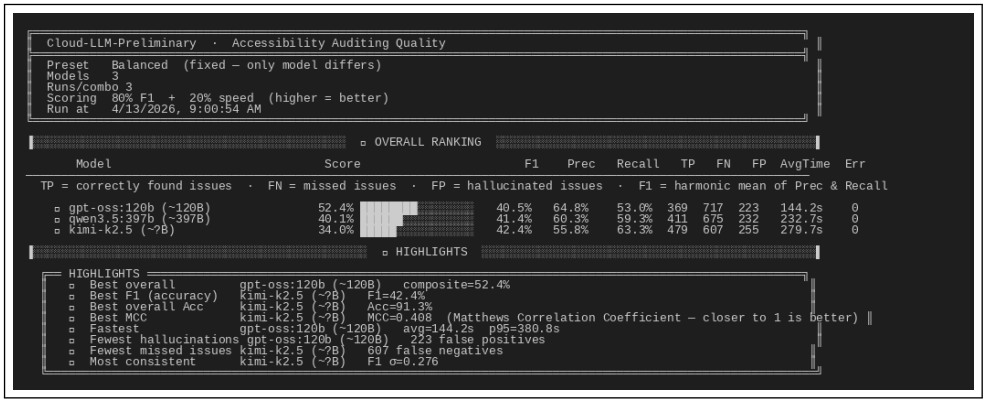



***Figure E.2.2a:*** *Study 2 Terminal-Based Multi-File Evaluation Evidence*

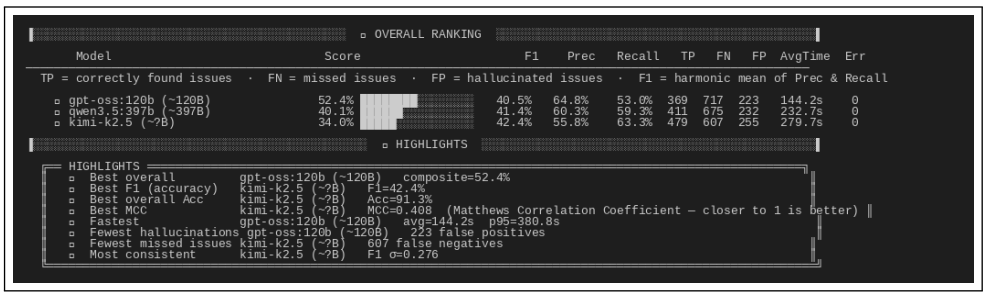



***Figure E.2.2b:*** *Study 2 Multi-File Performance by Model*

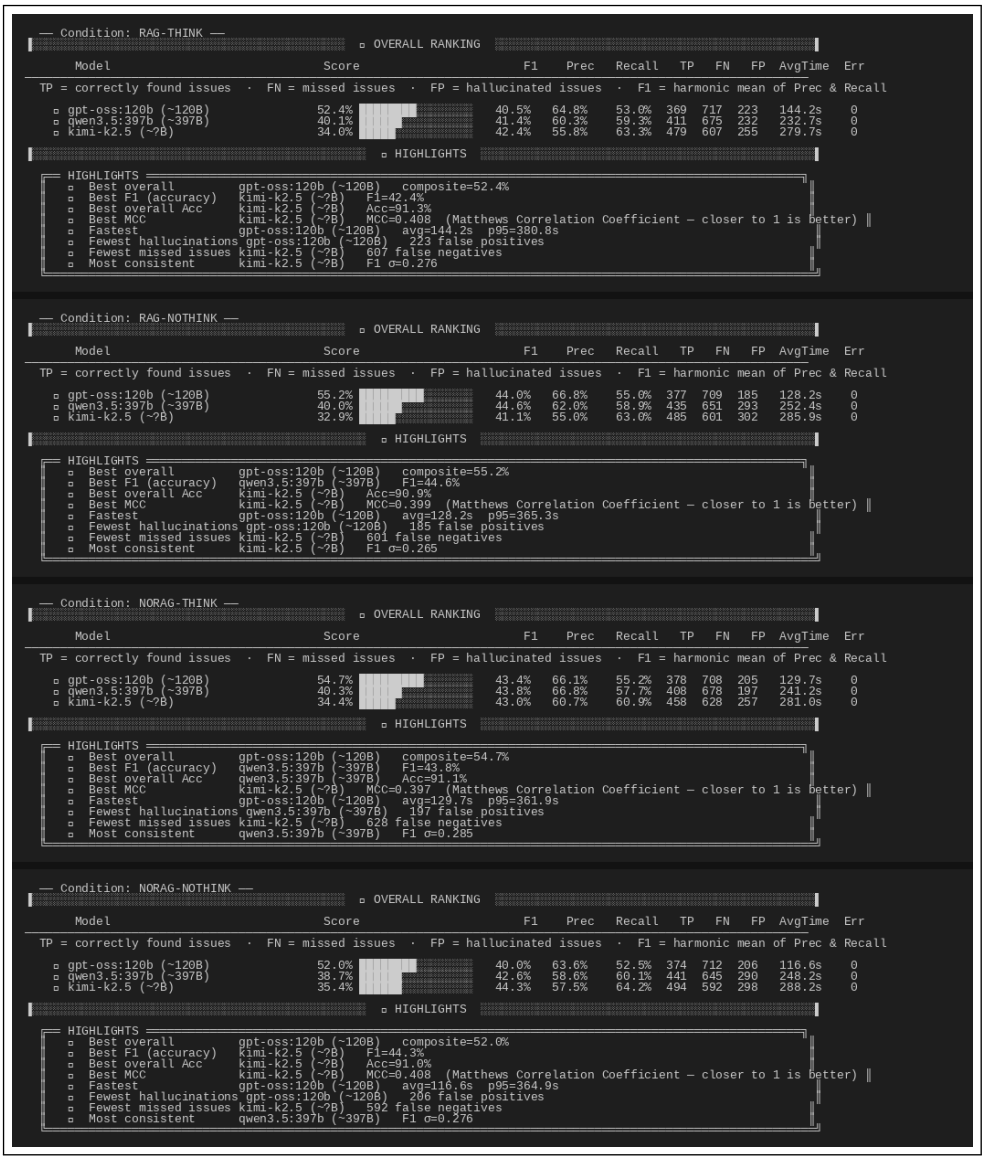



***Figure E.2.2c:*** *Study 2 Condition-Level Performance Across File Types*

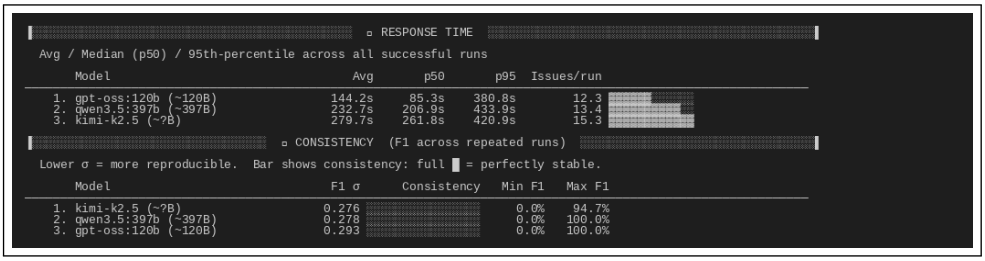



***Figure E.2.2d:*** *Study 2 Three-Run Aggregation Comparison*

### E.3 Study 3 Supplementary Notes

Study 3 evaluated advanced configurations after the candidate pool had been narrowed. The 128-token RAG chunk size was retained because it produced the strongest `rag-think` performance and the lowest false-positive count. The Kimi-Qwen ensemble was selected because the two models showed complementary strengths: Kimi performed strongly on structural HTML and ARIA reference issues, while Qwen performed better on CSS visual issues and TSX component issues.

The triple ensemble was not retained because it increased false positives. Under majority voting, additional models can improve recall, but they can also amplify hallucinations when incorrect issues are accepted by any two of the three models. This made the dual-model Kimi-Qwen consensus configuration more suitable for the final production configuration than the three-model alternative.

The cross-model voting mechanism used in the final Kimi-Qwen configuration paired findings using two criteria: line numbers within five lines of each other (waived if either model cited no line number), and a title-and-explanation Jaccard score of at least 0.30. The 0.30 threshold was computed over the combined title and explanation tokens rather than title tokens alone, so that issues described with different headline words but sharing substantive explanation vocabulary could still be matched. The within-model deduplication stage used a title-only Jaccard threshold of 0.40, applied separately to each model's cross-chunk findings before the cross-model comparison. Pairs passing both criteria were merged into a single agreed finding. Unmatched single-model findings were additionally admitted as quality-gated survivors if they cited a specific WCAG success criterion, carried

high or medium severity, and included a concrete line number; findings that failed all three gates were discarded.

### *E.3.1 Study 3 Ensemble Configuration Comparison*

Table E.3.1 summarises the Study 3 ensemble configuration comparison.

| Test | Configuration | Avg F1, All Conditions, 16 Test files | Condition Variance | Key Finding |
|---|---|---|---|---|
| **T39** | `kimi-k2.5` + `gpt-oss:120b`, `norag-nothink` only | 56.8% | N/A | First ensemble test; +10.7pp over best individual model in same condition |
| **T40** | `kimi-k2.5` + `gpt-oss:120b`, dual, all conditions | 69.0% | ±2.4pp | First full dual-model result |
| **T41** | `kimi-k2.5` + `gpt-oss:120b` + `qwen3.5:397b`, triple | 66.0% | ±7.3pp | Worse than dual; third model introduced false-positive noise |
| **T42** | `kimi-k2.5` + `qwen3.5:397b`, dual | 70.05% | ±1.8pp | Best result at the time; Qwen was a stronger partner for Kimi than GPT |
| **T47** | `kimi-k2.5` + `qwen3.5:397b`, final validation | 70.075% | Tight | `rag-think` best: F1 = 70.5%, FP = 424 |

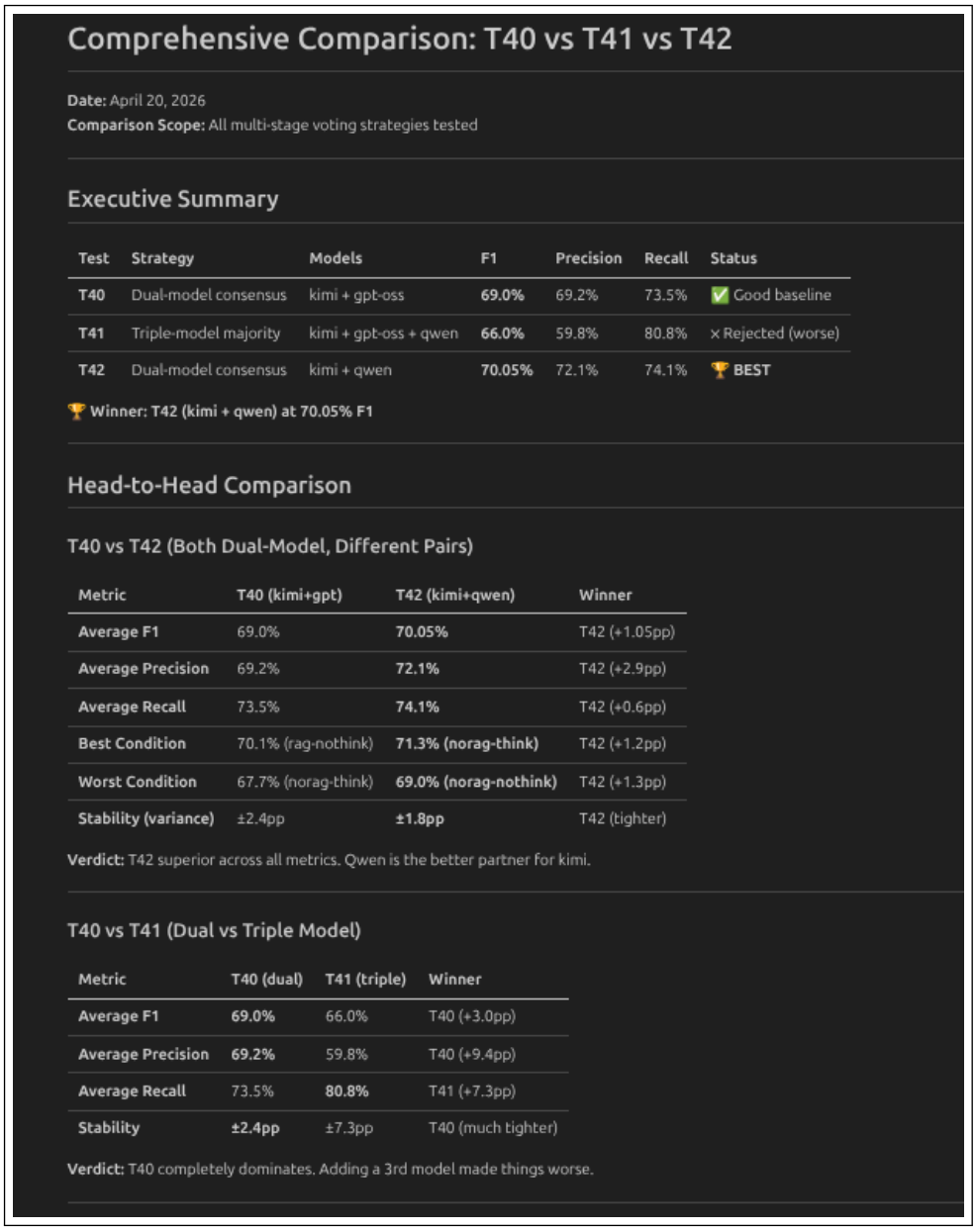
***Table E.3.1:*** *Study 3 Ensemble Configuration Comparison*

### *E.3.2 Study 3 Terminal and Graph Evidence*

Figure E.3.2a shows the Study 3 benchmarking configuration and overall ranking for the dual-model kimi-k2.5 and gpt-oss:120b ensemble (T40). Figure E.3.2b presents the comprehensive T40 vs T41 vs T42 comparison, including the executive summary of average F1, precision, and recall, head-to-head metrics between configurations, and the progression table from T37 to T42. Figure E.3.2c shows F1, precision, and recall broken down across all four experimental conditions for each ensemble configuration, illustrating the variance differences between dual-model and triple-model approaches. Figure E.3.2d shows the false-positive analysis for the T40 dual-model configuration (465 hallucinations) and the T41 triple-model configuration (829 hallucinations), illustrating the precision cost of adding a third model under majority voting.




***Figure E.3.2a:*** *Study 3 Terminal-Based Ensemble Testing Evidence*



## Comprehensive Comparison: T40 vs T41 vs T42

**Date:** April 20, 2026
**Comparison Scope:** All multi-stage voting strategies tested

### Executive Summary

| Test | Strategy | Models | F1 | Precision | Recall | Status |
|---|---|---|---|---|---|---|
| **T40** | Dual-model consensus | kimi + gpt-oss | **69.0%** | 69.2% | 73.5% | ✅ Good baseline |
| **T41** | Triple-model majority | kimi + gpt-oss + qwen | **66.0%** | 59.8% | 80.8% | × Rejected (worse) |
| **T42** | Dual-model consensus | kimi + qwen | **70.05%** | 72.1% | 74.1% | 🏆 **BEST** |

🏆 **Winner: T42 (kimi + qwen) at 70.05% F1**

### Head-to-Head Comparison

#### T40 vs T42 (Both Dual-Model, Different Pairs)

| Metric | T40 (kimi+gpt) | T42 (kimi+qwen) | Winner |
|---|---|---|---|
| **Average F1** | 69.0% | **70.05%** | T42 (+1.05pp) |
| **Average Precision** | 69.2% | **72.1%** | T42 (+2.9pp) |
| **Average Recall** | 73.5% | **74.1%** | T42 (+0.6pp) |
| **Best Condition** | 70.1% (rag-nothink) | **71.3% (norag-think)** | T42 (+1.2pp) |
| **Worst Condition** | 67.7% (norag-think) | **69.0% (norag-nothink)** | T42 (+1.3pp) |
| **Stability (variance)** | ±2.4pp | **±1.8pp** | T42 (tighter) |

**Verdict:** T42 superior across all metrics. Qwen is the better partner for kimi.

#### T40 vs T41 (Dual vs Triple Model)

| Metric | T40 (dual) | T41 (triple) | Winner |
|---|---|---|---|
| **Average F1** | **69.0%** | 66.0% | T40 (+3.0pp) |
| **Average Precision** | **69.2%** | 59.8% | T40 (+9.4pp) |
| **Average Recall** | 73.5% | **80.8%** | T41 (+7.3pp) |
| **Stability** | **±2.4pp** | ±7.3pp | T40 (much tighter) |

**Verdict:** T40 completely dominates. Adding a 3rd model made things worse.

***Figure E.3.2b:*** *Study 3 Dual-Model and Triple-Model F1 Comparison*

## Results by Condition (All Three Tests)

### rag-nothink

| Test | F1 | Precision | Recall |
|---|---|---|---|
| T40 | 70.1% | 71.2% | 73.4% |
| T41 | 65.8% | 59.4% | 80.4% |
| **T42** | **69.6%** | **72.0%** | **72.9%** |

Winner: **T42** (best precision, comparable to T40 F1)

### rag-think

| Test | F1 | Precision | Recall |
|---|---|---|---|
| T40 | 69.4% | 69.3% | 73.8% |
| T41 | 66.9% | 60.3% | 80.9% |
| **T42** | **70.3%** | **73.1%** | **74.6%** |

Winner: **T42** (best F1 and precision)

### norag-nothink

| Test | F1 | Precision | Recall |
|---|---|---|---|
| T40 | 68.5% | 67.8% | 73.1% |
| T41 | 65.5% | 60.1% | 80.7% |
| **T42** | **69.0%** | **71.4%** | **73.3%** |

Winner: **T42** (best across all metrics)

### norag-think

| Test | F1 | Precision | Recall |
|---|---|---|---|
| T40 | 67.7% | 68.5% | 73.6% |
| T41 | 65.7% | 59.3% | 82.1% |
| **T42** | **71.3%** | **71.8%** | **75.6%** |

Winner: **T42** (significantly higher F1 and precision)

***Figure E.3.2c:*** *Study 3 Condition Variance Across Ensemble Configurations*

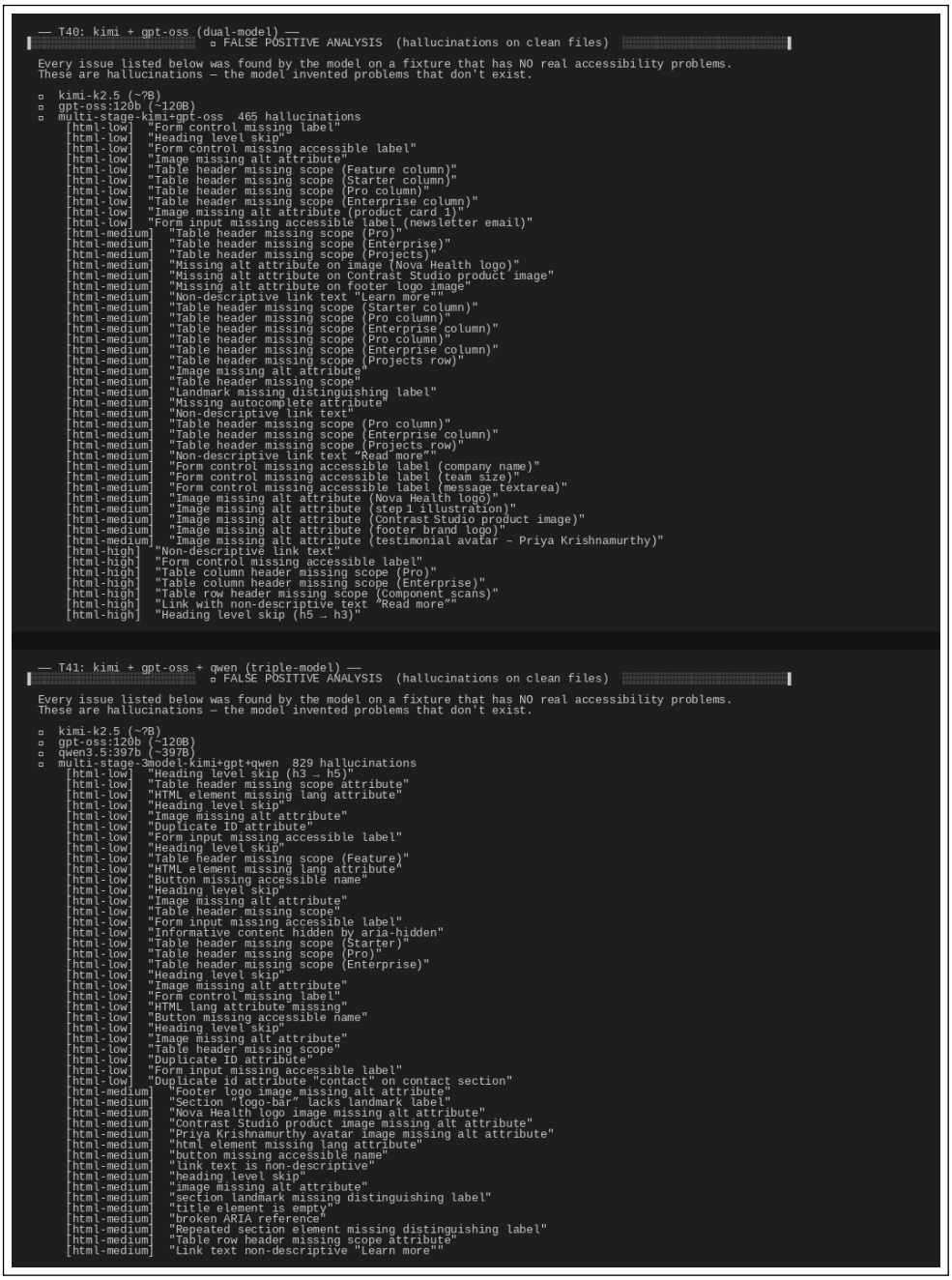



***Figure E.3.2d:*** *Study 3 False-Positive Comparison Across Ensemble Configurations*

## E.4 Study 4 Supplementary Notes

Study 4 refined the computational NASA-TLX prediction pipeline. This study focused on whether the system's predicted workload scores responded appropriately to issue count, severity, test-file difficulty, language type, and experimental condition before the real-world user validation study.

Table E.4 summarises the TLX refinements made during Study 4.

| **Refinement** | **Purpose** |
|---|---|
| **Severity weighting** | Increased contribution of critical and high-severity issues |
| **Extended WCAG matching** | Improved mapping between detected issues and TLX dimensions |
| **Raised subscale caps** | Allowed high-density test files to produce higher workload predictions |
| **Language difficulty modifier** | Reflected observed difficulty differences between HTML, CSS, JavaScript, and TSX |
| **JavaScript knowledge-base expansion** | Improved RAG support for dynamic interaction patterns |

***Table E.4:*** *Study 4 TLX Refinements*

The JavaScript RAG deficit remained a boundary condition. This suggests that static WCAG prose retrieval was less effective for dynamic JavaScript interaction patterns than the models' pre-training in some no-RAG conditions.

### *E.4.1 TLX Test 1 Ensemble Comparison Summary*

Table E.4.1 summarises TLX Test 1 ensemble comparison results.

| Configuration | Avg F1 | Avg Precision | Avg Recall | Avg Predicted TLX |
|---|---|---|---|---|
| **`kimi-k2.5` individual** | 0.49 | 0.83 | 0.36 | 44 |
| **`gpt-oss:120b` individual** | 0.43 | 0.80 | 0.32 | 46 |
| **`qwen3.5:397b` individual** | 0.41 | 0.83 | 0.30 | 43 |
| **Kimi-Qwen voting** | 0.55 | 0.59 | 0.54 | 65 |
| **Kimi-GPT voting** | 0.53 | 0.58 | 0.52 | 61 |
| **3-model voting** | 0.53 | 0.48 | 0.60 | 67 |

***Table E.4.1:*** *TLX Test 1 Ensemble Comparison Summary*

### *E.4.2 TLX Tests 1 to 3 Across Conditions*

Table E.4.2 summarises TLX Tests 1 to 3 for Kimi-Qwen voting across conditions.

| Condition | TLX T1 F1 | TLX T2 F1 | TLX T3 F1 | T2 TLX | T3 TLX |
|---|---|---|---|---|---|
| **`rag-think`** | 0.610 | 0.574 | 0.522 | 80 | 78 |
| **`rag-nothink`** | N/A | 0.560 | 0.536 | 78 | 78 |
| **`norag-think`** | N/A | 0.523 | 0.575 | 78 | 79 |
| **`norag-nothink`** | N/A | 0.474 | 0.551 | 73 | 78 |
| **Overall, Kimi-Qwen** | 0.55 | 0.533 | 0.546 | 77 | 78 |

***Table E.4.2:*** *TLX Tests 1 to 3, Kimi-Qwen Voting Across Conditions*

### *E.4.3 Study 4 Terminal and Graph Evidence*

Figure E.4.3a shows the Study 4 test harness configuration, identifying the voting mode, test file set, four experimental conditions, and the RAG and inference endpoints used. Figure E.4.3b presents per-test file results for the `rag-think` condition, including individual kimi and qwen F1 scores alongside the voted ensemble F1 and the predicted NASA-TLX workload scores with subscale breakdowns. Figure E.4.3c shows predicted TLX scores across all four conditions (`rag-think`, `rag-nothink`, `norag-think`, `norag-nothink`), including per-test file F1 and TLX values. Figure E.4.3d presents the TLX Test 3 refinement summary, documenting what changed from Test 2, the resulting overall F1 and TLX improvements, and the per-condition breakdown of changes.

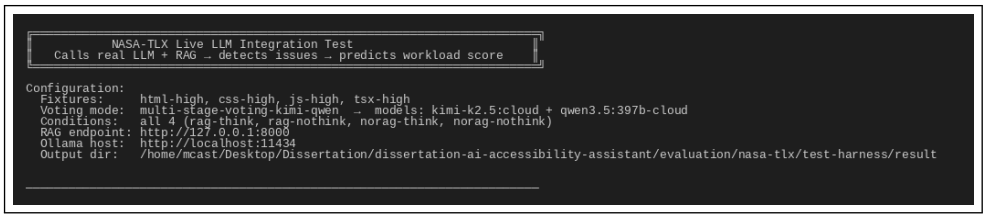



***Figure E.4.3a:*** *Study 4 Terminal-Based Computational NASA-TLX Testing Evidence*

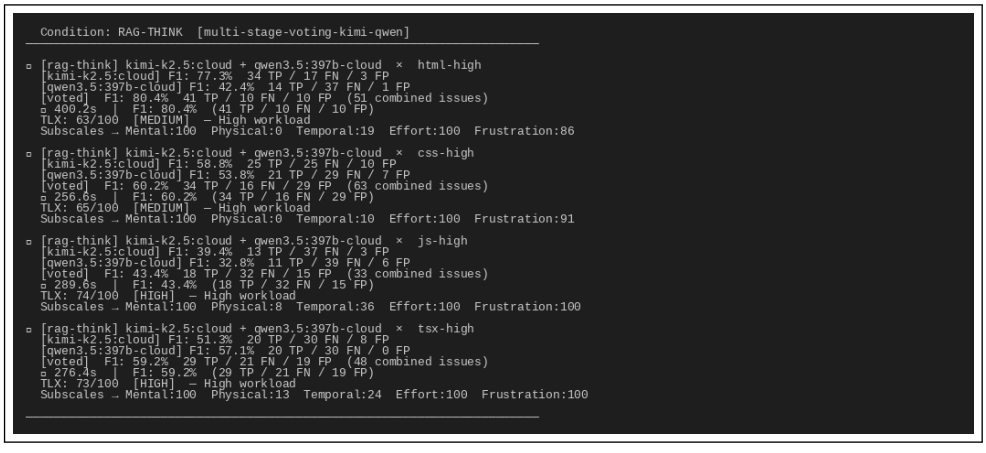



***Figure E.4.3b:*** *Study 4 Predicted TLX by Model Configuration*




***Figure E.4.3c:*** *Study 4 Predicted TLX Across Experimental Conditions*

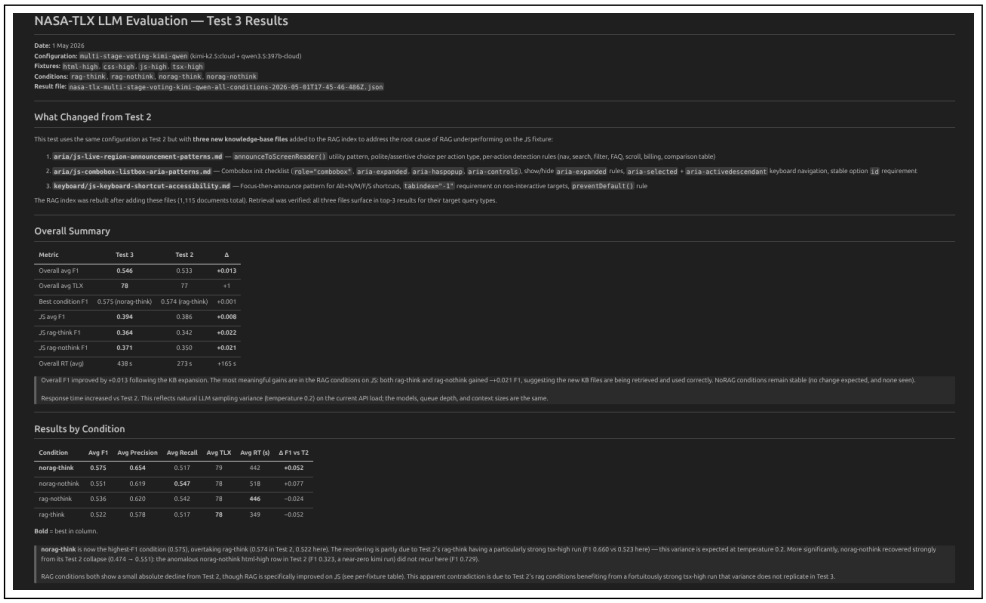



***Figure E.4.3d:*** *Study 4 TLX Refinement Impact Across Test Runs*

## E.5 Study 5 Supporting Details

Study 5 used two versions of the same website: the original defective version and the AI-remediated improved version. This validation study connected the benchmarked AI assistant to practical accessibility remediation and participant-facing workload evaluation.

Table E.5a summarises the Study 5 supporting materials.

| Material | Purpose |
|---|---|
| **Defective website source code** | Used as the real-world validation artefact |
| **Improved website source code** | Used to test whether AI-guided remediation reduced barriers |
| **AI issue list** | Provided the system's generated findings |
| **Expert review checklist** | Used to assess precision, recall gaps, and fix quality |
| **User task list** | Used to guide participant interaction with both page versions |
| **NASA-TLX questionnaire** | Used to collect subjective workload ratings |

***Table E.5a:*** *Study 5 Supporting Materials*

Table E.5b summarises the Study 5 user task types.

| Task Type | Purpose |
|---|---|
| **Complete a contact form** | Tests form labelling, instructions, and error support |
| **Navigate to specified content using keyboard controls** | Tests keyboard access and focus visibility |
| **Identify the purpose of images** | Tests alternative text quality |
| **Locate information using headings or links** | Tests structure, link purpose, and navigation clarity |

***Table E.5b:*** *Study 5 User Task Types*

Table E.5c summarises the NASA-TLX mapping used in Study 5.

| NASA-TLX Dimension | Example Accessibility Barrier Expected to Affect It |
|---|---|
| **Mental Demand** | Broken ARIA references, unclear labels, poor structure |
| **Physical Demand** | Keyboard navigation barriers and repeated interaction effort |
| **Temporal Demand** | Difficulty locating content or completing form tasks |
| **Performance** | Participant perception of task success or failure |
| **Effort** | Extra steps required to interpret or navigate the page |
| **Frustration** | Confusing labels, failed interactions, or inaccessible controls |

***Table E.5c:*** *NASA-TLX Mapping Used in Study 5*

## E.6 Study 6 Supporting Notes

Study 6 aligned findings from the AI Accessibility Assistant and established automated accessibility tools by issue category rather than exact wording. This was necessary because the tools analysed different representations of the same accessibility problem. Lighthouse and axe-core primarily evaluated rendered browser output, while Axe Linter, ESLint jsx-a11y, Webhint, stylelint-a11y, and the AI Accessibility Assistant operated closer to source-code or development-workflow contexts.

Table E.6a summarises the issue-alignment categories used in Study 6.

| Finding Type | Interpretation | Follow-up Action |
|---|---|---|
| **Reported by multiple tools** | High-confidence accessibility violation | Recorded as corroborated issue |
| **Reported by AI only** | Possible semantic issue or hallucination | Passed to expert review |
| **Reported by Lighthouse or axe-core only** | Possible AI recall gap | Recorded for comparison |
| **Reported by none but found by experts** | Missed by all automated methods | Recorded as manual-review gap |

***Table E.6a:*** *Study 6 Issue Alignment Categories*

Table E.6b summarises the supplementary tool comparison scope.

| Tool | Primary Analysis Target | Main Comparison Purpose |
|---|---|---|
| **AI Accessibility Assistant** | HTML, CSS, JavaScript, and TSX source files | Source-level semantic analysis and developer-facing remediation guidance |
| **Lighthouse** | Rendered browser DOM | Browser audit and page-level accessibility scoring |
| **axe-core** | Rendered DOM/rule engine | Rule-based accessibility issue detection |
| **Axe Linter** | Source code/editor workflow | Source-level linting comparison |
| **ESLint jsx-a11y** | JSX/TSX source code | React accessibility linting comparison |
| **Webhint** | Source, configuration, and browser hints | Configurable web quality and accessibility comparison |
| **stylelint-a11y** | CSS source code | CSS accessibility linting comparison |

***Table E.6b:*** *Supplementary Tool Comparison Scope*

Table E.6c summarises the architectural differences between tool types.

| Difference | Lighthouse / axe-core | AI Accessibility Assistant |
|---|---|---|
| **Input representation** | Rendered browser DOM | Source code files |
| **Runtime awareness** | Can inspect rendered page state | Limited to static source evidence |
| **Workflow position** | Usually after page is runnable | During coding |
| **Analysis style** | Rule-based pattern matching | LLM-based contextual reasoning |
| **Output purpose** | Audit result | Developer-facing explanation and remediation guidance |

***Table E.6c:*** *Architectural Differences Between Tool Types*

### *E.6.1 Study 6 Discussion Detail*

**AI Performance Compared to Prior Tool Studies.** The AI assistant's GT Recall of 50% on JavaScript and 57% on TSX substantially exceeds published baselines for leading automated tools (20–36% against WCAG 2.x defect sets [58]). Multi-tool comparison studies confirm limited single-tool coverage [26–28]. The comparison must be contextualised: the test files were designed to target issue types (ARIA state management, dynamic announcements, component relationships) where rule-based tools cannot compete, so the AI's advantage is artefact-type-dependent. On HTML, GT Recall is slightly ahead of axe-core and Axe Linter (20% vs 17%). The aggregate AI Accuracy of 83.5% is well above precision figures from recent LLM-based accessibility tool studies [37–39]. The aggregate F1 of 0.468 characterises overall system performance across the full multi-artefact test.

**Deterministic Tool Performance in Context.** The zero-recall result from Lighthouse, axe-core, and Axe Linter on JavaScript and TSX is consistent with their design scope: these tools inspect browser-rendered accessibility trees and cannot analyse JavaScript behavioural logic or TSX component source code without a browser runtime. Axe Linter's zero JavaScript and TSX recall reflects that the planted defects targeted ARIA state transitions, live region management, and focus logic, none of which are covered by axe-core's static source rules. The axe-core and Axe Linter HTML results (GT-TP=5, Accuracy=0.833, GT-Recall=0.167) show these tools are competitive with the AI on HTML GT Recall (17% vs 20%) because several planted landmark and ARIA-role issues fell within their rule sets.

**Practical Implications.** Despite the AI assistant's advantages on JavaScript and

TSX, supplementary tools contributed complementary value. Lighthouse achieved 100% Accuracy on HTML (3/3 genuine, 0 FP). axe-core and Axe Linter achieved 17% GT Recall on HTML with 83% Accuracy. ESLint jsx-a11y and Axe Linter each achieved 1/1 perfect Accuracy on TSX. stylelint-a11y is the only tool that detected any CSS GT issues. The recommended layered pipeline: use the AI assistant as the primary source-level analyser for all artefact types (especially JavaScript and TSX); apply Axe Linter for HTML/JSX, ESLint jsx-a11y for JSX/React, and stylelint-a11y for CSS; run Lighthouse and axe-core for rendered-DOM auditing. Findings should be consolidated, de-duplicated, and reviewed by a practitioner before treating as confirmed violations.

**Threats to Validity (Study 6).** Several threats should be acknowledged: (1) test files were synthetic and intentionally error-seeded: real-world issue prevalence may differ; (2) raw finding counts may include repeated or overlapping issues; (3) keyword-based GT matching is conservative and may miss valid findings phrased differently; (4) Lighthouse and axe-core were skipped for TSX (no rendered browser runtime), affecting cross-tool comparability; (5) the dual-model consensus depends on both models producing valid structured output; (6) the zero shared-finding rate increases complementary coverage but increases reconciliation effort in practitioner workflows.

# Chapter F: Evaluation Metric and NASA-TLX Mapping Notes

## F.1 Keyword-Match Ground-Truth Scoring

Each accessibility violation in the benchmark corpus was stored as an `IssueConcept`. An `IssueConcept` contained a unique identifier, a short description, and one or more lower-case keywords. During evaluation, the model response was converted to lower case and checked against the keyword list for each expected issue.

This approach was used because LLMs may describe the same accessibility issue using different wording. For example, a missing form label may be described as “input has no label”, “unlabelled form field”, or “missing accessible name”. Keyword matching allowed these variations to be recognised without requiring exact string equality.

The limitation is that a response may contain a correct keyword while still giving an incomplete or weak explanation. For this reason, Study 5 included expert developer review to assess whether findings and remediations were practically valid.

## F.2 Metric Interpretation

The evaluation used classification metrics to assess whether the system detected known accessibility violations while avoiding unsupported findings. F1 score was useful because it balances precision and recall, while MCC was included because it accounts for true positives, false positives, true negatives, and false negatives, making it more suitable when classes are unevenly distributed [51].

Table F.2 summarises how each metric was interpreted in this study.

| **Metric** | **Interpretation in this study** |
|---|---|
| **Precision** | Indicates how many reported issues were correct |
| **Recall** | Indicates how many ground-truth issues were detected |
| **F1 Score** | Balances false positives and false negatives |
| **Accuracy** | Indicates the proportion of correct classification decisions |
| **MCC** | Indicates balanced performance across TP, FP, TN, and FN |
| **Latency** | Indicates whether output is practical for developer workflow |
| **Fix Quality** | Indicates whether recommendations are correct and actionable |

***Table F.2:*** *Metric Interpretation Summary*

## F.3 Original NASA-TLX Rating Scale Definitions

NASA-TLX was selected because it provides a structured subjective workload assessment framework. The original NASA documentation describes NASA-TLX as a multidimensional rating procedure that produces an overall workload score using six subscales: Mental Demand, Physical Demand, Temporal Demand, Performance, Effort, and Frustration [12, 13].

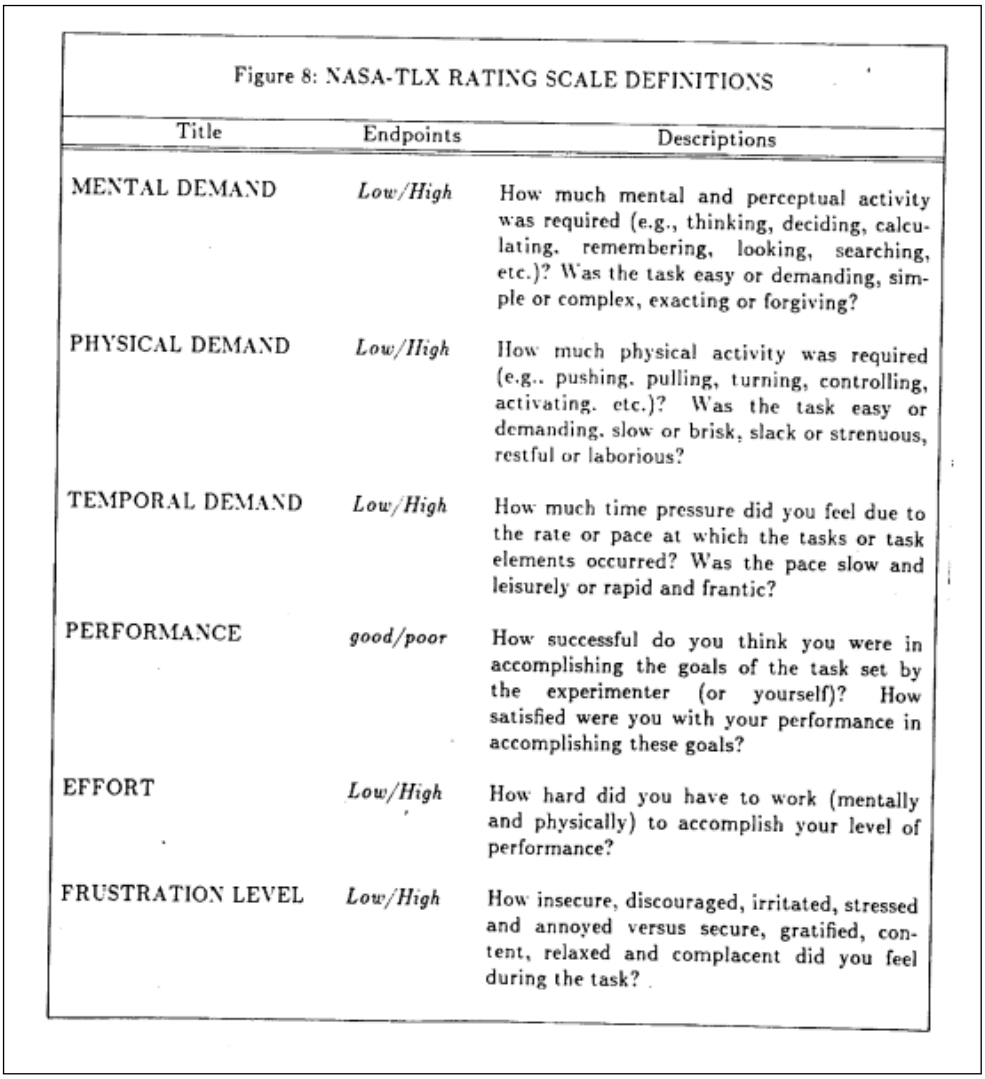


Figure 8: NASA-TLX RATING SCALE DEFINITIONS

| Title | Endpoints | Descriptions |
|---|---|---|
| MENTAL DEMAND | *Low/High* | How much mental and perceptual activity was required (e.g., thinking, deciding, calculating, remembering, looking, searching, etc.)? Was the task easy or demanding, simple or complex, exacting or forgiving? |
| PHYSICAL DEMAND | *Low/High* | How much physical activity was required (e.g., pushing, pulling, turning, controlling, activating, etc.)? Was the task easy or demanding, slow or brisk, slack or strenuous, restful or laborious? |
| TEMPORAL DEMAND | *Low/High* | How much time pressure did you feel due to the rate or pace at which the tasks or task elements occurred? Was the pace slow and leisurely or rapid and frantic? |
| PERFORMANCE | *good/poor* | How successful do you think you were in accomplishing the goals of the task set by the experimenter (or yourself)? How satisfied were you with your performance in accomplishing these goals? |
| EFFORT | *Low/High* | How hard did you have to work (mentally and physically) to accomplish your level of performance? |
| FRUSTRATION LEVEL | *Low/High* | How insecure, discouraged, irritated, stressed and annoyed versus secure, gratified, content, relaxed and complacent did you feel during the task? |

***Figure F.3:*** *Original NASA-TLX Rating Scale Definitions*

*Source:* NASA Task Load Index documentation [12, 13].

Figure F.3 shows the original NASA-TLX rating scale definitions for Mental Demand, Physical Demand, Temporal Demand, Performance, Effort, and Frustration. These dimensions were used in this study as the basis for both the computational workload prediction model and the participant self-report questionnaire used in Study 5.

## F.4 NASA-TLX Barrier Mapping

The computational NASA-TLX prediction model mapped detected accessibility barriers to workload dimensions as a heuristic proxy rather than a clinical measurement. NASA-TLX was selected for its structured framework across Mental Demand, Physical Demand, Temporal Demand, Performance, Effort, and Frustration [12, 13].

The mapping connected source-code issues to likely workload consequences. Unclear labels and broken ARIA relationships were mapped to Mental Demand; keyboard access barriers to Physical Demand, Effort, and Frustration.

Table F.4 summarises the barrier-to-NASA-TLX mapping.

| **Accessibility Barrier Type** | **NASA-TLX Dimensions Affected** | **Rationale** |
|---|---|---|
| **Missing or unclear labels** | Mental Demand, Frustration | Users must infer meaning or recover from uncertainty |
| **Missing image alternatives** | Mental Demand, Performance | Users may miss content or task-relevant information |
| **Keyboard access barriers** | Physical Demand, Effort, Frustration | Users may require repeated attempts or alternative navigation |
| **Poor focus visibility** | Effort, Frustration | Users may struggle to track current position |
| **Broken ARIA references** | Mental Demand, Performance | Assistive technology output may become confusing or incomplete |
| **Missing live region announcements** | Mental Demand, Temporal Demand | Users may not notice updates and may spend time checking state |
| **Error messages without clear association** | Mental Demand, Frustration, Performance | Users may not understand how to correct input errors |
| **Excessive motion without reduction support** | Mental Demand, Frustration | Users may experience distraction or discomfort |

***Table F.4:*** *Barrier-to-NASA-TLX Mapping*

## F.5 Expert Review Coding

Expert developer review was used to evaluate whether the system's findings and applied fixes were practically valid. Review comments were coded using the following scheme. This allowed expert feedback to be connected to precision, recall-gap identification, and remediation quality.

Table F.5 summarises the expert review coding scheme.

| Code | Meaning | Use in Analysis |
|---|---|---|
| **Confirmed genuine barrier** | The reviewer agreed that the issue was a real WCAG-related accessibility problem | Supports precision assessment |
| **Confirmed false positive** | The reviewer judged that the issue was not a genuine barrier | Identifies over-reporting |
| **Confirmed missing issue** | The reviewer identified a significant issue not reported by the system | Supports recall-gap analysis |
| **Confirmed appropriate remediation** | The reviewer judged that the applied fix resolved the issue | Supports fix quality assessment |
| **Remediation concern** | The reviewer identified a problem or side effect in an applied fix | Identifies fix-quality limitations |

***Table F.5:*** *Expert Review Coding Scheme*

# Chapter G: Participant Materials and Ethical Safeguards

This appendix summarises the participant-facing materials and ethical safeguards used during Study 5. These details support the ethical procedures described in Section 3.6.

## G.1 Participant Information Materials

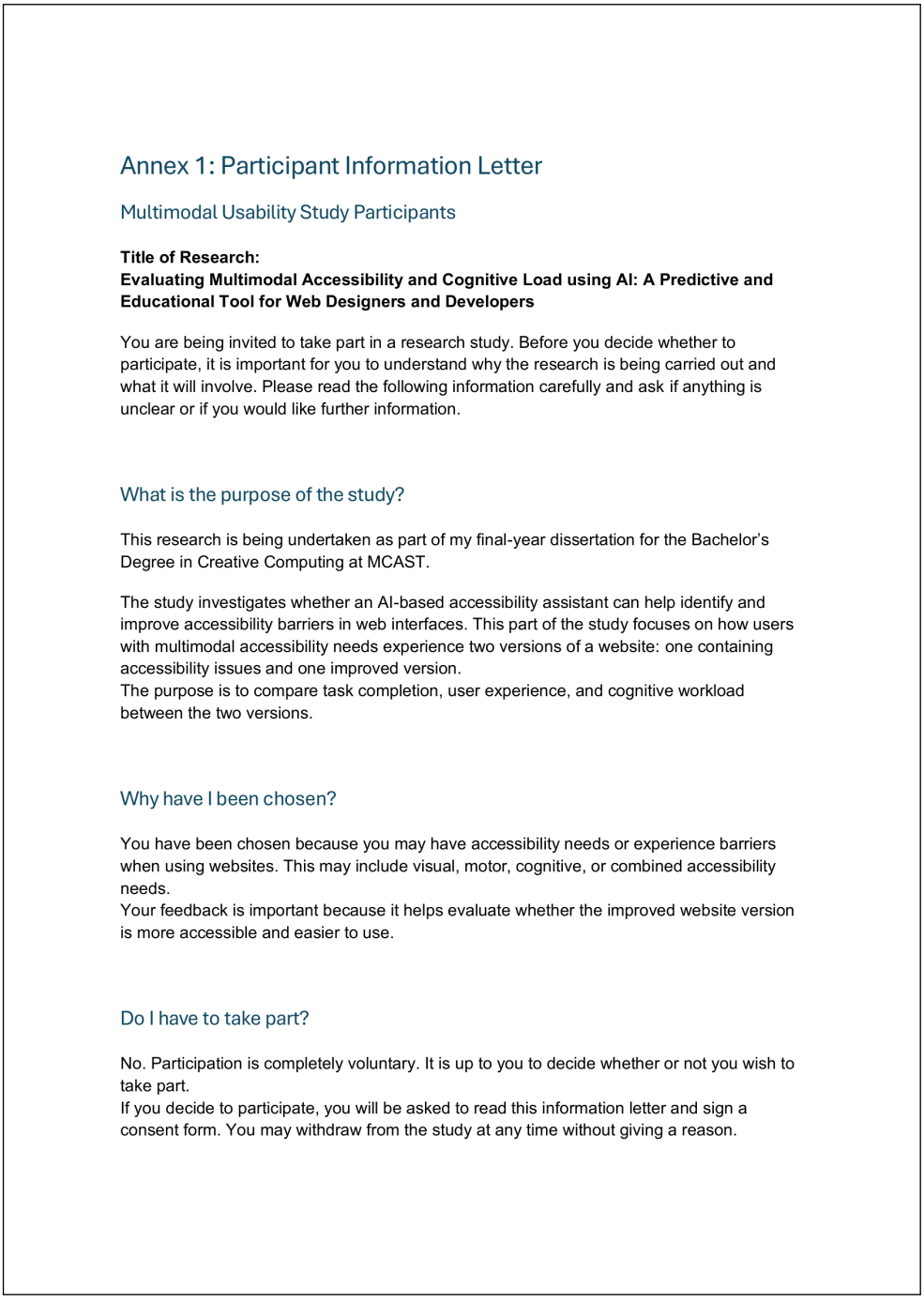
Participants received an information sheet before taking part in the study. The document explained the study purpose, the tasks involved, the type of data collected, the voluntary nature of participation, and the way responses would be stored and reported.

Table G.1 summarises the coverage of the participant information sheet.

| **Information Provided** | **Purpose** |
|---|---|
| **Study purpose** | Explained that the research evaluated an AI-assisted accessibility tool |
| **Task description** | Explained that participants would interact with two versions of a website |
| **Time commitment** | Informed participants of the expected session duration |
| **Data collected** | Explained that task feedback and NASA-TLX workload responses would be collected |
| **Voluntary participation** | Confirmed that participation was optional |
| **Right to withdraw** | Confirmed that participants could stop at any time without giving a reason |
| **Data handling** | Explained how responses would be stored, anonymised, and reported |
| **AI limitations** | Explained that AI-generated recommendations may be incorrect or incomplete |

***Table G.1:*** *Participant Information Sheet Coverage*

The participant information letters used in the study are presented below in blank form.



## Annex 1: Participant Information Letter

### Multimodal Usability Study Participants

**Title of Research:**
**Evaluating Multimodal Accessibility and Cognitive Load using AI: A Predictive and Educational Tool for Web Designers and Developers**

You are being invited to take part in a research study. Before you decide whether to participate, it is important for you to understand why the research is being carried out and what it will involve. Please read the following information carefully and ask if anything is unclear or if you would like further information.

### What is the purpose of the study?

This research is being undertaken as part of my final-year dissertation for the Bachelor's Degree in Creative Computing at MCAST.

The study investigates whether an AI-based accessibility assistant can help identify and improve accessibility barriers in web interfaces. This part of the study focuses on how users with multimodal accessibility needs experience two versions of a website: one containing accessibility issues and one improved version.
The purpose is to compare task completion, user experience, and cognitive workload between the two versions.

### Why have I been chosen?

You have been chosen because you may have accessibility needs or experience barriers when using websites. This may include visual, motor, cognitive, or combined accessibility needs.
Your feedback is important because it helps evaluate whether the improved website version is more accessible and easier to use.

### Do I have to take part?

No. Participation is completely voluntary. It is up to you to decide whether or not you wish to take part.
If you decide to participate, you will be asked to read this information letter and sign a consent form. You may withdraw from the study at any time without giving a reason.

***Figure G.1a:*** *Participant Information Letter for Accessibility Participants, Page 1*

### What will happen to me if I take part?

You will be asked to use two versions of a website:

- an original version containing accessibility issues;
- an improved version based on AI-generated accessibility recommendations.

For each version, you will complete a set of website tasks designed to evaluate navigation, form use, information retrieval, keyboard interaction, and cognitive workload. After each version, you will complete a NASA Task Load Index questionnaire. This asks you to rate your experience from 0 to 100 across areas such as mental demand, physical demand, time pressure, performance, effort, and frustration.

You may also be asked to provide optional comments about your experience.

### What are the possible disadvantages and risks of taking part?

There are no major risks expected. However, some tasks may feel slightly frustrating or difficult because one website version intentionally contains accessibility barriers.
You may stop at any time if you feel uncomfortable, tired, or do not wish to continue.

### What are the possible benefits of taking part?

You may not benefit directly from taking part. However, your feedback will help support research into more accessible websites and AI-assisted accessibility tools.
Your contribution may help developers and designers better understand how inaccessible web design affects real users.

### What if something goes wrong?

If you wish to complain or have any concerns about the way you have been approached or treated during this study, please contact:

**Researcher:** Mireya Francesca Vella
**Institution:** MCAST
**Programme:** Bachelor's Degree in Creative Computing
**Research Mentor/Supervisor:** Paul Pulis
**Email:** mireya.vella.i85114@mcast.edu.mt

***Figure G.1b:*** *Participant Information Letter for Accessibility Participants, Page 2*

### Will my details be kept confidential?

All information collected during this study will be kept confidential. Only the researcher will have access to the collected responses.

Your signed consent form will be stored separately from your task results and NASA-TLX responses. The results will only be linked to your participant code.

Any results included in the dissertation will be anonymised. This means that your name or identifying details will not be published.

By agreeing to participate, you are consenting to the retention and anonymised use of your responses for academic research purposes.

### What will happen to the results of the research study?

The results will be analysed and written up as part of my final dissertation for the Bachelor's Degree in Creative Computing at MCAST.

The findings may be used to compare the original and improved website versions in terms of accessibility, task completion, and cognitive workload.

### Who is organising the research?

The research is being conducted by **Mireya Francesca Vella** as part of the **Bachelor's Degree in Creative Computing at MCAST**, under the supervision of **Paul Pulis**.

### Who may I contact for further information?

If you would like more information about the research before deciding whether to participate, please contact:

**Researcher:** Mireya Francesca Vella
**Email:** mireya.vella.i85114@mcast.edu.mt

Thank you for your interest in this research.

***Figure G.1c:*** *Participant Information Letter for Accessibility Participants, Page 3*

## Annex 1: Participant Information Letter

**Title of Research: Evaluating Multimodal Accessibility and Cognitive Load using AI: A Predictive and Educational Tool for Web Designers and Developers**

You are being invited to take part in a research study. Before you decide whether to participate, it is important for you to understand why the research is being carried out and what it will involve. Please read the following information carefully and ask if anything is unclear or if you would like further information.

### What is the purpose of the study?

This research is being undertaken as part of my final-year dissertation for the Bachelor's Degree in Creative Computing at MCAST. The study focuses on evaluating an AI Accessibility Assistant designed to identify web accessibility issues and suggest improvements based on WCAG 2.2 guidelines.

The purpose of this specific study is to gather expert developer feedback on whether the issues identified by the AI are genuine accessibility violations and whether the suggested fixes are successful.

### Why have I been chosen?

You have been chosen because you have experience in web development, software development, accessibility, or related digital technologies. Your professional judgement is valuable for assessing the accuracy and usefulness of the AI assistant's accessibility feedback.

### Do I have to take part?

No. Participation is completely voluntary. It is up to you to decide whether or not you wish to take part. If you decide to participate, you will be asked to review this information sheet and complete a corresponding consent form.

You may withdraw from the study at any time without giving a reason.

***Figure G.1d:*** *Participant Information Letter for Developer Reviewers, Page 1*

### What will happen to me if I take part?

You will be asked to review a set of accessibility issues identified by the AI assistant. For each issue, you will compare the original website version with the improved version and answer whether:

- the issue is a genuine WCAG 2.2 accessibility violation;
- the fix applied in the improved version was successful;
- there are any important accessibility issues that the AI did not identify.

Your responses will be collected through a review table, questionnaire, or Google Form.

### What are the possible disadvantages and risks of taking part?

There are no disadvantages or risks foreseen in taking part in this study. The task only involves reviewing website examples and providing professional feedback.

### What are the possible benefits of taking part?

You may not benefit directly from taking part. However, your contribution will support research into AI-assisted accessibility evaluation and may help improve how developers identify and fix accessibility issues in web interfaces.

### What if something goes wrong?

If you wish to complain or have any concerns about any aspect of the way you have been approached or treated during this study, please contact:

**Researcher:** Mireya Francesca Vella
**Institution:** MCAST
**Programme:** Bachelor's Degree in Creative Computing
**Research Mentor/Supervisor:** Paul Pulis
**Email:** mireya.vella.i85114@mcast.edu.mt

***Figure G.1e:*** *Participant Information Letter for Developer Reviewers, Page 2*

### Will my details be kept confidential?

All information collected during this study will be kept strictly confidential. Only the researcher carrying out the study will have access to the collected responses.

Any data used in the dissertation will be presented in an anonymised form. No personally identifiable information will be published or shared. By agreeing to participate, you are consenting to the retention and anonymised use of your responses for academic research purposes.

### What will happen to the results of the research study?

The results will be analysed and written up as part of my final dissertation for the Bachelor's Degree in Creative Computing at MCAST. Findings may be used to discuss the accuracy, usefulness, and limitations of the AI Accessibility Assistant.

### Who is organising the research?

The research is being conducted by **Mireya Francesca Vella** as part of the **Bachelor's Degree in Creative Computing at MCAST**, under the supervision of **Paul Pulis**.

### Who may I contact for further information?

If you would like more information about the research before deciding whether to participate, please contact:

**Researcher:** Mireya Francesca Vella
**Institution:** MCAST
**Email:** mireya.vella.i85114@mcast.edu.mt

Thank you for your interest in this research.

***Figure G.1f:*** *Participant Information Letter for Developer Reviewers, Page 3*

## G.2 Consent Procedure

Written informed consent was obtained before any task began. Participants confirmed that they understood the study purpose, the tasks involved, the type of data collected, and their right to withdraw. Separate consent forms were used for professional developer reviewers and participants with accessibility needs, as the two groups contributed different forms of evidence.

Consent forms were stored separately from study responses to protect participant confidentiality. Signed consent forms, names, email addresses, signatures, and identifiable responses should not be included in the dissertation appendix. Only blank templates or anonymised screenshots should be included as evidence.

## G.3 Professional Developer Reviewer Consent Form

This section presents the blank consent form used for professional developer reviewers. Signed forms and identifiable reviewer details were excluded from the appendix to protect participant confidentiality.

## Annex 2: Consent Form

**Title of Research: Evaluating Multimodal Accessibility and Cognitive Load using AI: A Predictive and Educational Tool for Web Designers and Developers**

**Researcher:** Mireya Francesca Vella
**Institution:** MCAST
**Programme:** Bachelor's Degree in Creative Computing
**Research Mentor/Supervisor:** Paul Pulis
**Researcher Email:** mireya.vella.i85114@mcast.edu.mt

### Consent Statement

Please read each statement carefully and tick each box to confirm your consent.

| Statement | Tick |
|---|---|
| I confirm that I have read and understood the Participant Information Letter for this study. | ☐ |
| I have had the opportunity to ask questions about the study and receive clear answers. | ☐ |
| I understand that my participation is voluntary. | ☐ |
| I understand that I may withdraw from the study at any time without giving a reason. | ☐ |
| I understand that I will be asked to review AI-identified accessibility issues and assess whether they are genuine WCAG 2.2 violations. | ☐ |
| I understand that I will be asked to compare original and improved website versions and judge whether the fixes were successful. | ☐ |
| I understand that my responses will be kept confidential and used only for academic research purposes. | ☐ |
| I understand that any results included in the dissertation will be anonymised and will not identify me personally. | ☐ |
| I voluntarily agree to participate in this research study as an expert reviewer. | ☐ |

### Participant Details

**Participant Name:** _____

**Participant ID:** _____

**Participant Signature:** _____

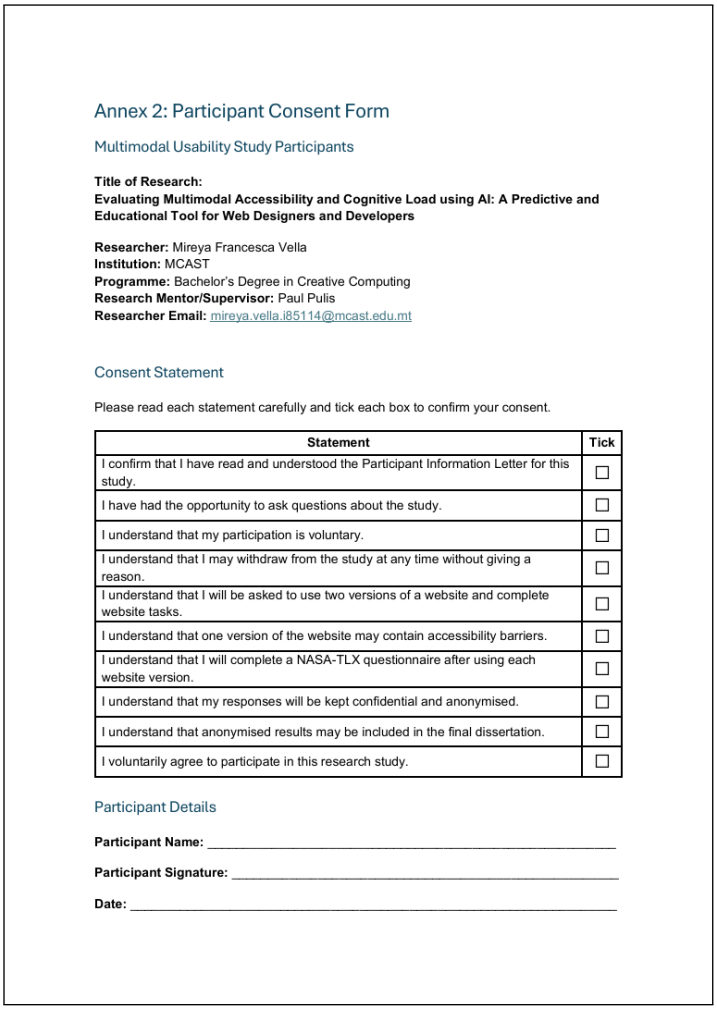
**Date:** _____

***Figure G.3:*** *Professional Developer Reviewer Consent Form*

The developer reviewer consent form confirmed that the reviewer understood the purpose of the study, the review task, the type of feedback requested, voluntary participation, the right to withdraw, and how their responses would be anonymised and reported.

## G.4 Accessibility Participant Consent Form

This section presents the blank consent form for participants with accessibility needs. Signed forms and identifiable details were excluded to protect participant confidentiality.



Annex 2: Participant Consent Form

Multimodal Usability Study Participants

**Title of Research:**
**Evaluating Multimodal Accessibility and Cognitive Load using AI: A Predictive and Educational Tool for Web Designers and Developers**

**Researcher:** Mireya Francesca Vella
**Institution:** MCAST
**Programme:** Bachelor's Degree in Creative Computing
**Research Mentor/Supervisor:** Paul Pulis
**Researcher Email:** mireya.vella.i85114@mcast.edu.mt

Consent Statement

Please read each statement carefully and tick each box to confirm your consent.

| Statement | Tick |
|---|---|
| I confirm that I have read and understood the Participant Information Letter for this study. | ☐ |
| I have had the opportunity to ask questions about the study. | ☐ |
| I understand that my participation is voluntary. | ☐ |
| I understand that I may withdraw from the study at any time without giving a reason. | ☐ |
| I understand that I will be asked to use two versions of a website and complete website tasks. | ☐ |
| I understand that one version of the website may contain accessibility barriers. | ☐ |
| I understand that I will complete a NASA-TLX questionnaire after using each website version. | ☐ |
| I understand that my responses will be kept confidential and anonymised. | ☐ |
| I understand that anonymised results may be included in the final dissertation. | ☐ |
| I voluntarily agree to participate in this research study. | ☐ |

Participant Details

**Participant Name:** _____

**Participant Signature:** _____

**Date:** _____

***Figure G.4a:*** *Accessibility Participant Consent Form, Page 1*

The participant consent form confirmed that the participant understood the purpose of the study, the website tasks, the NASA-TLX questionnaire, voluntary participation, the right to withdraw, data protection measures, and the fact that the website and AI tool were being evaluated rather than the participant's ability.

## G.5 Accessibility of Research Materials

All participant-facing materials were made available in accessible formats. This included the participant information sheet, consent form, task instructions, and NASA-TLX questionnaire. Screen-reader-compatible digital copies were provided where needed, and verbal explanation or individual assistance was available where required.

For participants with visual accessibility needs, testing was designed to allow use of their preferred device settings, assistive technologies, and navigation methods. A short setup or practice task was used where necessary to confirm that the participant could access the materials before beginning the main tasks.

## G.6 User Testing Task Sheet

This section presents the task sheet used during the multimodal usability study. The task sheet recorded participant code, date, website version, task completion, whether help was needed, and optional notes.

Annex 3: Multimodal Usability Study Results Form

**Participant Code:** _____
**Date:** _____
**Website Version:** ☐ Original / Defective ☐ Improved
**Order:** Defecitve first, Improved after

Task Completion

| Task No. | Task Description | Completed? | Needed Help? | Notes |
|---|---|---|---|---|
| 1 | Scroll down the page to find the **Pricing** section and identify the monthly price and user limit of the **Pro** plan. | ☐ Yes ☐ No ☐ Partially | ☐ Yes ☐ No | |
| 2 | Near the top of the page, find the **Contact Us** form, fill in all required fields (**email address, full name, and message**), and attempt to submit it. | ☐ Yes ☐ No ☐ Partially | ☐ Yes ☐ No | |
| 3 | Using the footer **"Jump to section"** navigation, go to **Meet the Team** and identify the name and role of the **Chief Technology Officer**. | ☐ Yes ☐ No ☐ Partially | ☐ Yes ☐ No | |
| 4 | Using the footer **"Jump to section"** navigation, go to the **FAQ** section, open one item, and read its answer. | ☐ Yes ☐ No ☐ Partially | ☐ Yes ☐ No | |
| 5 | Scroll through the page to find the **Blog** section and read the title and description of the article related to **accessibility** or **low-vision users**. | ☐ Yes ☐ No ☐ Partially | ☐ Yes ☐ No | |
| 6 | Using only the keyboard (**Tab** and **Enter/Space**), navigate to and activate the **"Get Started Today"** button at the top of the page. | ☐ Yes ☐ No ☐ Partially | ☐ Yes ☐ No | |
| 7 | Using the footer **"Jump to section"** navigation, go to **Testimonials** and advance to the next testimonial using the carousel buttons. | ☐ Yes ☐ No ☐ Partially | ☐ Yes ☐ No | |

***Figure G.6:*** *User Testing Task Sheet*

The task sheet included the tasks completed on both the defective website and the improved website. The tasks were presented using a controlled order and avoided language that suggested which version was expected to perform better.

Table G.6 summarises the user testing task overview.

| Task Type | Purpose |
|---|---|
| **Complete a contact form** | Tests form labelling, instructions, and error support |
| **Navigate to specified content using keyboard controls** | Tests keyboard access and focus visibility |
| **Identify the purpose of images** | Tests alternative text quality |
| **Locate information using headings or links** | Tests structure, link purpose, and navigation clarity |

***Table G.6:*** *User Testing Task Overview*

## G.7 NASA-TLX Questionnaire Used in Study 5

This section presents the adapted NASA-TLX questionnaire used with participants after completing the website tasks. The questionnaire asked participants to rate Mental Demand, Physical Demand, Temporal Demand, Performance, Effort, and Frustration from 0 to 100.

NASA-TLX Questionnaire

Please rate each item from 0 to 100

Use a score from **0 to 100**, where **0 means very low** and **100 means very high**.

| NASA-TLX Category | Question | Score /100 |
|---|---|---|
| **Mental Demand** | How mentally demanding was this task? | ____ / 100 |
| **Physical Demand** | How physically demanding was this task? | ____ / 100 |
| **Temporal Demand** | How hurried or rushed was the pace of the task? | ____ / 100 |
| **Performance** | How successful were you in accomplishing what you were asked to do? | ____ / 100 |
| **Effort** | How hard did you have to work to accomplish your level of performance? | ____ / 100 |
| **Frustration** | How insecure, discouraged, irritated, stressed, or annoyed did you feel? | ____ / 100 |

Optional Comments

Was anything difficult, confusing, inaccessible, or frustrating?

_____

_____

_____

_____

***Figure G.7:*** *Adapted NASA-TLX Questionnaire*

The questionnaire included the six NASA-TLX workload dimensions used after each website condition: Mental Demand, Physical Demand, Temporal Demand, Performance, Effort, and Frustration. This appendix section records the participant-facing version of the instrument, while Appendix F.3 records the original NASA-TLX scale definitions.

Table G.7 summarises the NASA-TLX dimensions used in the participant questionnaire.

| NASA-TLX Dimension | Participant-Facing Meaning |
|---|---|
| **Mental Demand** | How mentally demanding the task felt |
| **Physical Demand** | How physically demanding the interaction felt |
| **Temporal Demand** | How much time pressure the participant felt |
| **Performance** | How successful the participant felt when completing the task |
| **Effort** | How hard the participant had to work to complete the task |
| **Frustration** | How insecure, discouraged, irritated, stressed, or annoyed the participant felt |

***Table G.7:*** *NASA-TLX Dimensions Used in the Participant Questionnaire*

## G.8 Data Protection Measures

Table G.8 summarises the data protection measures applied during Study 5.

| Measure | Implementation |
|---|---|
| **Pseudonymisation** | Participants were assigned numeric codes |
| **Data minimisation** | Only task responses and workload ratings required for the study were collected |
| **Separate consent storage** | Consent forms were stored separately from response data |
| **Secure storage** | Data were stored on an encrypted local device |
| **Access control** | Data were accessible only to the researcher and dissertation supervisor |
| **Reporting approach** | Results were reported in aggregated and pseudonymised form |
| **Medical privacy** | No diagnosis or clinical details were collected |

***Table G.8:*** *Data Protection Measures*

## G.9 Risk Mitigation Measures

Table G.9 summarises the risk mitigation measures applied during participant-facing testing.

| Risk | Mitigation |
|---|---|
| **Fatigue from screen interaction** | Short sessions and rest breaks |
| **Frustration with inaccessible content** | Participants were warned in advance and could stop at any time |
| **Confusion about study purpose** | Participants were told the website and tool were being evaluated, not their ability |
| **Assistive technology difficulties** | A practice task confirmed setup before the main tasks |
| **Privacy concerns** | No identifiable data were included in analysis or reporting |

***Table G.9:*** *Risk Mitigation Measures*

## G.10 Evidence of Participant Materials

The following participant-facing materials were included as appendix evidence in blank or anonymised form.

Table G.10 summarises the participant material evidence included in the appendix.

| Material | Appendix Evidence Included |
|---|---|
| **Developer reviewer consent form** | Blank template only |
| **Accessibility participant consent form** | Blank template only |
| **Participant information sheet** | Blank or anonymised copy |
| **User testing task sheet** | Blank or anonymised copy |
| **NASA-TLX questionnaire** | Blank questionnaire template |
| **Testing screenshots** | Anonymised screenshots showing the structure of the forms or testing materials |

***Table G.10:*** *Participant Material Evidence*